

Projeto vapor

**vapor: computação certificada, do termo
algébrico ao registrador — arquitetura,
inovações, finanças verificáveis e uma bancada
aberta**

São Paulo

2026

Projeto vapor

**vapor: computação certificada, do termo algébrico ao
registrador — arquitetura, inovações, finanças verificáveis
e uma bancada aberta**

Monografia que descreve a arquitetura, as inovações e a avaliação do sistema vapor — um compilador e runtime de tensores certificados, escrito em Elixir sem dependências, e as camadas construídas sobre ele, com ênfase nas rodadas 0.13 (finanças e mesa de operações) e 0.14 (a bancada aberta).

Monografia técnica do projeto vapor (versão 0.14.0)

São Paulo

2026

RESUMO

O *vapor* é um compilador e runtime de tensores escrito em Elixir sem nenhuma dependência externa, que trata a reprodutibilidade numérica e a verificabilidade como requisitos de projeto, e não como propriedades desejáveis. Programas são termos de uma álgebra simbólica; o plano de controle, na máquina virtual do Erlang (BEAM), os reescreve só por identidades exatas, aloca registradores com um algoritmo cujo resultado é conferido por um verificador provado no assistente de provas Lean e extraído para Elixir, e emite diretamente os bits de máquina de cinco alvos — x86-64 (AVX2 e AVX-512), AArch64 NEON, RISC-V com a extensão vetorial e SPIR-V — sem assembler, sem C e sem LLVM. O código gerado nunca executa dentro da máquina virtual: roda em processos isolados que podem falhar sem derrubar nada. Cada compilação atravessa uma escada de verificação de seis degraus e sai com um certificado Ed25519 determinístico, co-assinável. A garantia central é que, na política canônica, todo substrato produz os mesmos bits. Sobre esse núcleo, catorze rodadas construíram um ecossistema: modelos de linguagem de fronteira, imagem, áudio, documentos, agentes cuja execução é um objeto de prova e, nas últimas rodadas, mesas de resolução de problemas em que uma busca propõe e um verificador decide. Esta monografia descreve a arquitetura e as inovações — o que são, para quem servem, para que, por que e como — e aprofunda a rodada 0.13, que aplica o princípio a finanças: calendários e curvas iguais à biblioteca de referência QuantLib, Monte Carlo compilado com o gerador dentro da álgebra e bits idênticos ao oráculo exato, backtests julgados por quatro portões de ruído (entre eles um certificado de ausência de antecipação por invariância de prefixo), arbitragem decidida exatamente pelo lema de Farkas e um livro de ofertas cujo diário encadeado por hash é julgado por um motor ingênuo independente — que encontrou um defeito real. A rodada 0.14 troca as mesas por categoria por uma *bancada aberta*: o usuário, ou um modelo de linguagem, escreve qualquer problema numa linguagem pura e saneada; uma fornalha de estratégias o busca contra a busca aleatória com o mesmo orçamento; e uma pedra de toque reconfere o certificado sem confiar na busca — além de ciência com evidência que dispensa gabarito (leis de conservação provadas sobre os racionais) e de uma suíte que decide se uma diferença de avaliação de IA é real. A avaliação usa oráculos externos e, para cada verificação, um controle que poderia ter falhado.

Palavras-chave: compiladores. reprodutibilidade numérica. verificação de software. certificados. finanças quantitativas.

ABSTRACT

vapor is a tensor compiler and runtime written in Elixir with no external dependencies, which treats numerical reproducibility and verifiability as design requirements rather than desirable properties. Programs are terms of a symbolic algebra; the control plane, on the Erlang virtual machine (BEAM), rewrites them only by exact identities, allocates registers with an algorithm whose result is checked by a verifier proved in the Lean proof assistant and extracted to Elixir, and emits machine bits for five targets — x86-64 (AVX2 and AVX-512), AArch64 NEON, RISC-V with the vector extension and SPIR-V — with no assembler, no C and no LLVM. Generated code never runs inside the virtual machine: it runs in isolated processes that may fail without bringing anything down. Every compilation climbs a six-rung verification ladder and leaves with a deterministic, co-signable Ed25519 certificate. The central guarantee is that, under the canonical policy, every substrate produces the same bits. On this core, fourteen rounds built an ecosystem: frontier language models, image, audio, documents, agents whose execution is a proof object and, in the last rounds, problem-solving desks in which a search proposes and a checker decides. This monograph describes the architecture and the innovations — what they are, for whom, for what, why and how — and details round 0.13, which applies the principle to finance: calendars and curves equal to the QuantLib reference library, Monte Carlo compiled with the generator inside the algebra and bits identical to the exact oracle, backtests judged by four noise gates (among them a certificate of no look-ahead by prefix invariance), arbitrage decided exactly by Farkas’ lemma, and a limit order book whose hash-chained journal is judged by an independent naive engine — which found a real defect. Round 0.14 replaces the per-category desks by an *open bench*: the user, or a language model, writes any problem in a pure, sanitised language; a furnace of strategies searches it against random search at the same budget; and a touchstone re-checks the certificate without trusting the search — together with science whose evidence needs no reference answer (conservation laws proved over the rationals) and a suite that decides whether an AI evaluation difference is real. The evaluation uses external oracles and, for each check, a control that could have failed.

Keywords: compilers. numerical reproducibility. software verification. certificates. quantitative finance.

LISTA DE ILUSTRAÇÕES

Figura 1 – A redução canônica de 16 acumuladores ($r8 \rightarrow r4 \rightarrow r2 \rightarrow r$). Toda ISA e a GPU executam esta mesma árvore; por isso os bits coincidem.	17
Figura 2 – Visão geral: o plano de controle na BEAM produz bits e certificados; o código gerado executa em processos isolados; o oráculo exato é a referência. Nenhum byte emitido executa dentro da máquina virtual.	24
Figura 3 – A cadeia de <i>failover</i> do despacho.	27
Figura 4 – A eclusa de substratos no console: cada substrato com o veredito de admissão e a impressão numérica medida.	28
Figura 5 – As quatro classes de garantia, da base confiável (topo) ao núcleo provado (base).	31
Figura 6 – O padrão proponente/verificador com controle. A busca pode ser complexa e não confiável; o verificador é pequeno e independente; o controle mostra que ele tem poder.	38
Figura 7 – A mesa de lógica no console: a afirmação, a decisão e a prova conferível.	39
Figura 8 – A mesa de finanças: quatro portas, um analisador, um motor e um certificado calculado por outro caminho.	43
Figura 9 – Curva DI ($DI1 + LTN + NTN-F$, <i>flat-forward</i> , $DU/252$) no console: taxa zero, <i>forward</i> instantâneo, nós e o certificado de reprecificação.	45
Figura 10 – O sorriso de Vogt: o ajuste SVI, a densidade neutra ao risco (negativa onde há arbitragem) e $g(k)$ de Durrleman abaixo de zero.	47
Figura 11 – Monte Carlo no worker nativo: europeia, asiáticas e barreira, com o selo de bits idênticos (oráculo e 2 <i>threads</i>) e a comparação com a BEAM em binary64.	50
Figura 12 – VaR normal em caudas grossas: as previsões de um passo, as exceções, os testes de Kupiec e Christoffersen e a zona de Basileia.	51
Figura 13 – Trinta cruzamentos de médias sobre ruído: o melhor parece bom, e os portões dizem que é ruído (DSR 0,10; <i>Reality Check</i> $p = 0,78$).	54
Figura 14 – Cotações de <i>calls</i> com uma borboleta que paga: o portfólio de arbitragem, o custo exato e o pagamento em cada vértice.	55
Figura 15 – O livro de ofertas e os seus três verificadores independentes: o motor ingênuo, os invariantes e o feed de mercado.	56
Figura 16 – O livro de ofertas no console: profundidade, o diário encadeado (cada entrada selecionável), a prova de Merkle do primeiro negócio, o feed ITCH e os relatórios FIX.	58

Figura 17 – Hawkes: os parâmetros plantados e ajustados, o Poisson como controle e o teste de reescala do tempo.	60
Figura 18 – Uma sessão de 1 500 passos: 12 436 eventos refeitos pelo motor ingênuo, limites pré-negociação conferidos, feed ITCH igual ao livro; preços, estoques dos formadores e profundidade.	61
Figura 19 – O espaço de trabalho: a régua de Golomb de 7 marcas achada no ótimo (25); a linha tracejada é a busca aleatória com o mesmo orçamento, que não passou de 28; abaixo, a partilha do portfólio.	67
Figura 20 – Rodada 0.13: Monte Carlo no worker contra a BEAM em binary64 (6–23×), e o custo do juiz ingênuo contra o do motor ($\approx 8,5 \mu\text{s}$ por evento) para 12 500 eventos.	73

LISTA DE TABELAS

Tabela 1 – O vapor nas cinco perguntas	14
Tabela 2 – Dimensionamento dos tipos de registrador da KIR por ISA	25
Tabela 3 – Políticas numéricas	29
Tabela 4 – A escada de verificação	30
Tabela 5 – Camadas sobre o núcleo e a garantia que cada uma herda	32
Tabela 6 – Invariâncias testadas bit a bit	34
Tabela 7 – As suposições usuais e a inversão do vapor	40
Tabela 8 – Tarefas da mesa e o certificado de cada uma	43
Tabela 9 – Calendários conferidos dia a dia contra o QuantLib 1.43	44
Tabela 10 – Os limites da Alembic e o teste que tenta quebrar cada um	64
Tabela 11 – Oráculos externos usados nos testes	69
Tabela 12 – Testes por rodada (execução completa de <code>mix test</code>)	69
Tabela 13 – As 23 verificações da rodada 0.13: valor, controle e limiar	71
Tabela 14 – As 20 verificações da rodada 0.14: a bancada aberta	72
Tabela 15 – GEMV nesta máquina (ms por chamada; todos com os mesmos bits) .	72
Tabela 16 – Propriedades de verificabilidade em sistemas representativos	77
Tabela 17 – Onde cada número é produzido	87

LISTA DE ABREVIATURAS E SIGLAS

ABI	<i>Application Binary Interface</i>
ANBIMA	Associação Brasileira das Entidades dos Mercados Financeiro e de Capitais
BEAM	máquina virtual do Erlang/OTP
CSCV	<i>Combinatorially Symmetric Cross-Validation</i>
DRUP	<i>Delete Reverse Unit Propagation</i> (formato de refutação SAT)
DSR	<i>Deflated Sharpe Ratio</i>
DU	dias úteis
FIX	<i>Financial Information eXchange</i>
FTAP	teorema fundamental da precificação de ativos
HFT	<i>High-Frequency Trading</i>
IEEE-754	padrão de aritmética de ponto flutuante
ISA	<i>Instruction Set Architecture</i>
ITCH	protocolo de dados de mercado da Nasdaq (TotalView-ITCH 5.0)
KCL	lei de Kirchhoff das correntes
KIR	representação intermediária de kernels do vapor
LP	programação linear
MCP	<i>Model Context Protocol</i>
PBO	<i>Probability of Backtest Overfitting</i>
RVV	extensão vetorial do RISC-V (1.0)
SPIR-V	representação intermediária de <i>shaders</i> do Vulkan
VaR	<i>Value at Risk</i>

SUMÁRIO

1	INTRODUÇÃO	11
1.1	O problema	11
1.2	A tese	12
1.3	O que, para quem, para que, por que, como	13
1.4	Contribuições	13
1.5	Como ler esta monografia	15
2	FUNDAMENTOS	16
2.1	Aritmética de ponto flutuante	16
2.2	Compiladores, do termo ao registrador	17
2.3	Testar, provar, certificar	18
2.4	Criptografia aplicada: hashes, árvores e assinaturas	19
2.5	A máquina virtual do Erlang	20
2.6	Sinal ou ruído: testes, controles, tamanho e poder	20
2.7	Finanças quantitativas: o mínimo necessário	21
3	ARQUITETURA	23
3.1	Visão geral	23
3.2	Álgebra e programas	23
3.3	Compilação	25
3.3.1	Reescrita exata	25
3.3.2	KIR, tipos de registrador e o fator de grupo	25
3.3.3	Alocação sem <i>spill</i> , verificada	25
3.3.4	Codificação binária própria	26
3.4	Substratos e isolamento	26
3.4.1	O worker	26
3.4.2	O fabric	27
3.4.3	Protocolo, memória e <i>failover</i>	27
3.4.4	A eclusa de substratos	28
3.5	Modelo numérico	29
3.6	Escada de verificação e certificado	30
3.7	Árbitro: o <i>roofline</i> de três tetos	30
3.8	Modelo de garantias	31
3.9	Camadas sobre o núcleo	32
4	INOVAÇÕES	33

4.1	Numérica canônica portátil	33
4.2	Desempenho sem reassociar	34
4.3	Compilar sem confiar no compilador	35
4.4	Contenção total do código gerado	36
4.5	Certificados portáteis, co-assináveis e transparentes	36
4.6	Eclusas: admissão por medição	37
4.7	Uma busca propõe, um verificador decide — com controles	38
4.8	Repetir é verificar	40
4.9	Primeiros princípios: as inversões	40
5	FINANÇAS VERIFICÁVEIS: A RODADA 0.13	42
5.1	A mesa como arquitetura	42
5.2	Dinheiro exato e calendários por regra	42
5.3	Curvas de juros	44
5.4	Opções	46
5.5	Monte Carlo compilado para o núcleo	48
5.6	Risco de mercado e carteiras	50
5.7	<i>Backtests</i> com portões de ruído	51
5.7.1	Portão 1: invariância de prefixo	52
5.7.2	Portões 2–4: seleção	53
5.8	Arbitragem decidida exatamente	53
5.9	O livro de ofertas como objeto verificável	56
5.10	Protocolos e risco pré-negociação	57
5.11	Microestrutura, cada modelo com a medida que diz se serve	59
5.12	Uma sessão de bolsa que se audita	60
5.13	O que este capítulo não afirma	62
6	A BANCADA ABERTA: A RODADA 0.14	63
6.1	Escrutínio do pedido	63
6.2	Alembic: uma linguagem pura e saneada	64
6.3	Athantor e Touchstone	64
6.4	Crucible: evidência sem gabarito	65
6.5	Assay: essa diferença é real?	66
6.6	Interfaces: o terminal primeiro	66
6.7	O que este capítulo não afirma	67
7	AVALIAÇÃO	68
7.1	Metodologia	68
7.2	Testes	68
7.3	Qualidade: sinal ou ruído	70

7.4	Desempenho	70
7.5	Revisões independentes	71
7.6	Ameaças à validade	73
8	TRABALHOS RELACIONADOS	75
8.1	Compiladores de tensores	75
8.2	Reprodutibilidade numérica	75
8.3	Compiladores verificados e validação de tradução	75
8.4	Algoritmos certificadores e verificação de saídas	76
8.5	Inferência de modelos de linguagem	76
8.6	Finanças quantitativas	76
8.7	Síntese	77
9	CONCLUSÃO	78
9.1	Síntese	78
9.2	Respostas às cinco perguntas	78
9.3	Limitações	79
9.4	Trabalhos futuros	80
9.5	Considerações finais	80
	REFERÊNCIAS	81
	APÊNDICES	86
	APÊNDICE A – REPRODUÇÃO	87
A.1	Ambiente	87
A.2	Comandos por afirmação	87
A.3	Os documentos	88
	APÊNDICE B – EXERCÍCIOS	89
	APÊNDICE C – GLOSSÁRIO	92

1 INTRODUÇÃO

Um número produzido por um computador carrega, implicitamente, uma afirmação: *este é o resultado do cálculo que você pediu*. Na maior parte da computação científica e da aprendizagem de máquina contemporâneas, essa afirmação não pode ser conferida por quem a recebe. O cálculo atravessa uma pilha de bibliotecas, compiladores, *drivers* e aceleradores que ninguém audita por inteiro; o resultado muda quando o mesmo programa roda em outra máquina, com outro número de *threads* ou numa placa de vídeo; e, cada vez mais, o número vem de um modelo estatístico cuja saída é plausível por construção, e plausível não é o mesmo que verdadeiro.

Esta monografia descreve o **vapor**, um sistema que toma a posição oposta: um número só sai acompanhado do objeto que permite julgá-lo. O **vapor** começou como um compilador e *runtime* de tensores com uma garantia incomum — *os mesmos bits em todo substrato* — e cresceu, ao longo de treze rodadas de desenvolvimento, até um ecossistema que serve modelos de linguagem, lê documentos escaneados, resolve problemas de engenharia e lógica e, na rodada mais recente, opera uma mesa de finanças e de negociação. O fio condutor de todas as rodadas é a mesma separação: *uma busca propõe, um verificador decide*.

1.1 O PROBLEMA

Quatro dores motivam o trabalho. Cada uma é real, mensurável e mal atendida pelo estado da prática.

Pilhas que ninguém audita.

Uma instalação típica de aprendizagem profunda ocupa gigabytes: o *framework*, bibliotecas numéricas de fornecedores, compiladores just-in-time, *drivers*. O **vapor** mediu, para a sua apresentação, cerca de 3 010 MB de *wheels* comprimidos para uma instalação de PyTorch com CUDA, contra 1,76 MB do núcleo do **vapor** (plano de controle compilado e binários nativos) e 8,7 MB do sistema inteiro, com as treze rodadas, sem contar o *runtime* da linguagem em nenhum dos lados. O tamanho não é o problema em si; o problema é que cada componente é um ponto de confiança, e a confiança não é transitiva.

Números que mudam de máquina para máquina.

A aritmética de ponto flutuante não é associativa: $(a + b) + c$ e $a + (b + c)$ podem diferir no último bit, e numa soma de milhões de termos essas diferenças se acumulam

(GOLDBERG, 1991; HIGHAM, 2002). Uma redução paralela que muda de ordem com o número de *threads*, uma instrução *fused multiply-add* que existe numa arquitetura e não em outra, uma implementação diferente de *exp*: qualquer uma basta para que o mesmo programa produza resultados diferentes. Em aprendizagem de máquina isso torna experimentos irreproduzíveis; em finanças, transforma um número de risco num achado de auditoria esperando para acontecer.

Ruído plausível.

Modelos generativos produzem saídas com a forma estatística de respostas corretas. Sem um mecanismo que separe sinal de ruído — um controle que uma implementação quebrada produziria, um teste que poderia ter falhado —, uma saída bem formatada é indistinguível de uma saída correta. O mesmo vale, em escala menor, para qualquer procedimento de busca: o melhor de cinquenta *backtests* parece bom mesmo sobre dados aleatórios (BAILEY; López de Prado, 2014; HARVEY; LIU; ZHU, 2016).

Mercados sem verificabilidade.

Na indústria financeira, as falhas mais caras raramente são de velocidade; são de prova. Um preço que muda quando o cálculo muda de servidor, uma estratégia que olhou, sem querer, o preço de amanhã, um livro de ofertas cujo comportamento ninguém de fora consegue auditar. Reguladores exigem que firmas com acesso ao mercado tenham controles de risco pré-negociação e possam demonstrar que os tinham (U.S. Securities and Exchange Commission, 2010; Comissão Europeia, 2017); demonstrar é exatamente o que as arquiteturas usuais dificultam.

1.2 A TESE

A tese deste trabalho é que **verificabilidade pode ser uma propriedade de arquitetura**, obtida por construção e a custo baixo, se três decisões forem tomadas desde o início:

1. **A semântica é simbólica e fixa.** Um programa é um termo de uma álgebra cujos operadores têm significado definido bit a bit (inclusive as funções transcendentais, implementadas como microprogramas sobre $+$, $-$ e $\times$ corretamente arredondados). O paralelismo vem de saídas independentes, nunca de reassociar uma soma.
2. **O código gerado é isolado e o compilador não é confiável.** O vapor emite bits de máquina diretamente, mas toda alocação de registradores é conferida por um verificador provado; todo binário roda num processo que pode morrer sem

consequências; e cada compilação produz um certificado com o trabalho contado e as evidências de cada degrau de verificação.

3. **Toda resposta vem com o seu juiz.** Nas camadas de aplicação, o procedimento que produz uma resposta é separado do procedimento que a confere — e o segundo é deliberadamente mais simples, independente do primeiro, ou ambos.

A rodada 0.13 testa a tese num domínio em que ela tem consequências econômicas diretas. Se a dor de finanças é verificabilidade, então um sistema construído para a verificabilidade deveria produzir, sem esforço extraordinário, coisas que a prática usual não produz: números de risco com os mesmos bits em qualquer máquina, *backtests* que provam não ter olhado o futuro, arbitragens decididas e não estimadas, livros de ofertas auditáveis por terceiros. O Capítulo 5 mostra que isso é o que acontece. A rodada 0.14 faz o teste inverso: em vez de um domínio escolhido, qualquer problema que o usuário escreva (Capítulo 6).

1.3 O QUE, PARA QUEM, PARA QUE, POR QUE, COMO

A Tabela 1 resume o trabalho nas cinco perguntas que organizam esta monografia. Cada inovação do Capítulo 4 é apresentada com uma ficha que responde às mesmas perguntas.

1.4 CONTRIBUIÇÕES

As contribuições deste trabalho, todas implementadas e testadas no repositório, são:

1. Uma **numérica canônica portátil**: um conjunto de operadores com semântica fixa bit a bit, incluindo divisão corretamente arredondada e funções transcendentais como microprogramas, que produz os mesmos bits em x86-64 (AVX2 e AVX-512), AArch64, RISC-V (qualquer VLEN de 128 a 512) e Vulkan, e que é invariante ao número de *threads*, ao tamanho do lote e ao armazenamento dos pesos (seção 4.1).
2. Um **compilador sem cadeia de ferramentas externa** que emite bits de máquina com codificadores próprios, valida cada instrução contra o GNU binutils e o `spirv-val` nos testes, e confere toda alocação de registradores com um verificador provado e extraído do Lean (seção 3.3).
3. **Contenção total** do código gerado: nenhum byte emitido executa na máquina virtual; quatro classes de falha viram mensagens tipadas (seção 3.4).

Tabela 1 – O vapor nas cinco perguntas

pergunta	resposta
O quê	Um compilador e <i>runtime</i> de tensores certificados (álgebra simbólica $\rightarrow$ bits de máquina em cinco alvos, executados em processos isolados, com certificado), e um ecossistema de aplicações construído sobre ele: modelos de linguagem, documentos, agentes, mesas de engenharia, lógica, ciência e finanças.
Para quem	Equipes que precisam <i>demonstrar</i> um resultado e não apenas obtê-lo: reguladas (finanças, saúde, setor público), que auditam modelos de IA, que precisam de reprodutibilidade científica; pesquisadores de compiladores e verificação; estudantes que queiram ver um sistema completo, do termo ao registrador, em código legível.
Para que	Para que um número computado possa ser julgado por quem o recebe: os mesmos bits em qualquer substrato, um certificado do que foi compilado, um verificador independente para cada resposta, e um controle que mostra que a verificação poderia ter falhado.
Por que	Porque as falhas que mais custam — irreprodutibilidade, ruído plausível, auditoria impossível — são falhas de prova, e não de desempenho; e porque a prova, feita por construção, custa pouco.
Como	Semântica simbólica fixa; reescrita só por identidades exatas; emissão direta de bits com alocação verificada por um verificador provado em Lean; execução em processos isolados; escada de verificação de seis degraus; certificados Ed25519 determinísticos; e, nas aplicações, a separação proponente/verificador com controles calibrados.

4. Uma **escada de verificação** de seis degraus que termina num certificado co-assinável por nós independentes (seção 3.6).
5. O padrão **proponente/verificador com controles** aplicado a domínios inteiros — lógica, engenharia, ciência e finanças —, inclusive a propostas vindas de modelos de linguagem pelo protocolo MCP (seção 4.7).
6. Na rodada 0.13, **finanças verificáveis**: invariância de prefixo como certificado de ausência de antecipação; arbitragem decidida pelo lema de Farkas em aritmética racional; Monte Carlo como programa canônico, com o gerador dentro da álgebra; um livro de ofertas julgado por um motor ingênuo independente; e uma sessão de bolsa em que o *backtest* é o próprio código da bolsa (Capítulo 5).
7. Na rodada 0.14, a **bancada aberta**: uma linguagem pura e saneada para escrever qualquer problema; uma fornalha de estratégias que o busca contra a busca aleatória com o mesmo orçamento; uma pedra de toque que reconfere o certificado; leis de conservação provadas sobre $\mathbb{Q}$; e uma suíte que decide se uma diferença de avaliação de IA é real (Capítulo 6).

1.5 COMO LER ESTA MONOGRAFIA

O texto foi escrito para três leitores ao mesmo tempo. Ao longo dos capítulos, caixas marcam o nível:

Para começar (graduação)

Explicações intuitivas e exemplos concretos, para quem tem a formação de um bacharelado em computação: ponto flutuante, compiladores, criptografia e estatística no nível de um primeiro curso.

Aprofundamento (mestrado)

Os mecanismos com a precisão necessária para reimplementá-los: formatos, algoritmos, cotas de erro, os argumentos de correção.

Questão em aberto (doutorado)

O que não está resolvido — aqui ou na literatura —, formulado como pergunta de pesquisa, com a evidência que o **vapor** oferece e o que faltaria para respondê-la.

O Capítulo 2 reúne os fundamentos de que os demais capítulos dependem e pode ser pulado por quem já os domina. O Capítulo 3 descreve a arquitetura; o Capítulo 4, as inovações, cada uma com a sua ficha; o Capítulo 5 aprofunda a rodada 0.13; o Capítulo 6, a 0.14; o Capítulo 7 apresenta a avaliação e as ameaças à validade; o Capítulo 8 situa o trabalho; o Capítulo 9 conclui com as limitações e os trabalhos futuros. O Apêndice A lista os comandos que reproduzem cada número citado; o Apêndice B propõe exercícios por nível.

Uma advertência de método atravessa o texto: **nenhuma afirmação de superioridade é feita sem a comparação que a sustenta**, e onde o **vapor** perde — em velocidade de jogo de xadrez, em predição de estrutura de proteínas, em latência de bolsa — isso é dito. Os números citados foram medidos numa máquina modesta (um Xeon de 2,1 GHz com duas vCPUs, AVX-512, sem GPU física nem contadores de desempenho), e o Apêndice A diz como refazê-los.

2 FUNDAMENTOS

Este capítulo reúne o que os capítulos seguintes pressupõem. Cada seção começa no nível de um curso de graduação e termina onde o **vapor** precisa de mais. O leitor que já domina um tema pode pular a seção correspondente sem perda.

2.1 ARITMÉTICA DE PONTO FLUTUANTE

Para começar (graduação)

Um número de ponto flutuante `binary32` (o `float` do C) ocupa 32 bits: 1 de sinal, 8 de expoente e 23 de fração. Ele representa $(-1)^s \cdot 1, f \cdot 2^{e-127}$ e, portanto, só cerca de 7 dígitos decimais. Cada operação aritmética calcula o resultado exato e o *arredonda* para o número representável mais próximo. Como cada arredondamento perde um pouco, a ordem das operações importa: em `binary32`, $(10^8 + 1) - 10^8 = 0$, porque $10^8 + 1$ não é representável e arredonda para 10^8 ; mas $(10^8 - 10^8) + 1 = 1$.

O padrão IEEE-754 (IEEE, 2019) define os formatos, o arredondamento ao mais próximo com desempate para o par (RNE) e o comportamento de zeros com sinal, infinitos, NaN e números subnormais. O fato central para este trabalho é o *modelo padrão*: para $op \in \{+, -, \times, \div\}$ e operandos longe do *underflow*,

$$\text{fl}(x \text{ op } y) = (x \text{ op } y)(1 + \delta), \quad |\delta| \leq u = 2^{-24} \text{ (binary32)}. \quad (2.1)$$

Duas consequências organizam o **vapor**. A primeira é a **não associatividade**: uma soma de n termos tem tantos resultados possíveis quanto ordens de redução. A segunda é que, apesar disso, o erro de *qualquer* ordem é limitado: para uma soma ou um produto interno de n termos, Wilkinson (1963) e Higham (2002) mostram que

$$\left| \text{fl}\left(\sum_{i=1}^n x_i y_i\right) - \sum_{i=1}^n x_i y_i \right| \leq \gamma_n \sum_{i=1}^n |x_i y_i|, \quad \gamma_n = \frac{nu}{1 - nu}. \quad (2.2)$$

O **vapor** usa as duas consequências de modos complementares. Para *reprodutibilidade*, fixa uma única ordem: cada produto interno tem exatamente 16 acumuladores, reduzidos numa árvore fixa (Figura 1), e o paralelismo vem de processar várias *linhas* independentes ao mesmo tempo, nunca de reassociar uma soma. Para *verificação*, usa a cota (2.2), calculada em racionais diádicos exatos, como o envelope dentro do qual qualquer substrato conforme tem de cair.

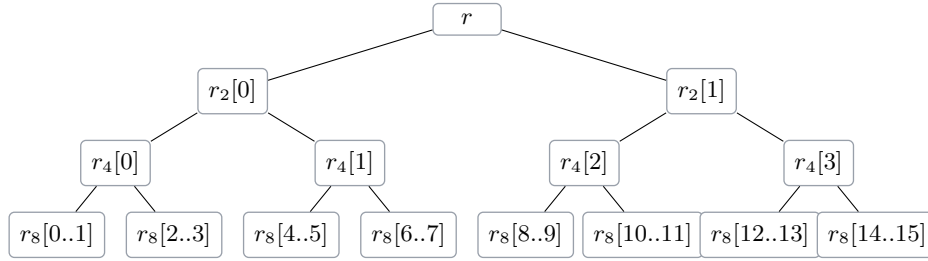


Figura 1 – A redução canônica de 16 acumuladores ($\mathbf{r8} \rightarrow \mathbf{r4} \rightarrow \mathbf{r2} \rightarrow \mathbf{r}$). Toda ISA e a GPU executam esta mesma árvore; por isso os bits coincidem.

Aprofundamento (mestrado)

Por que $\text{fl}(a + b)$ em **binary32 pode ser calculado exatamente em **binary64**:** a soma exata de dois **binary32** cabe em $24 + 24 + 2$ bits significativos no pior caso relevante, e um arredondamento duplo $\mathbb{R} \rightarrow \text{binary64} \rightarrow \text{binary32}$ é inócuo quando $53 \geq 2 \cdot 24 + 2$ (HIGHAM, 2002). O oráculo do **vapor** (**Vapor.F32**) usa esse fato para somas e produtos e calcula o *fused multiply-add* em racionais diádicos com um único arredondamento — validado contra o **fmaf** do *hardware* em 800 000 operações, inclusive subnormais e zeros com sinal.

Zeros com sinal e reescrita.

Identidades algébricas que parecem óbvias não valem bit a bit. $x + 0 = x$ é falsa para $x = -0$, porque $(-0) + (+0) = +0$; já $x + (-0) = x$ vale para todo x . Um compilador que reescreve expressões precisa distinguir as duas, e o **vapor** só admite as identidades válidas para todos os valores IEEE — o que é provado num modelo de bits em Lean (seção 3.3).

Subnormais, FTZ e DAZ.

GPUs e alguns compiladores descartam números subnormais (*flush-to-zero*, FTZ, na saída; *denormals-are-zero*, DAZ, na entrada). Isso não é um erro de arredondamento — é outra semântica. O **vapor** trata essas diferenças como impressões numéricas de um substrato, medidas antes de admiti-lo (seção 3.4).

2.2 COMPILADORES, DO TERMO AO REGISTRADOR

Para começar (graduação)

Um compilador traduz um programa de uma representação para outra em etapas: análise, representação intermediária (IR), otimizações, seleção de instruções, alocação de registradores e emissão de código. Os registradores são poucos (16 vetoriais no AVX2, 32 no AVX-512); quando faltam, compiladores comuns “derramam” valores na

memória (*spill*).

O *vapor* segue o esquema clássico com três particularidades. A IR de alto nível é uma *álgebra de termos* (seção 3.2); a IR de baixo nível (KIR) é escrita uma vez sobre registradores virtuais *tipados*, que cada *backend* dimensiona; e a alocação de registradores é a de *linear scan* (POLETTTO; SARKAR, 1999) sem caminho de *spill*: se faltam registradores, o compilador muda a forma do código (menos linhas por iteração, um fator de grupo menor, uma fronteira de região) em vez de derramar.

Codificação de instruções.

Emitir bits sem assembler exige conhecer a codificação de cada ISA. No x86-64, uma instrução vetorial moderna leva um prefixo VEX (AVX2) ou EVEX (AVX-512) que codifica registradores, largura e máscaras, seguido do *opcode*, do byte ModR/M e, quando há memória, do SIB e do deslocamento (Intel Corporation, 2024). No AArch64, cada instrução é uma palavra de 32 bits. No RISC-V vetorial, o tipo do vetor (*vtype*) codifica a largura do elemento (SEW) e o agrupamento de registradores (LMUL) (RISC-V International, 2021). No SPIR-V, um módulo é uma sequência de palavras com seções em ordem fixa (The Khronos Group, 2024). Errar um bit produz uma instrução diferente — válida ou não —, e por isso todo construtor de kernel é conferido nos testes contra o GNU *binutils* e o *spirv-val*.

2.3 TESTAR, PROVAR, CERTIFICAR

Para começar (graduação)

Testar mostra que um programa funciona nos casos testados. *Provar* mostra que funciona em todos, mas exige um modelo formal e muito esforço. *Certificar* é um meio-termo poderoso: o programa produz, junto com a resposta, uma evidência (um *certificado*) que um segundo programa, bem mais simples, confere. Se o conferidor é correto, a resposta aceita é correta — mesmo que o primeiro programa tenha erros.

A ideia tem várias formas na literatura: *proof-carrying code* (NECULA, 1997), em que o código carrega a prova da sua segurança; validação de tradução (PNUELI; SIEGEL; SINGERMAN, 1998), em que cada compilação é conferida em vez de o compilador ser provado; compiladores verificados como o CompCert (LEROY, 2009), em que o compilador inteiro é provado; e, em lógica, os formatos de refutação como DRUP e DRAT (GOLDBERG; NOVIKOV, 2003; WETZLER; HEULE; JR., 2014), em que um *solver* SAT entrega uma prova de insatisfatibilidade que um verificador independente reexecuta.

O *vapor* combina as três: um núcleo pequeno de algoritmos é provado em Lean 4 (MOURA; ULLRICH, 2021) e *extraído* para Elixir (o verificador de alocação, a decisão do

envelope, a aritmética modular); cada compilação é validada por uma escada de verificação (seção 3.6); e as aplicações produzem certificados que verificadores independentes conferem.

Aprofundamento (mestrado)

O critério de de Bruijn. Um sistema de prova é confiável na medida em que o seu verificador é pequeno e independente do procedimento de busca. O *vapor* aplica o critério fora da lógica: o verificador de um livro de ofertas é um motor ingênuo com listas e ordenação, sem nenhuma estrutura de dados compartilhada com o motor de produção; o verificador de uma arbitragem só multiplica racionais; o verificador de um *backtest* recalcula o sinal sobre históricos truncados. Em todos os casos, aceitar exige menos código e menos suposições do que produzir.

2.4 CRIPTOGRAFIA APLICADA: HASHES, ÁRVORES E ASSINATURAS

Para começar (graduação)

Uma função de *hash* criptográfica como a SHA-256 transforma qualquer dado em 32 bytes de forma que é inviável achar dois dados com o mesmo *hash*. Encadeando *hashes* — $h_i = H(h_{i-1} || d_i)$ — obtém-se um diário em que mudar uma entrada antiga muda todos os *hashes* seguintes. Uma *árvore de Merkle* (MERKLE, 1987) resume n dados num único *hash* (a raiz) e permite provar que um dado pertence ao conjunto mostrando só $\log_2 n$ *hashes*.

O *vapor* usa a construção das árvores do *Certificate Transparency* (RFC 6962 e RFC 9162 (LAURIE; LANGLEY; KASPER, 2013; LAURIE; MESSERI; STRADLING, 2021)): folhas são $H(0x00 || d)$ e nós internos $H(0x01 || e || d)$, o que impede confundir uma folha com um nó. As assinaturas são Ed25519 (BERNSTEIN et al., 2012), determinísticas: a mesma mensagem e a mesma chave produzem a mesma assinatura, o que permite a co-assinatura de certificados por nós independentes.

Bytes canônicos.

Assinar um valor exige uma única codificação dele. O *vapor* usa o CBOR determinístico (RFC 8949, §4.2) para tudo que é assinado ou endereçado por conteúdo: ordem de chaves fixa, inteiros na forma mais curta, ponto flutuante na menor largura que preserva o valor. A alternativa óbvia na BEAM — `term_to_binary` com a opção determinística — só é estável dentro de uma mesma versão do Erlang/OTP, o que impediria dois nós com versões diferentes de concordar sobre os bytes de um certificado.

2.5 A MÁQUINA VIRTUAL DO ERLANG

A BEAM (ARMSTRONG, 2003) executa milhões de processos leves isolados, que se comunicam por mensagens e são supervisionados: um processo que falha é reiniciado por quem o supervisiona, sem derrubar os demais. O *vapor* usa a BEAM como *plano de controle* — compilação, verificação, orquestração, serviço HTTP — e mantém todo código gerado *fora* dela: as funções nativas (NIFs), que executariam no espaço de endereçamento da VM, são proibidas por um teste de auditoria. Código de máquina roda em processos do sistema operacional falados por *ports*, com quadros de comprimento prefixado; um processo que morre no meio de um quadro não dessincroniza a VM.

2.6 SINAL OU RUÍDO: TESTES, CONTROLES, TAMANHO E PODER

Para começar (graduação)

Um teste estatístico tem dois erros possíveis: rejeitar uma hipótese verdadeira (erro tipo I, cuja probabilidade é o *tamanho* do teste, tipicamente 5%) e aceitar uma falsa (erro tipo II; a probabilidade de evitá-lo é o *poder*). Um teste que nunca rejeita nada não serve para nada, mesmo que nunca erre do primeiro tipo.

O *vapor* organiza a sua avaliação de qualidade como uma bateria de verificações em que cada linha tem quatro campos: o valor medido, um **controle** (o valor que uma implementação quebrada, ingênua ou trapaceira produziria), um limiar e o veredito. Uma verificação só é aceita se o controle cai do outro lado do limiar — isto é, se ela *poderia ter falhado*. Os testes da rodada 0.13 vão além: o tamanho do teste de Kupiec é medido em 400 séries simuladas sob a hipótese nula, e o seu poder, numa série em que a hipótese é falsa (seção 5.6).

Seleção e testes múltiplos.

Quando muitas hipóteses são testadas e só a melhor é relatada, o nível de significância nominal perde o sentido: o máximo de N estatísticas sob a nula cresce com N . Em finanças, isso é o *backtest overfitting* (BAILEY et al., 2017; HARVEY; LIU; ZHU, 2016); o *vapor* o trata com o índice de Sharpe deflacionado (BAILEY; López de Prado, 2014), a probabilidade de sobreajuste por validação cruzada combinatória (BAILEY et al., 2017) e o *Reality Check* de White (2000).

2.7 FINANÇAS QUANTITATIVAS: O MÍNIMO NECESSÁRIO

Para começar (graduação)

Desconto. Um real amanhã vale menos que um real hoje. O fator de desconto $D(t)$ é o preço hoje de 1 pago em t ; uma curva de juros é a função $t \mapsto D(t)$. No Brasil, contratos de DI e títulos públicos contam o tempo em dias úteis sobre 252: $D = (1 + r)^{-DU/252}$.

Arbitragem. Uma arbitragem é uma carteira que não custa nada (ou rende algo hoje), nunca perde e às vezes ganha. Mercados razoáveis não têm arbitragens duradouras.

Opções. Uma *call* europeia paga $\max(S_T - K, 0)$ no vencimento T . Sob o modelo de Black e Scholes (1973) e Merton (1973), o seu preço tem fórmula fechada; a volatilidade implícita é o σ que reproduz um preço observado.

O teorema fundamental.

Harrison e Kreps (1979) e Harrison e Pliska (1981) mostraram que, num mercado com um número finito de estados, não há arbitragem se e somente se existem *preços de estado* estritamente positivos ψ_s tais que todo ativo vale $\sum_s X_{is}\psi_s$, em que X_{is} é o pagamento do ativo i no estado s . Com custos de transação (preços de compra e venda), a condição vira $\text{bid}_i \leq \sum_s X_{is}\psi_s \leq \text{ask}_i$ (JOUINI; KALLAL, 1995). O teorema é um caso do **lema de Farkas** (FARKAS, 1902): de dois sistemas lineares, exatamente um tem solução. Essa observação — a arbitragem como alternativa de Farkas — é a base da seção 5.8.

Monte Carlo.

Para derivativos sem fórmula fechada, o preço é a esperança do pagamento descontado, estimada pela média de trajetórias simuladas; o erro-padrão cai como $N^{-1/2}$ e técnicas como variáveis de controle o reduzem (GLASSERMAN, 2003). Um estimador é *viciado* se a sua esperança difere do preço; o exemplo clássico é esquecer o termo $-\sigma^2/2$ de Itô no logaritmo do preço.

Risco.

O *Value at Risk* a 99% é a perda que só é excedida em 1% dos dias. Um modelo de VaR se testa contando as exceções: sob o modelo correto, elas seguem uma binomial, e o teste de Kupiec (1995) compara a proporção observada com a nominal; o de Christoffersen (1998) verifica se as exceções se agrupam no tempo.

Microestrutura.

Uma bolsa casa ordens num **livro de ofertas**: ordens de compra e venda limitadas esperam em filas por preço e, dentro do preço, por ordem de chegada (prioridade preço–

tempo); uma ordem que cruza o melhor preço do outro lado é executada contra as que esperam. A literatura de microestrutura modela a chegada das ordens (HAWKES, 1971; BACRY; MASTROMATTEO; MUZY, 2015), a formação de mercado (AVELLANEDA; STOIKOV, 2008) e a execução ótima de grandes ordens (ALMGREN; CHRISS, 2000).

3 ARQUITETURA

Este capítulo descreve o **vapor** como ele está implementado e testado: o fluxo que leva um programa simbólico até bits de máquina, os processos em que esses bits executam, o modelo numérico que os torna iguais em todo lugar, a escada de verificação que termina num certificado e o modelo de garantias que diz o que está provado, o que está testado e o que é confiável. As inovações que essa arquitetura torna possíveis são o assunto do Capítulo 4; aqui o objetivo é que o leitor entenda *como as peças se encaixam*.

3.1 VISÃO GERAL

A Figura 2 mostra o sistema em três faixas. Na faixa de cima, o *plano de controle* roda na BEAM e faz tudo que é simbólico: verifica os tipos do programa, o reescreve, o baixa para a representação de kernels, aloca registradores, emite os bits e conduz a escada de verificação. Na faixa do meio, dois processos do sistema operacional executam o código gerado: o **vapor-worker** (CPU, nativo ou interpretado) e o **vapor-fabric** (GPU via Vulkan). Na faixa de baixo, o *oráculo* — uma implementação exata da semântica, em Elixir puro — serve de referência e de último recurso.

Três números dão a escala: o núcleo (álgebra, compilação, emissores, verificação e *runtime*) tem 12 853 linhas de Elixir, o código nativo dos processos 4 717 linhas de Zig, e as provas em Lean 1 515 linhas; o repositório inteiro, com as camadas das treze rodadas, tem cerca de 78 000 linhas de Elixir. O arquivo `mix.exs` declara `deps: []`: nenhuma biblioteca externa entra no produto. As ferramentas externas (`binutils`, `spirv-val`, QEMU, Lean, PyTorch, QuantLib, ngspice) aparecem apenas nos testes, como oráculos.

Para começar (graduação)

Por que separar o “plano de controle” do “plano de dados”? A mesma divisão aparece em roteadores de rede: decidir *o que* fazer é lento, raro e cheio de lógica; fazer é rápido, repetitivo e perigoso se der errado. No **vapor**, decidir é compilar e verificar (BEAM, linguagem segura, processos supervisionados); fazer é executar bits de máquina (processos descartáveis). Se a execução falhar, o plano de controle continua de pé e pode tentar outro substrato.

3.2 ÁLGEBRA E PROGRAMAS

Um programa **vapor** é um termo de uma *álgebra livre simbólica*: cada operador é um *nome* com semântica fixa, definida pelo oráculo, e não uma função anônima da linguagem

BEAM — plano de controle (deps: [])

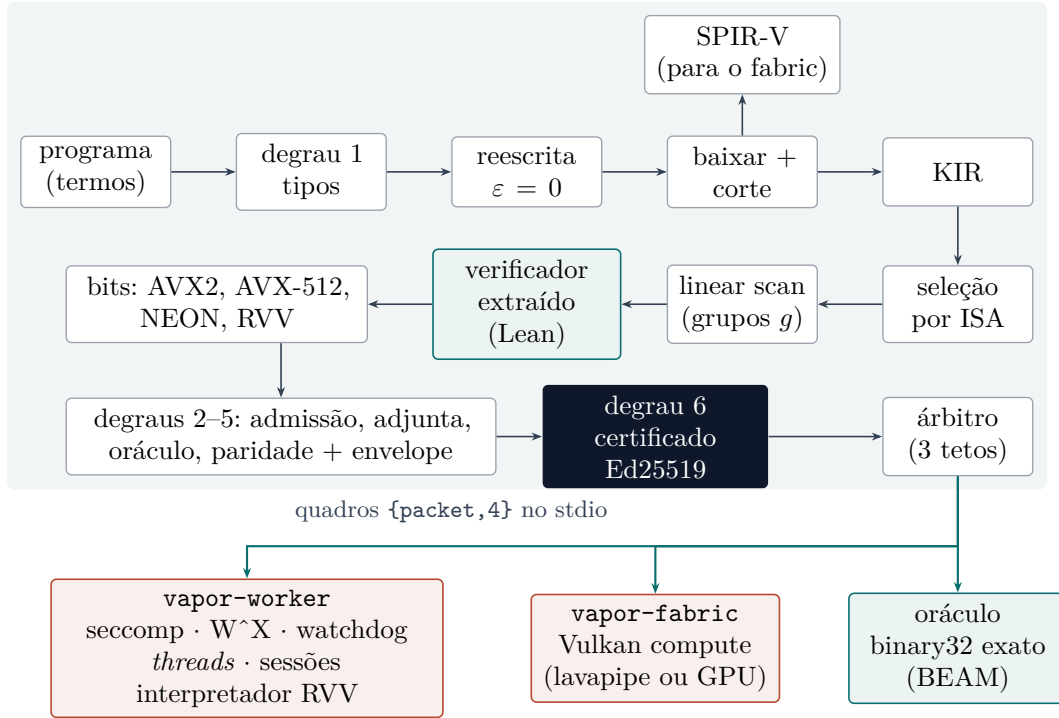


Figura 2 – Visão geral: o plano de controle na BEAM produz bits e certificados; o código gerado executa em processos isolados; o oráculo exato é a referência. Nenhum byte emitido executa dentro da máquina virtual.

hospedeira. A distinção parece pequena, mas decide o projeto: um termo simbólico pode ser inspecionado, reescrito, baixado para qualquer alvo e assinado; uma *closure* só pode ser chamada. (O antecessor do **vapor** guardava *closures* nos termos, o que nenhum emissor consegue traduzir para código de máquina.)

Definição 3.1 (Programa). *Um programa é uma tupla (I, C, T, O, σ) : entradas I com formas e tipos, constantes C (pesos), um conjunto de termos T sobre os geradores, saídas $O \subseteq T$ e, opcionalmente, um mapa de realimentação σ que liga uma saída à entrada do passo seguinte (programas recorrentes).*

Os geradores são poucos: **input**, **const**, **ew** (operações elemento a elemento — **add**, **sub**, **mul**, **fma**, **neg**, **relu** e as funções canônicas), **qgemv** (contração com matriz quantizada), **gemm_i8** (contração em $\mathbb{Z}/2^{32}\mathbb{Z}$) e alguns operadores de modelo (**gather_row**, **rope**, **attention**, **sample**). Toda a pilha de modelos de linguagem, visão, áudio, difusão e, na rodada 0.13, o Monte Carlo financeiro é escrita com esses geradores, sem operadores novos por domínio.

Duas extensões tornam a álgebra prática. As **dimensões semi-dinâmicas**, escritas $\{dyn, S, \max\}$, permitem que um comprimento de sequência varie até um máximo; todas as cotas são estabelecidas no máximo, e lemas de monotonicidade provados em Lean

garantem que o certificado vale para todo $S \leq \max$. Os **programas recorrentes** realizam um **scan** sobre um monoide afim sem materializar a sequência: o laço inteiro roda no worker atrás de uma única travessia da BEAM, com saída por passo em *streaming*.

3.3 COMPILAÇÃO

3.3.1 Reescrita exata

A reescrita só admite identidades válidas bit a bit para todos os valores IEEE-754, zeros com sinal incluídos. São admitidas, por exemplo, $\text{neg}(\text{neg } x) \rightarrow x$, $x \cdot 1 \rightarrow x$, $x + (-0) \rightarrow x$ e $x - (+0) \rightarrow x$; *não* é admitida $x + (+0) \rightarrow x$, porque $(-0) + (+0) = +0$. Desde a rodada 0.8, as regras estão provadas em Lean num modelo de bits da IEEE-754 — e a prova corrigiu uma afirmação falsa sobre NaN que estava na documentação. O dobramento de constantes avalia a própria semântica declarada, de modo que “calcular em tempo de compilação” e “calcular em tempo de execução” dão os mesmos bits.

3.3.2 KIR, tipos de registrador e o fator de grupo

Os kernels são escritos *uma vez* numa representação intermediária portátil, a KIR, sobre registradores virtuais *tipados*. Cada *backend* dimensiona os tipos (Tabela 2). A ideia central é o **fator de grupo** g , que generaliza o LMUL do RISC-V vetorial para todas as ISAs: um *strip* com $g = 4$ é um grupo m4 no RVV, quatro registradores ymm em passo travado no AVX2, quatro zmm no AVX-512 e quatro q no NEON.

Tabela 2 – Dimensionamento dos tipos de registrador da KIR por ISA

tipo	RVV	AVX2	AVX-512	NEON
:strip	mg	g ymm	g zmm	g q
:f16l (16 pistas f32)	m4, vl=16	2 ymm	1 zmm	4 q
:i32acc	$m4g$	g ymm	g zmm	2 q

O tipo :f16l é o que sustenta a garantia central: *16 pistas de binary32*, a largura canônica da redução, ocupam um registrador zmm, dois ymm, quatro q ou um grupo m4 com $vl = 16$ — e a árvore de redução de 16 acumuladores (Figura 1) é executada pista a pista do mesmo modo em todas.

3.3.3 Alocação sem *spill*, verificada

A vivacidade é calculada por ponto fixo sobre o grafo de fluxo de controle (valores vivos na aresta de retorno cobrem o laço inteiro), com *early clobber* para as instruções de alargamento do RVV. O alocador é um *linear scan* (POLETTTO; SARKAR, 1999) com blocos alinhados de tamanho 1, 2, 4 ou 8.

Não existe caminho de *spill*. Se faltam registradores, o alocador devolve o ponto de pressão e o compilador muda a *forma* do código: tenta um g menor; no GEMV de 4 bits, tenta menos linhas por iteração ($R \in \{4, 2, 1\}$); em último caso, o *cut sweep* fecha a região fundida naquele ponto. Toda alocação aceita é revalidada por `check_alloc/3`, extraído do Lean e provado correto (teorema `checkAlloc_sound`): grupos alinhados, dentro do arquivo de registradores, fora dos reservados, e nenhum par de valores simultaneamente vivos compartilhando registrador.

Aprofundamento (mestrado)

Verificar em vez de provar o alocador. Provar correta uma heurística de *linear scan* com preferências (registradores *caller-saved* primeiro, depois “vizinho” ocupado, depois ordem da ISA), blocos alinhados e *early clobber* seria caro e frágil: qualquer ajuste na heurística invalidaria a prova. Provar o *verificador* é barato — ele só confere quatro condições sobre o resultado — e vale para qualquer heurística, presente ou futura. É o mesmo raciocínio da validação de tradução (PNUELI; SIEGEL; SINGERMAN, 1998), aplicado a uma fase do compilador. O custo aparece na rodada 0.13: o verificador extraído é quadrático no número de intervalos, e compilar um passo desenrolado de Monte Carlo leva cerca de 0,6 s por ISA (seção 5.5).

3.3.4 Codificação binária própria

O vapor emite bits sem assembler. No x86-64 codifica REX, VEX de 3 bytes e EVEX (sempre com deslocamento de 32 bits, nunca o *disp8·N* comprimido), ModR/M e SIB, e segue a ABI System V. No AArch64 emite palavras A64/NEON segundo a AAPCS64 (*x18* nunca alocado). No RV64GCV monta `vtype = vlmul[2:0] | vsew[5:3] | vta | vma` e transforma desvios condicionais em `b<inverso> +8; jal` para alcance de ± 1 MiB. Para SPIR-V, um montador simbólico interna tipos e constantes, ordena as seções lógicas e marca `NoContraction` em toda multiplicação e soma da política canônica (o que impede o *driver* de fundi-las numa FMA).

Os testes validam *cada instrução emitida* — todo construtor de kernel, nos quatro *backends*, em todo fator de grupo e política — contra o GNU binutils (inclusive EVEX e `rv64gcv`) e cada módulo contra o `spirv-val`. O produto nunca invoca essas ferramentas.

3.4 SUBSTRATOS E ISOLAMENTO

3.4.1 O worker

O `vapor-worker` é um executável Zig estático, sem `libc`, de 240 a 340 KB. A mensagem de abertura fixa o número de *threads*; o *pool* de *threads* e os contadores de eventos do kernel são criados *antes* de instalar o filtro `seccomp-BPF`, sincronizado em todas

as *threads*. A lista de chamadas de sistema permitidas é curta (ler, escrever, abrir, fechar, mapear, proteger memória, relógio, temporizador, sinais e saída). Código nativo é copiado para uma página de leitura e escrita, que vira leitura e execução antes da chamada (W^X); um *watchdog* por SIGALRM mata código que não termina. O modo emulado interpreta RV64GCV com semântica RVV 1.0 fiel, checando todo acesso contra os *buffers* vinculados e oferecendo um modo *poison* que escreve 1s em elementos agnósticos — prova de que os kernels não dependem deles.

Cada chamada carrega um *descriptor de partição*: um contador, um grão, ponteiros que avançam por unidade e *scratch* por *thread*. Como as unidades são linhas independentes, o resultado é o mesmo para qualquer número de *threads*. Programas ficam residentes em *sessões* (abrir, passo, fechar): pesos mapeados uma vez, estado mantido entre passos, e cada passo escreve só as entradas dadas e devolve só as saídas pedidas.

3.4.2 O fabric

O *vapor-fabric* é um *daemon* Zig com *bindings* Vulkan escritos à mão, que executa a cadeia completa de computação sem janela: instância, dispositivo, *buffers*, módulos de *shader* com o SPIR-V emitido pela BEAM, *pipelines*, *descriptor sets*, um *command buffer* por execução, submissão e *fence* com prazo. Queda do *driver* ou perda do dispositivo matam só o *daemon*.

3.4.3 Protocolo, memória e *failover*

Ambos os processos falam quadros com comprimento prefixado de 4 bytes na entrada e saída padrão; um processo que morre no meio de um quadro não dessincroniza a BEAM. As tabelas internas são dimensionadas pelo próprio quadro, de modo que uma contagem hostil não aloca mais do que o quadro descreve. Pesos de 64 KiB ou mais vão uma única vez para um arquivo em `/dev/shm` endereçado pelo SHA-256 do conteúdo e são mapeados *copy-on-write* pelo worker e importados sem cópia pelo fabric; réplicas e reinícios mapeiam o mesmo arquivo.

O despacho segue uma cadeia de *failover* (Figura 3). Sob a política canônica todos os elos computam os mesmos bits, então reroteamento *nunca muda a resposta*; o oráculo não falha para um programa certificado.

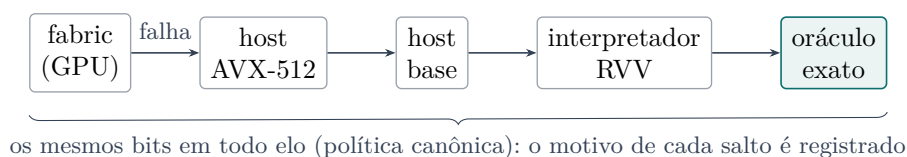


Figura 3 – A cadeia de *failover* do despacho.

Os testes provocam as quatro classes de falha do código gerado — instrução ilegal (SIGILL), acesso inválido (SIGSEGV), laço infinito (SIGALRM) e chamada de sistema proibida (SIGSYS). Em todas, só o worker morre; o processo da BEAM dono da porta reporta `{:worker_crashed, {:signal, ...}}`, reinicia o worker e a unidade seguinte roda.

3.4.4 A eclusa de substratos

Desde a rodada 0.10, um substrato novo só recebe programas canônicos depois de *medido*: sondas de resposta conhecida produzem um veredito — `:canonical` (bits iguais ao oráculo), `:envelope` (dentro da cota provada, com a impressão numérica: FMA, FTZ/DAZ, ordem de soma, bits de mantissa) ou `:refused` — e o despachante o respeita. Por ela entraram o Metal (testado por um *shim*, não num Mac), a Tenstorrent e qualquer PJRT (via StableHLO, rodado no XLA de CPU) e o FreeBSD (compilado, não executado). A Figura 4 mostra a tela de admissão no console.

vapor

model none

library 0 files

Ask

Chat

answers with their sources

Read

Documents

zip, PDF, Office, images

Vision

text in photos and scans

Listen

a spoken digit

Make

Studio

graphs of image, sound, video, 3D

Draw

a digit, by diffusion

Simulate

Physics

chaos, twins, replay

Networks

structure against its null

Measure

Fusion

which method fits

Quality

signal or noise

Airlock

the model's contract

Substrates

hardware, admitted by measurement

Training

a model vapor trained

Trust

Ledger

verify it yourself

Dossier

evidence for an auditor

no text model

Theme

Portugués

Substrates

Hardware computes only after it is measured. Each substrate runs the admission probes; the oracle's exact answer decides whether it reproduces the canonical bits, stays within the proved error envelope (and how: fused multiply-add, flushed subnormals, reduction order), or is refused.

Admit every substrate present

	FMA	FTZ	DAZ	reduction order	significant bits	signed zero	NaN select	division	functions	
host	canonical	false	false	false	canonical	24	ieee	ieee	correctly_rounded	canonical
host_avx512	canonical	false	false	false	canonical	24	ieee	ieee	correctly_rounded	canonical
rvv_emulated	canonical	false	false	false	canonical	24	ieee	ieee	correctly_rounded	canonical
fabric	canonical	false	false	false	canonical	24	ieee	ieee	correctly_rounded	canonical
oracle	canonical	false	false	false	canonical	24	ieee	ieee	correctly_rounded	canonical

Figura 4 — A eclusa de substratos no console: cada substrato com o veredito de admissão e a impressão numérica medida.

3.5 MODELO NUMÉRICO

O oráculo `Vapor.F32` implementa *binary32 exato* na BEAM (seção 2.1). Sobre ele, duas políticas (Tabela 3).

Tabela 3 – Políticas numéricas

política	semântica	garantia
<code>:canonical</code>	multiplicação e soma arredondadas separadamente; redução em árvore fixa de 16 pistas; $\div$ corretamente arredondada; funções transcendentais como microprogramas fixos	bits idênticos em x86, ARM, RVV, interpretador e GPU
<code>:fast</code>	FMA fundida onde o operador permite	dentro do envelope rigoroso em todos os substratos

O envelope.

Para cada saída, `Vapor.Verify.Envelope` calcula em racionais diádicos exatos o valor real e uma cota válida para *qualquer* substrato conforme — arredondamento correto, FMA fundida ou não, qualquer ordem de redução, *underflow* gradual ou FTZ. Para contrações sobre entradas exatas usa a forma de Wilkinson $\gamma_n S + a$ (Equação 2.2), decidida pelo `within_envelope/4` extraído do Lean; para o resto, análise de erro corrente. O verificador nunca arredonda; só as cotas são encurtadas, e só para cima.

Versão da semântica.

Os bits de um programa são função dos seus termos *e* da definição das funções canônicas, e os certificados registram a versão. Na versão 1, $a/b = a \cdot \text{rcp}(b)$, a até 1 ulp; desde a versão 2 (rodada 0.6), a divisão é corretamente arredondada (Markstein com resíduo exato de Dekker, sem FMA), de modo que $+$, $-$, $\times$ e $\div$ são IEEE em todo substrato. As transcendentais são idênticas em todo substrato, mas não corretamente arredondadas (até 1–2 ulps, medido).

Para começar (graduação)

Reprodutível não é o mesmo que exato. Os bits canônicos do *vapor* não são “o valor real”: são *um* resultado em *binary32*, sempre o mesmo. O valor real está dentro do envelope. A garantia canônica responde a “por que deu diferente na outra máquina?” (não dá); o envelope responde a “quão longe da verdade pode estar?”.

3.6 ESCADA DE VERIFICAÇÃO E CERTIFICADO

Cada compilação atravessa seis degraus (Tabela 4). Os três primeiros são estáticos; os três seguintes executam o código nos substratos disponíveis.

Tabela 4 – A escada de verificação

degrau	estabelece	como
1	tipos, formas, realimentação	verificação do programa
—	reescrita exata	só identidades provadas
2	admissão	alocação aceita pelo verificador extraído em toda ISA; ausência de transbordamento int8 no máximo das extensões; limites do SPIR-V
3	identidade adjunta	$\langle Wx, v \rangle = \langle x, W^\top v \rangle$ entre o kernel compilado e a transposta exata (pega erros de <i>layout</i>)
4	oráculo diferencial	cada substrato de CPU igual, bit a bit, ao oráculo exato
5	paridade + envelope	resumos FNV-1a iguais entre substratos; toda saída dentro do envelope
6	certificado	assinatura Ed25519 com quórum por co-assinatura

O certificado contém o SHA-256 do programa, de cada *blob* de máquina e de cada módulo SPIR-V; o resumo das fontes Lean dos verificadores; as evidências de cada degrau; e o **trabalho contado** (FLOPs, bytes, instruções por ISA). Ele *não* contém nada que dependa do *host* — nem tempos, nem a decisão de despacho. Por isso nós independentes que refazem a escada produzem cargas *idênticas byte a byte*, codificadas em CBOR determinístico, e podem co-assinar; a verificação exige um quórum de assinaturas válidas de chaves distintas confiáveis. Um nó de borda confere assinaturas e *hashes* em cerca de 2 ms e executa sem refazer a escada.

Aprofundamento (mestrado)

Por que tirar a decisão de despacho do certificado. A versão original da diretriz do projeto pedia que o certificado registrasse em que substrato rodar. Isso tornaria o certificado dependente da máquina que o produziu e impediria a co-assinatura. O escrutínio da diretriz trocou a decisão pelo *trabalho contado*, que é portátil: cada nó toma a sua decisão a partir do trabalho certificado e do seu próprio perfil (seção 3.7). A lição geral: um certificado deve conter só o que outro verificador independente consegue reproduzir.

3.7 ÁRBITRO: O ROOFLINE DE TRÊS TETOS

O despachante escolhe o substrato pelo menor tempo previsto

$$t = \sum_{\text{regiões}} \max\left(\frac{F}{\pi}, \frac{Q}{\beta}, \frac{I}{\iota}\right) + \text{sobrecustos}, \quad (3.1)$$

em que F e Q são FLOPs e bytes contados do *schedule*, π e β os tetos de cálculo e de banda do perfil declarado, e I a contagem estática das instruções do corpo de cada laço quente *como emitido*, vezes o número de iterações, sobre a vazão de emissão ι . O terceiro teto é o que torna o modelo honesto: o GEMV de 4 bits é limitado por emissão de instruções (cerca de 1,1 instrução por peso), não por banda, e o modelo de dois tetos de Williams, Waterman e Patterson (2009) erra por cerca de $5\times$. Com os três, o GEMV 4096×4096 é previsto em 71,7 ms e observado em 72–75 ms.

3.8 MODELO DE GARANTIAS

Toda afirmação do vapor pertence a uma de quatro classes, e a documentação diz a qual (Figura 5).

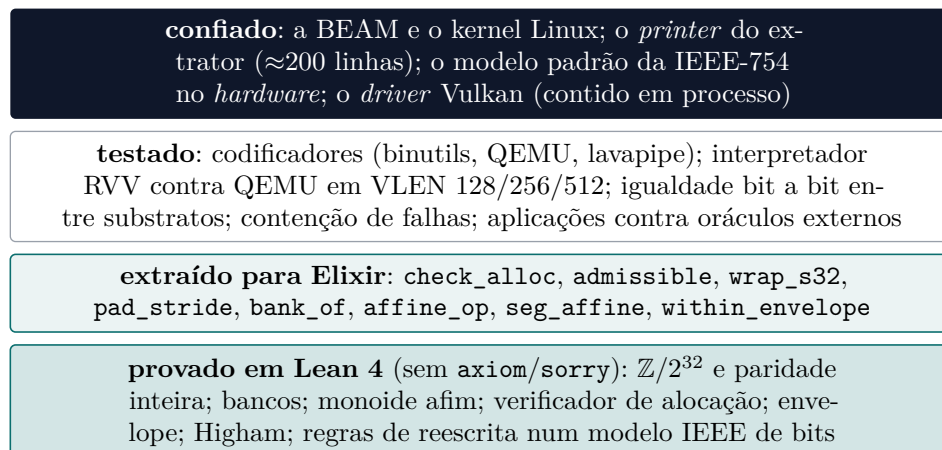


Figura 5 – As quatro classes de garantia, da base confiável (topo) ao núcleo provado (base).

O extrator lê os termos *elaborados* do Lean — os mesmos sobre os quais os teoremas falam — e falha em qualquer construção fora do fragmento suportado. O Lean calcula mais de 480 vetores de conformidade que o Elixir extraído precisa reproduzir, e o módulo gerado carrega o resumo das fontes Lean, conferido por um teste de auditoria sem precisar do *toolchain*. A extração é de tempo de *build*, não de execução: rodar o Lean em produção reintroduziria a pilha gigante que o projeto quer eliminar.

Questão em aberto (doutorado)

Quão pequena pode ser a base confiável de um compilador de tensores? O vapor confia no modelo padrão da IEEE-754 tal como implementado pelo *hardware* e no *printer* do extrator. A primeira suposição poderia ser trocada por testes exaustivos para binary32 (são 2^{64} pares para operações binárias — inviável) ou por uma especificação formal do *hardware*; a segunda, por um extrator verificado. Os codificadores de instruções são *testados* contra o binutils, que é ele mesmo não verificado. Uma linha de pesquisa natural é ligar os codificadores a uma semântica formal de ISA (como as

especificações Sail de ARM e RISC-V) e provar que a palavra emitida decodifica para a instrução pretendida. Fica em aberto se isso pode ser feito sem perder a propriedade `deps: []` e o custo de compilação baixo.

3.9 CAMADAS SOBRE O NÚCLEO

As camadas construídas nas treze rodadas usam o núcleo *sem alterá-lo*: tudo que produzem é programa certificado ou dado canônico (Tabela 5). Isso é uma regra de arquitetura verificada por teste — a rodada 0.6 encontrou e corrigiu uma violação real, em que o motor de geração lia a configuração de uma família de modelo.

Tabela 5 – Camadas sobre o núcleo e a garantia que cada uma herda

camada	conteúdo	garantia herdada
modelos (0.2–0.8)	Llama, Qwen, Mixtral, DeepSeek-V3, Gemma, Phi, Mamba, Whisper, ViT, CLIP, VAE, U-Net, DiT, pela eclusa de modelos	mesmos bits em todo subtrato, <i>thread</i> , lote e réplica
saída restrita (0.3)	gramáticas de bytes sobre o vocabulário (JSON Schema)	só tokens admitidos; nenhum prefixo aceito é beco sem saída
agentes e RAG (0.3)	diário encadeado + Merkle + atestado; índices determinísticos	repetir é verificar
documentos e OCR (0.4–0.8)	PDF, Office, JPEG, CCITT, JBIG2, tabelas	bit a bit iguais a libjpeg, libtiff, jbig2dec
estúdio (0.9)	grafos com cache por conteúdo	raiz de Merkle por execução
bancada, engenharia, lógica (0.12)	EDOs, circuitos, estruturas, SAT, LP	certificado calculado fora do <i>solver</i>
finanças e mesa (0.13)	curvas, opções, Monte Carlo, risco, <i>backtests</i> , arbitragem, livro de ofertas	compila para o núcleo (Monte Carlo); CBOR canônico e Merkle (diários)

A rodada 0.13 é a primeira desde a bancada (0.12) a voltar a *compilar para o núcleo*: o passo de uma trajetória de Monte Carlo, inclusive o gerador de números aleatórios, é escrito como termos da álgebra e herda a garantia central. O Capítulo 5 descreve como.

4 INOVAÇÕES

Este capítulo isola as inovações do **vapor**. Cada uma é apresentada com uma *ficha* que responde às cinco perguntas da Tabela 1 — o quê, para quem, para que, por que, como —, seguida da evidência que a sustenta e do seu limite. O critério de inclusão é exigente: só entra o que (i) não é prática corrente nos sistemas comparáveis do Capítulo 8, (ii) está implementado e testado no repositório e (iii) tem uma medida que poderia ter falhado.

4.1 NUMÉRICA CANÔNICA PORTÁTIL

Ficha 1 — Os mesmos bits em todo substrato

O quê. Uma semântica de ponto flutuante fixa bit a bit para todos os operadores da álgebra, inclusive divisão e funções transcendentais, realizada igualmente em x86-64 (AVX2, AVX-512), AArch64, RISC-V (VLEN 128 a 512), Vulkan e no oráculo.

Para quem. Quem precisa que um resultado numérico seja o mesmo em qualquer máquina: auditoria de modelos de IA, ciência reprodutível, risco financeiro regulado.

Para que. Para que “rodei de novo e deu diferente” deixe de ser possível e para que a comparação entre máquinas vire um teste de igualdade, não de tolerância.

Por que. Porque a irreprodutibilidade vem de escolhas arbitrárias (ordem de redução, FMA, implementação de exp), não de limitações físicas; fixadas as escolhas, a igualdade é de graça.

Como. Redução em árvore fixa de 16 acumuladores; multiplicação e soma arredondadas separadamente; divisão corretamente arredondada; transcendentais como microprogramas sobre $+$, $-$, $\times$; paralelismo só entre saídas independentes.

A literatura de reprodutibilidade numérica oferece duas respostas: somas reprodutíveis por pré-arredondamento ou acumuladores longos (DEMME; NGUYEN, 2015), que custam um fator constante em cada redução, e modos determinísticos de bibliotecas (PyTorch Contributors, 2024), que garantem igualdade só na mesma máquina e na mesma versão. O **vapor** toma um terceiro caminho: em vez de tornar a soma insensível à ordem, *fixa a ordem* e tira o paralelismo de outro lugar. Um GEMV tem milhares de linhas independentes; processar R linhas por iteração, cada uma com os seus 16 acumuladores, dá paralelismo de instrução e de *threads* sem tocar em nenhuma soma.

Evidência.

A garantia é testada em todas as formas de paralelismo do sistema (Tabela 6). Cada linha da tabela é um teste que compara bits, não uma tolerância.

Tabela 6 – Invariâncias testadas bit a bit

a resposta não depende de	como é testado
ISA	AVX2, AVX-512, NEON e RVV (sob QEMU) iguais ao oráculo
VLEN do RISC-V	128, 256 e 512 iguais
GPU	Vulkan (lavapipe) igual à CPU no modelo inteiro, com amostragem
número de <i>threads</i>	$1 = 2 = 3$ <i>threads</i>
lote	tokens de uma sequência iguais sozinha ou num lote contínuo, com qualquer fatiamento do <i>prefill</i>
lugar do cache	KV paginado igual ao contíguo
réplica	qualquer réplica dá a mesma resposta; réplicas entre nós de um <i>cluster</i> comparadas bit a bit
armazenamento dos pesos	bf16 residente igual a f32 sobre os pesos arredondados
especulação	saída da decodificação especulativa (linear e em árvore) igual à do alvo sozinho
esparsidade	MoE esperso (e em 4 bits) igual ao denso

Limite.

As transcendentais são iguais em todo lugar mas não corretamente arredondadas (1–2 ulps). A garantia é sobre os substratos medidos; um substrato novo precisa passar pela eclusa (seção 4.6). O texto normalizado depende da versão do Unicode do OTP, registrada mas não eliminada.

4.2 DESEMPENHO SEM REASSOCIAR

Ficha 2 — Velocidade que não troca bits

O quê. Técnicas de desempenho de fronteira — MoE esperso, cache latente (MLA), janela deslizante em anel, decodificação especulativa em árvore, paralelismo de tensor, GPU residente — cada uma reformulada para não mudar um bit.

Para quem. Operadores de inferência que hoje escolhem entre velocidade e reprodutibilidade.

Para que. Para que otimizar não exija revalidar.

Por que. Porque cada uma dessas técnicas é, no fundo, uma escolha de *quais* saídas calcular ou *onde* guardar dados, não de *como* somar.

Como. MoE por predicação de linha; paralelismo de tensor por coluna com *all-gather* (nunca por linha com *all-reduce*, que reassociaria); especulação que verifica $k + 1$ linhas do alvo num passo; janela como cache circular na tabela de blocos.

Evidência.

MoE esparsa: $2,6\times$ no *decode*, bits iguais ao denso. MLA latente: $85\times$ menos memória de cache. Janela em anel: $7,9\times$ mais sequências a 32k tokens. GPU com sessões residentes: 10–12 ms por token contra 72 ms sem sessão, os bits da CPU. GEMV f32 2048²: $2,24\times$ com 2 *threads* na mesma máquina de 2 vCPUs, sem mudar um bit.

Limite.

O paralelismo de tensor por linha com *all-reduce*, o mais comum na prática, é recusado; o custo é mais tráfego de rede. O GEMV de 4 bits continua limitado por emissão de instruções.

4.3 COMPILAR SEM CONFIAR NO COMPILADOR

Ficha 3 — Alocação sem *spill*, conferida por um verificador provado

O quê. Um compilador que emite bits de cinco alvos sem assembler, sem C e sem LLVM, em que cada alocação de registradores é aceita só depois de conferida por um verificador provado em Lean e extraído para Elixir.

Para quem. Pesquisadores de compiladores verificados; equipes que precisam de uma cadeia de compilação auditável e pequena.

Para que. Para que a fase mais propensa a erros sutis (alocação) não precise ser confiada.

Por que. Provar uma heurística é caro e frágil; provar o verificador é barato e vale para qualquer heurística.

Como. *Linear scan* com blocos alinhados e o fator de grupo g (o LMUL do RVV generalizado a todas as ISAs); sem caminho de *spill* — falta de registrador muda a forma do código; `checkAlloc_sound` em Lean.

O “sem *spill*” é uma escolha de primeiros princípios. Um *spill* é uma escrita e uma leitura de memória introduzidas pelo compilador, invisíveis no programa-fonte e difíceis de contar no trabalho certificado. Mudando a forma do código (menos linhas por iteração, grupos menores, fronteiras de região), o compilador mantém o trabalho contado exato e o código sempre dentro dos registradores. A consequência medida está no árbitro: o modelo de três tetos acerta o GEMV de 4 bits porque conta as instruções *como emitidas* (seção 3.7).

Limite.

O verificador extraído é quadrático; compilar programas grandes desenrolados é lento (cerca de 0,6 s por passo e por ISA no Monte Carlo da rodada 0.13).

4.4 CONTENÇÃO TOTAL DO CÓDIGO GERADO

Ficha 4 — Nenhum byte emitido executa na máquina virtual

O quê. Todo código gerado roda em processos do sistema operacional com `seccomp`, `W^X` e *watchdog*; funções nativas na VM são proibidas por um teste de auditoria.

Para quem. Quem serve modelos ou executa código gerado num serviço de longa duração.

Para que. Para que um erro do compilador, do *driver* ou da GPU custe uma requisição, não o serviço.

Por que. Porque o compilador não é provado inteiro; a contenção limita o que um erro dele pode fazer.

Como. Quadros com comprimento prefixado; tabelas dimensionadas pelo quadro; reinício supervisionado; *failover* que não muda bits.

Evidência.

As quatro classes de falha (`SIGILL`, `SIGSEGV`, `SIGALRM`, `SIGSYS`) são provocadas nos testes; em todas só o worker morre e a unidade seguinte roda. Um “interpretador seguro” em função nativa foi *recusado* no escrutínio da diretriz: compartilharia o espaço de endereçamento da VM.

Limite.

O worker é Linux; um worker para macOS (com `MAP_JIT`, sem `seccomp`) não foi escrito. Sob `qemu-user` o `seccomp` não está disponível (o isolamento por processo continua).

4.5 CERTIFICADOS PORTÁTEIS, CO-ASSINÁVEIS E TRANSPARENTES

Ficha 5 — Um certificado que nós independentes reproduzem byte a byte

O quê. Um certificado Ed25519 da compilação, sem nenhum dado dependente do *host*, codificado em CBOR determinístico, que vários nós podem refazer e co-assinar, e que pode ser ancorado num log de transparência.

Para quem. Cadeias de suprimento de modelos; auditores; reguladores (os dossiês mapeiam cada evidência a dispositivos do AI Act europeu e da ISO/IEC 42001, e dizem onde não há nenhuma).

Para que. Para que confiar num binário signifique confiar num quórum, não num fornecedor.

Por que. Porque um certificado só tem valor se outro verificador independente consegue

reproduzi-lo.

Como. Trabalho contado em vez de decisão de despacho; CBOR (RFC 8949 §4.2) em vez de `term_to_binary`; log Merkle RFC 9162 com *checkpoints* C2SP e testemunhas que recusam retrocesso e bifurcação.

Evidência.

Payloads idênticos entre nós; verificação de borda em cerca de 2 ms; num dossiê, nenhuma de 209 alterações de um bit é aceita; o verificador do log roda no próprio navegador.

4.6 ECLUSAS: ADMISSÃO POR MEDIÇÃO

Ficha 6 — Nada entra sem ser medido na fronteira

O quê. Três eclusas com a mesma forma: a de *modelos* transforma um *checkpoint* num contrato conferido; a de *substratos* mede um acelerador antes de lhe dar programas; a de *documentos* lê arquivos com a proveniência de cada trecho até a página e o *hash*.

Para quem. Quem integra componentes de terceiros — modelos, aceleradores, documentos.

Para que. Para que o núcleo não conheça famílias, fornecedores ou formatos, e para que a recusa venha com o motivo e o reparo.

Por que. Porque o custo de integração cresce com o que o núcleo precisa saber; uma eclusa concentra esse conhecimento num lugar só.

Como. Contratos (`:causal_lm`, `:encoder`, `:codec`, `:map`); adaptadores por dados (JSON), *blueprint* ou topologia; sondas de resposta conhecida com impressão numérica; leitores próprios de zip, PDF, Office, JPEG, CCITT, JBIG2.

Evidência.

Os adaptadores novos são conferidos contra o próprio `transformers` — o que achou um defeito real de *rotary* parcial, admitido e calculado errado em silêncio, num caminho de referência. O decodificador JPEG é igual ao `libjpeg` bit a bit; CCITT G3/G4, ao `libtiff`; JBIG2, ao `jbig2dec` (43 de 43 fluxos).

4.7 UMA BUSCA PROPÕE, UM VERIFICADOR DECIDE — COM CONTROLES

Ficha 7 — Toda resposta com o seu juiz, todo juiz com o seu controle

O quê. Um padrão de arquitetura aplicado a domínios inteiros: o procedimento que produz uma resposta (busca, *solver*, modelo de linguagem, pessoa) é separado do que a confere, e cada verificação é acompanhada de um *controle* — o valor que uma implementação quebrada produziria — que mostra que ela poderia ter falhado.

Para quem. Quem usa saídas de modelos generativos ou de buscas heurísticas e precisa separar sinal de ruído.

Para que. Para que “a saída parece certa” seja substituída por “a saída foi conferida, e a conferência tem poder”.

Por que. Porque um verificador sem controle pode estar aceitando tudo; e porque a proposta pode vir de qualquer lugar — inclusive de um modelo de linguagem — se o juiz for independente dela.

Como. Portões calibrados que *recusam existir* se os controles negativos e positivos se sobrepõem; verificações com valor, controle, limiar e veredito; verificadores pequenos (refutações DRUP, KKT, Kirchhoff, Farkas, motor ingênuo); propostas recebidas pelo MCP.

O padrão aparece em três níveis. No **nível de portão**, um detector de ruído (de texto, imagem ou áudio) é ajustado a controles negativos (ruído, sinal embaralhado, repetição) e positivos (sinal real retido) e se recusa a julgar se a margem entre eles for nula. No **nível de verificação**, a suíte de qualidade (`mix_vapor.quality`) tem, em cada linha, um controle; na rodada 0.12 foram 147 verificações, todas aprovadas, e a rodada 0.13 acrescenta 23 (Capítulo 7). No **nível de domínio**, cada mesa (lógica, engenharia, ciência, finanças) devolve com a resposta um certificado calculado fora do *solver* — e aceita a *proposta* de qualquer um, conferindo só ela.

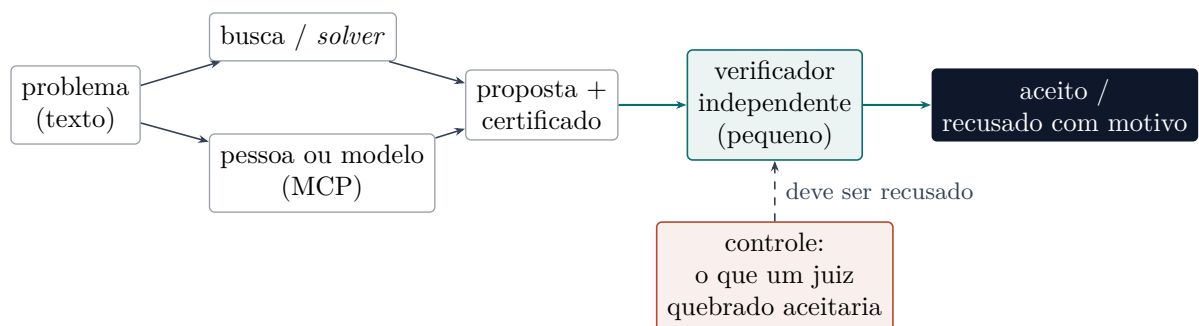


Figura 6 – O padrão proponente/verificador com controle. A busca pode ser complexa e não confiável; o verificador é pequeno e independente; o controle mostra que ele tem poder.

Evidência.

Na lógica, a satisfatibilidade de $S(3) = 13$ é decidida com refutação DRUP e a refutação truncada é rejeitada. Na engenharia, o MEF com elemento QM6 fica a 0,9% da viga analítica enquanto o Q4 trava a 29% — e o certificado de equilíbrio é calculado fora do *solver*. Nas finanças, o juiz ingênuo do livro de ofertas achou um defeito real que o motor de produção e o próprio juiz compartilhavam (seção 5.9). A Figura 7 mostra a mesa de lógica no console.

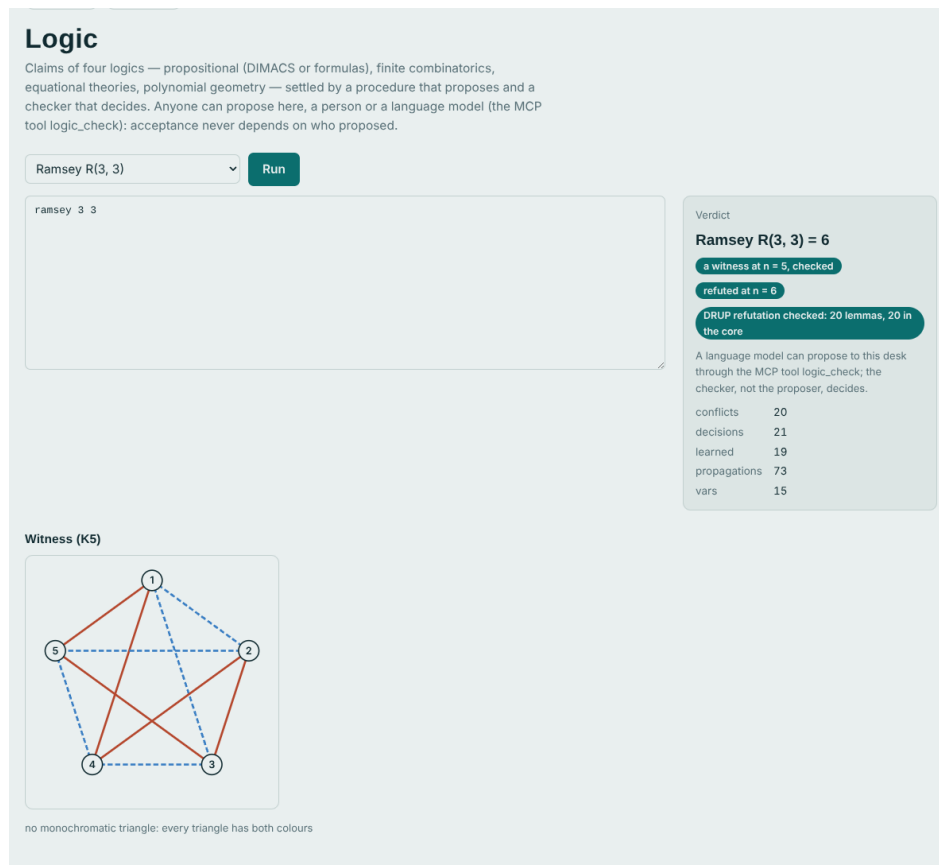


Figura 7 – A mesa de lógica no console: a afirmação, a decisão e a prova conferível.

Limite.

O padrão exige que exista um verificador mais simples que a busca. Isso vale para problemas em NP e para a otimização convexa (dualidade), mas não para toda pergunta: “esta estratégia vai dar lucro?” não tem certificado — tem, no máximo, portões que dizem quando um *backtest* é indistinguível de ruído (seção 5.7).

4.8 REPETIR É VERIFICAR

Ficha 8 — Execuções que são objetos de prova

O quê. Agentes, grafos do estúdio, sessões de bolsa e arquivos salvos cuja execução produz um diário encadeado por *hash*, fechado por uma raiz de Merkle e atestado, de modo que reexecutar é a forma de conferir.

Para quem. Quem precisa auditar o que um agente ou um processo automatizado fez.

Para que. Para que “o que aconteceu” seja recalculável, não narrado.

Por que. Porque, com bits canônicos, toda decisão local é re-derivável em qualquer substrato.

Como. Diário gravado antes de cada passo; repetição que recalcula sem agir; retomada do disco sem repetir ações; apagamento por titular.

A rodada 0.13 estende a ideia ao mercado: a mesma semente de uma sessão de bolsa reproduz *a mesma cabeça de hash* do diário, e outra semente, outra cabeça. Os arquivos salvos das mesas passam a ser *assinados* (Ed25519 sobre "vapor-archive-v1\n" seguido do manifesto), e um manifesto reescrito é recusado.

4.9 PRIMEIROS PRINCÍPIOS: AS INVERSÕES

As inovações acima têm uma raiz comum: cada uma inverte uma suposição que a prática trata como dada. A Tabela 7 as reúne.

Tabela 7 – As suposições usuais e a inversão do vapor

suposição usual	inversão
reprodutibilidade custa desempenho	o paralelismo vem de saídas independentes; a ordem da soma é fixa e de graça
o compilador é confiável ou é provado inteiro	o compilador é não confiável; uma fase difícil é conferida por um verificador provado
código gerado roda no processo do <i>runtime</i>	nenhum byte gerado roda na VM
o certificado diz onde rodar	o certificado diz o trabalho; cada nó decide
um modelo é aceito porque carrega	um modelo é aceito porque cumpre um contrato conferido contra a referência
uma saída boa é uma saída plausível	uma saída boa passou num juiz que tem poder (controle)
um <i>backtest</i> é um modelo da bolsa	o <i>backtest</i> é o código da bolsa (seção 5.12)
antecipação se evita com disciplina	antecipação se <i>certifica</i> ausente, por invariância de prefixo (seção 5.7)
arbitragem se estima	arbitragem se decide, em racionais, pelo lema de Farkas (seção 5.8)

Questão em aberto (doutorado)

Uma teoria dos controles. A prática de acompanhar cada verificação de um controle que “deveria ser recusado” lembra o *mutation testing* e a análise de poder estatístico, mas não é nenhum dos dois: o controle é *escolhido* para representar o erro mais plausível (esquecer o termo de Itô, contar dias corridos em vez de úteis, normalizar pela amostra inteira). Falta uma teoria que diga quais controles bastam para dar a um conjunto de verificações uma garantia de cobertura — por exemplo, uma noção de “controle adversarial mínimo” por classe de erro, com cotas sobre a probabilidade de um defeito passar despercebido. O **vapor** oferece um corpus de 170 pares (verificação, controle) em domínios variados como ponto de partida empírico.

5 FINANÇAS VERIFICÁVEIS: A RODADA

0.13

Finanças é o domínio em que “o número saiu do computador” vale menos. Um preço que muda quando o cálculo muda de servidor, um *backtest* que olhou o dia seguinte sem querer, a melhor de cinquenta estratégias apresentada como se fosse a única, um livro de ofertas que ninguém de fora consegue auditar: são as dores reais da indústria, e todas são dores de **verificabilidade**, não de velocidade. Este capítulo descreve como a rodada 0.13 aplica a tese do Capítulo 1 a esse domínio.

Para começar (graduação)

Glossário mínimo. Uma *mesa* é a equipe (e o sistema) que precifica e negocia instrumentos. *P&L* é o resultado (*profit and loss*). Um *backtest* simula uma estratégia sobre dados históricos. *Bid* e *ask* são os melhores preços de compra e de venda oferecidos; a diferença é o *spread*. Uma *ordem limitada* só executa ao preço dado ou melhor; uma *ordem a mercado*, ao que houver. HFT (*high-frequency trading*) é a negociação automatizada em escalas de micro a nanossegundos.

5.1 A MESA COMO ARQUITETURA

A Figura 8 mostra o desenho comum a todas as tarefas. O usuário escreve o problema como uma mesa o escreveria — `di1 F26 = 15.02%, call S=100 K=100 T=1 r=5% vol=20%, sell 50 @ 101.30 owner=bia` —; um analisador próprio o lê; um motor o resolve; e a resposta sai com um certificado calculado por um caminho diferente do que a produziu. A mesma entrada chega por quatro portas: a API Elixir (`Vapor.Finance.run/2`), a linha de comando (`mix vapor.finance`), o console web (*Mercados*) e o servidor MCP, pelo qual um modelo de linguagem pode pedir um cálculo ou submeter uma proposta para conferência.

A Tabela 8 resume as tarefas e o objeto que permite julgar cada resposta. As seções seguintes descrevem cada uma.

5.2 DINHEIRO EXATO E CALENDÁRIOS POR REGRA

Dinheiro.

Um valor monetário é um inteiro e uma escala decimal: $\{c = 12345, e = 2\}$ representa 123,45. Soma, subtração e produto são exatos; todo arredondamento é um ato

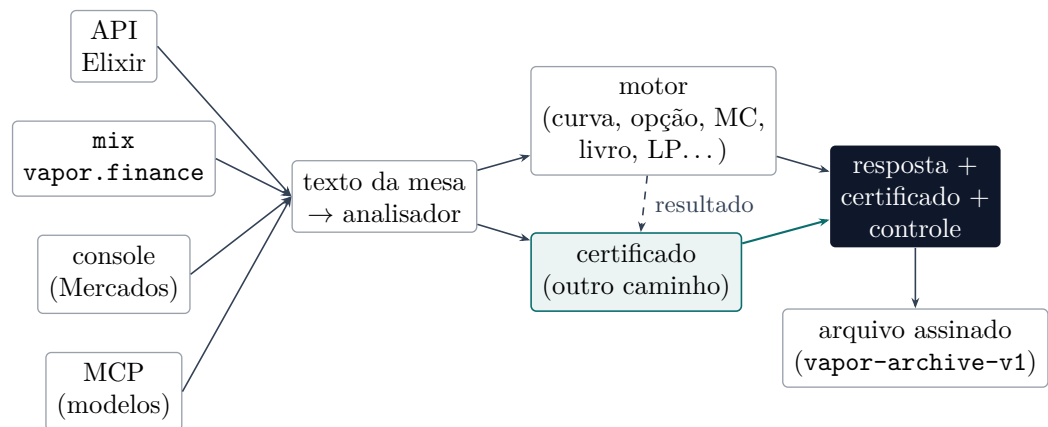


Figura 8 – A mesa de finanças: quatro portas, um analisador, um motor e um certificado calculado por outro caminho.

Tabela 8 – Tarefas da mesa e o certificado de cada uma

tarefa	certificado
calendário e dinheiro	feriados por regra, iguais ao QuantLib por 89 anos; rateio que soma exatamente
curva	todo instrumento reprecificado; <i>forwards</i> negativos apontados
opções	paridade; gregas contra diferenças finitas; limites de não arbitragem <i>antes</i> de resolver; $g(k)$ de Durrleman
Monte Carlo	bits = oráculo exato = outra contagem de <i>threads</i> ; intervalo de 99% contra a forma fechada
risco	Kupiec, Christoffersen, zona de Basileia; tamanho e poder medidos
carteira	condições KKT; contribuições de risco iguais aos orçamentos
<i>backtest</i>	quatro portões: invariância de prefixo, Sharpe deflacionado, PBO, <i>Reality Check</i>
arbitragem	<i>ou</i> o portfólio de arbitragem <i>ou</i> os preços de estado, em racionais exatos
livro de ofertas	diário SHA-256 + raiz de Merkle; motor ingênuo independente; feed ITCH; FIX
sessão de bolsa	os três certificados do livro + a mesma cabeça de <i>hash</i> ao repetir
microestrutura	teste de reescala do tempo; o artigo reproduzido; forma fechada = ótimo numérico

explícito com modo nomeado (*half-even, half-up, down, floor...*), implementado por uma única primitiva testada nos empates dos dois lados do zero. O rateio usa o método do *maior resto* (Hamilton): R\$ 100,00 em três partes dá 33,34 + 33,33 + 33,33, e a soma é o total *por construção*; o certificado diz também que cada parte está a menos de uma unidade da sua fração exata. O controle: o mesmo rateio por piso em binary64 perde um centavo (99,99).

O fator da convenção brasileira, $(1 + r)^{DU/252}$, é calculado com uma raiz inteira em aritmética de inteiros grandes e *truncado* na oitava casa, como faz a ANBIMA. Para $r = 13,65\%$ e um dia útil, $(1,1365)^{1/252} = 1,00050788\dots$; o módulo `decimal` do Python com 60 dígitos concorda — a decisão na oitava casa não é um acidente de ponto flutuante.

Calendários.

Cada feriado é uma *regra*: data fixa, n -ésimo dia da semana, ou festa móvel pelo cômputo gregoriano. A Páscoa sai do algoritmo anônimo de 1876 (Meeus/Jones/Butcher); o Carnaval é a Páscoa menos 48 e 47 dias; a Sexta-feira Santa, menos 2; Corpus Christi, mais 60. Os fechamentos que nenhuma regra produz (11 de setembro de 2001, o furacão Sandy, dias de luto presidencial) são *dados*, cada um com o motivo. A Tabela 9 mostra a conferência contra o QuantLib ([The QuantLib Contributors, 2026](#)).

Tabela 9 – Calendários conferidos dia a dia contra o QuantLib 1.43

calendário	regras	feriados em dia útil idênticos
ANBIMA (= B3)	nacionais + móveis; 20 de novembro desde 2024 (Lei 14.759/2023)	903 (1990–2078)
NYSE	datas observadas; MLK desde 1998; Juneteenth desde 2022; fechamentos especiais	848
TARGET	Sexta Santa, Segunda de Páscoa, 1º de maio e 26 de dezembro desde 2000	401

Na primeira comparação, a NYSE divergiu num único dia — 27/04/1994, o luto pelo presidente Nixon — e o TARGET em datas anteriores a 2000, quando o sistema não existia. As duas diferenças viraram dado e regra, e o teste passou a exigir igualdade dia a dia. As seis bases de contagem (DU/252, ACT/360, ACT/365F, 30/360, 30E/360, ACT/ACT ISDA) concordam com o QuantLib a 10^{-14} nos casos de borda.

Para começar (graduação)

Por que dinheiro não é double. 0,1 não tem representação binária finita. Somar dez vezes 0,10 em binary64 não dá exatamente 1,00, e um rateio feito com pisos em binary64 pode perder um centavo. O erro é minúsculo por operação, mas é um erro *contábil*: o total não fecha. Inteiros com escala decimal fecham por construção.

5.3 CURVAS DE JUROS

A curva é escrita como uma mesa a escreve (Figura 9). O *bootstrap* põe um nó no último fluxo de cada instrumento e acha, pelo método de Brent, o fator de desconto que o reprecifica, com os nós anteriores fixos. A interpolação é *flat-forward* (log-linear no desconto, a convenção da curva DI) ou linear na taxa zero. Os fluxos seguem as convenções publicadas: o DI1 tem $PU\ 100\ 000/(1+r)^{DU/252}$; a LTN paga 1 000 no dia útil seguinte ao vencimento; a NTN-F paga cupom semestral $1\ 000 \cdot (\sqrt{1,1} - 1)$.

O certificado

reprecifica cada instrumento a partir da curva pronta e dá o pior erro relativo ($4,5 \cdot 10^{-16}$ na curva DI do exemplo); lista os *forwards* entre nós e *aponta os negativos*. O exemplo “cotação inconsistente” (janeiro a 15%, julho a 9%) é reprecificado, mas sai com o *forward* negativo nomeado: reprecificar não basta para uma curva ser plausível, e o certificado diz as duas coisas. O controle: o mesmo DI1 contado em dias corridos erra o PU em $2,5 \cdot 10^{-3}$. Depósitos simples conferem com o `PiecewiseLogLinearDiscount` do QuantLib a 10^{-13} .

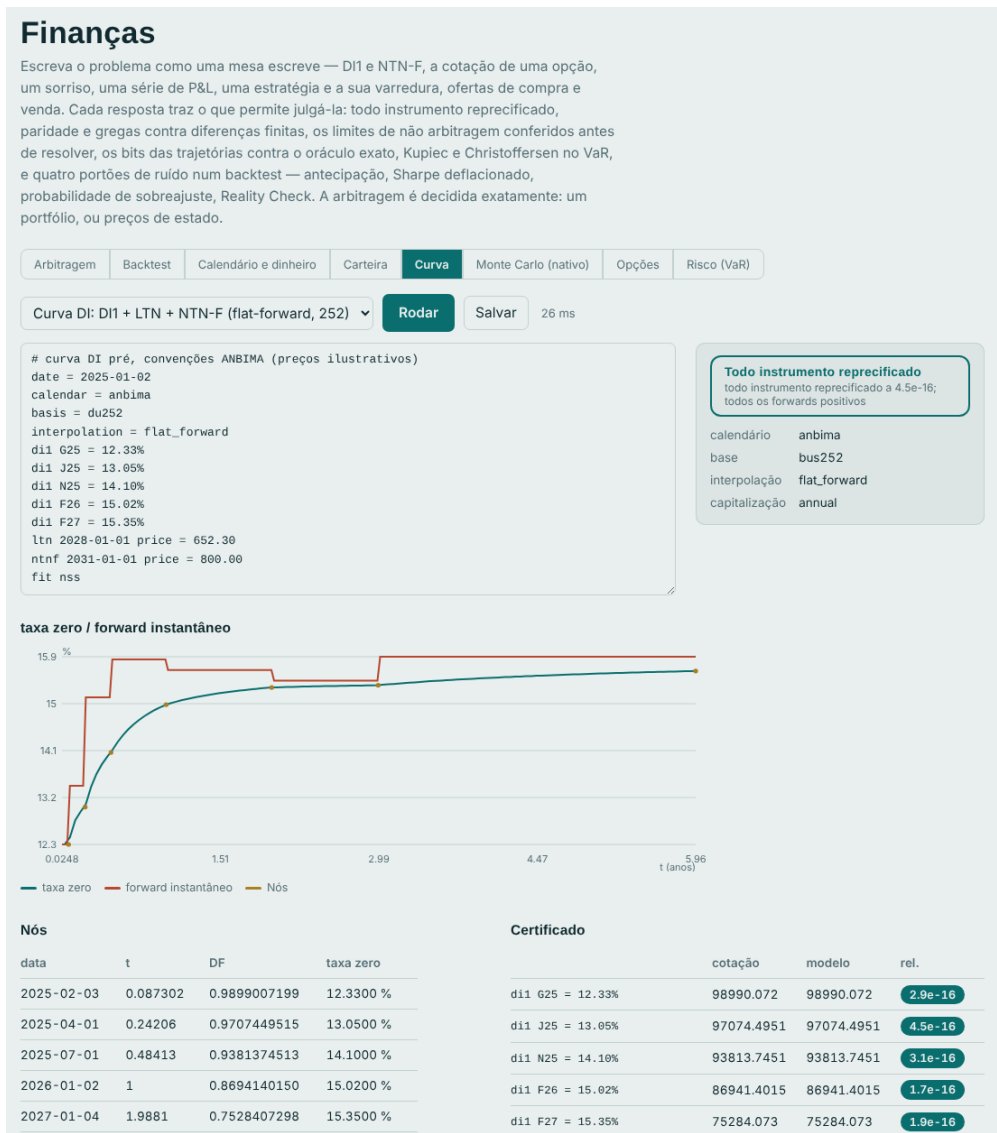


Figura 9 – Curva DI (DI1 + LTN + NTN-F, *flat-forward*, DU/252) no console: taxa zero, *forward* instantâneo, nós e o certificado de reprecificação.

5.4 OPÇÕES

O módulo de opções cobre Black–Scholes–Merton com dividendos (BLACK; SCHOLLES, 1973; MERTON, 1973), Black-76, Bachelier (taxas negativas), árvores de Cox–Ross–Rubinstein e Leisen–Reimer (LEISEN; REIMER, 1996), Heston (HESTON, 1993) pela integral de Lewis com a função característica “*little trap*” (ALBRECHER et al., 2007), e o sorriso SVI (GATHERAL; JACQUIER, 2014). Duas decisões de projeto merecem destaque.

Limites antes de resolver.

A volatilidade implícita é a raiz de $\text{BSM}(\sigma) = p$. Para uma *call*, BSM é estritamente crescente em σ e

$$\max\left(Se^{-qT} - Ke^{-rT}, 0\right) < \text{BSM}(\sigma) < Se^{-qT}, \quad \sigma \in (0, \infty). \quad (5.1)$$

Um preço fora desse intervalo não tem volatilidade implícita — e não é “mal condicionado”: é uma arbitragem. O *vapor* confere os limites *antes* de chamar o *solver* e recusa com o limite violado; dentro deles, devolve o erro de reprecificação, a vega e quanto um *tick* de preço move σ (o condicionamento, em unidades de mercado).

Arbitragem de borboleta no sorriso.

Uma fatia do SVI dá a variância total $w(k)$ em função do *log-strike* k . Gatheral e Jacquier (2014) mostram que a densidade neutra ao risco implícita é não negativa se e só se

$$g(k) = \left(1 - \frac{k w'(k)}{2 w(k)}\right)^2 - \frac{w'(k)^2}{4} \left(\frac{1}{w(k)} + \frac{1}{4}\right) + \frac{w''(k)}{2} \geq 0. \quad (5.2)$$

O *vapor* avalia g numa grade fina e devolve o intervalo em que ela é negativa. A fatia que Gatheral e Jacquier (2014) atribuem a Vogt — o exemplo clássico de arbitragem de borboleta — é detectada (g mínimo $-0,033$ em $k \in [0,645; 1,255]$); um sorriso calmo ajustado sai com $g \geq 0$ (Figura 10).

Contra o QuantLib.

BSM, Δ e Γ a 10^{-12} , vega a 10^{-10} ; americana de Leisen–Reimer com 801 passos a 10^{-10} (4,486076, a *put* da tabela 1 de Longstaff e Schwartz (2001)); Heston a 10^{-9} (o integrador adaptativo de Gauss–Kronrod do QuantLib e a Gauss–Legendre composta daqui concordam até a nona casa). O CRR daqui é o de livro-texto e o do QuantLib usa a aproximação aditiva em log; diferem em $1,6 \cdot 10^{-5}$, e a documentação diz qual é qual. O LSM de Longstaff e Schwartz (2001) sobre a mesma *put* dá $4,4788 \pm 0,0143$ (o artigo: 4,472).

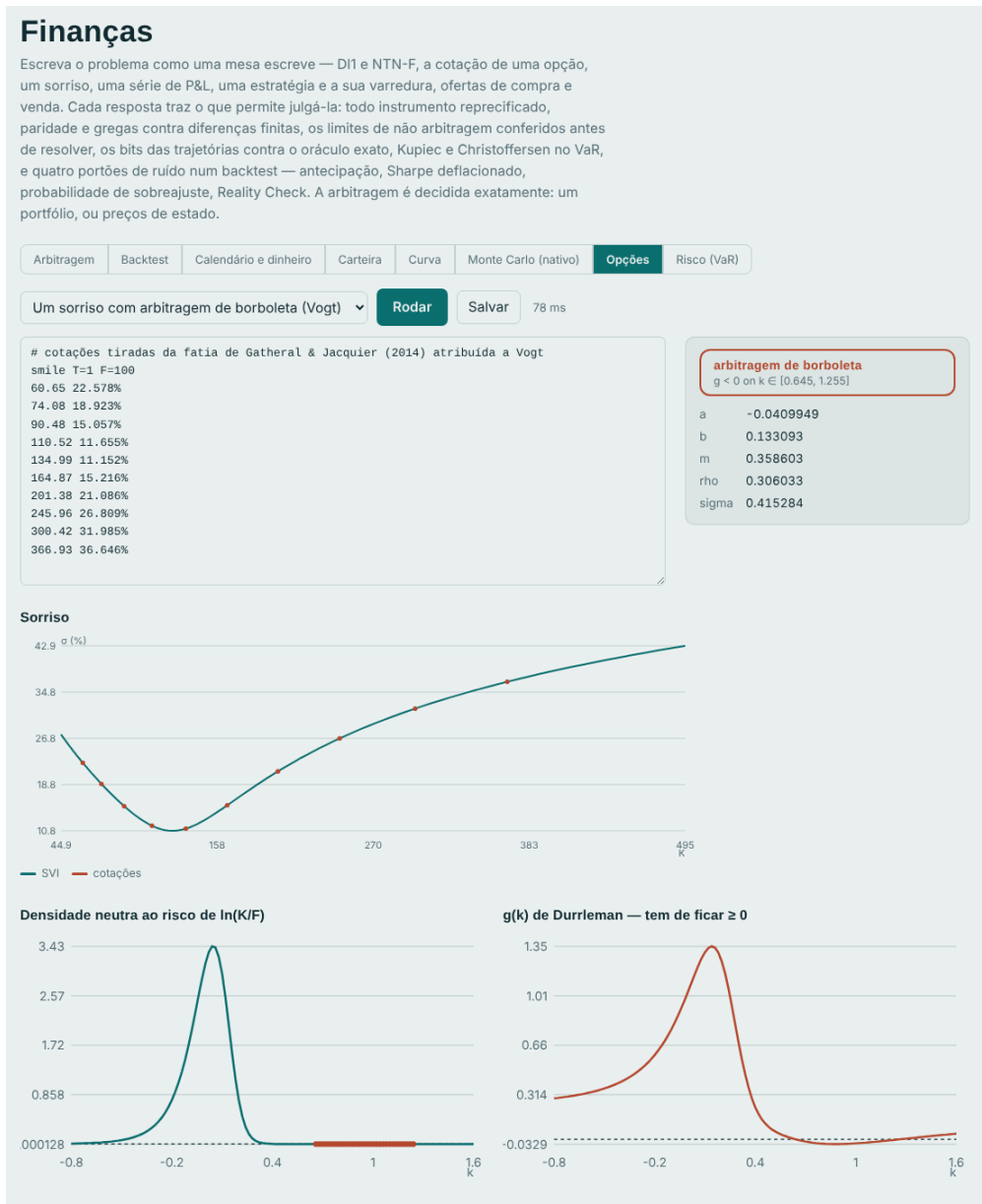


Figura 10 – O sorriso de Vogt: o ajuste SVI, a densidade neutra ao risco (negativa onde há arbitragem) e $g(k)$ de Durrleman abaixo de zero.

5.5 MONTE CARLO COMPILADO PARA O NÚCLEO

A dor: um número de risco que muda quando o cálculo muda de máquina, de contagem de *threads* ou para uma GPU — reduções paralelas em outra ordem, uma FMA aqui e não ali, outro exp. A resposta do **vapor**: escrever o passo da trajetória como termos da álgebra e herdar a garantia central.

O passo.

Para o log-preço L , com $\mu\Delta = (r - q - \sigma^2/2)\Delta$ e $\sigma\sqrt{\Delta}$ pré-calculados, o programa recorrente atualiza, por trajetória e por passo,

$$L \leftarrow L + \mu\Delta + \sigma\sqrt{\Delta} \Phi^{-1}(u), \quad A \leftarrow A + e^L, \quad G \leftarrow G + L, \quad v \leftarrow v \cdot [L > \ln(B/S_0)], \quad (5.3)$$

em que A e G acumulam as médias aritmética e geométrica (opções asiáticas) e v marca as trajetórias vivas acima de uma barreira B . Φ^{-1} é o algoritmo AS241 de [Wichura \(1988\)](#) (PPND7, 7 dígitos — o que binary32 comporta), com log e rsqrt canônicos.

O gerador dentro da álgebra.

Para que a trajetória inteira rode no worker, o gerador também precisa ser termo da álgebra — e exato em binary32. O **vapor** usa o gerador de [Wichmann e Hill \(1982\)](#): três geradores de Lehmer

$$s_1 \leftarrow 171 s_1 \bmod 30269, \quad s_2 \leftarrow 172 s_2 \bmod 30307, \quad s_3 \leftarrow 170 s_3 \bmod 30323, \quad (5.4)$$

combinados como $u = (s_1/30269 + s_2/30307 + s_3/30323) \bmod 1$.

Proposição 5.1. *Cada atualização de estado do gerador de Wichmann–Hill é calculada exatamente em binary32 pelo programa*

$$p = a \cdot s, \quad q = \lfloor p \oslash m \rfloor, \quad s' = p - m \cdot q,$$

em que $\oslash$ é a divisão corretamente arredondada e $\lfloor y \rfloor$ é calculado por $r = (y \oplus 2^{23}) \ominus 2^{23}$, seguido de $r \ominus 1$ quando $r > y$.

Esboço. Os produtos $a \cdot s$ são inteiros menores que $171 \cdot 30268 = 5\,175\,828 < 2^{23}$ (e analogamente para os outros dois), logo representáveis exatamente com 24 bits de significando: p é exato. Para $0 \leq y < 2^{23}$, somar 2^{23} leva y ao binade $[2^{23}, 2^{24})$, em que o ulp é 1; o arredondamento ao mais próximo produz o inteiro mais próximo de y , e subtrair 2^{23} é exato (Sterbenz). Se o arredondamento subiu, $r > y$ e a correção desce um. O quociente p/m nunca é inteiro (pois m é primo e $0 < s < m$) e, para estes três módulos, nunca arredonda para o inteiro seguinte — o que se confere exaustivamente, para os cerca de

91 mil estados possíveis —, de modo que $\lfloor p \oslash m \rfloor = \lfloor p/m \rfloor$; dois seletores no programa corrigem s' para $[0, m)$ por segurança. Finalmente, $m \cdot q \leq p < 2^{23}$ é exato e a subtração de inteiros abaixo de 2^{23} também. $\square$

O resultado é que a BEAM só semeia: sementes, geração, transformação normal, passo e acumulação rodam no worker, com os mesmos bits em todo substrato.

Certificados, na mesma resposta.

(i) A primeira chamada, compilada para 64 pistas e rodada pelo *oráculo exato*, é igual bit a bit às 64 primeiras pistas do worker. (ii) A simulação inteira, repetida num worker com 2 *threads*, dá os mesmos bits. (iii) As mesmas uniformes em binary64 na BEAM, com Φ^{-1} de precisão total, diferem no máximo $2 \cdot 10^{-5}$ no *payoff*. (iv) O intervalo de 99% contém a forma fechada — BSM para a europeia, [Kemna e Vorst \(1990\)](#) para a asiática geométrica discreta. (v) A asiática aritmética usa a geométrica como variável de controle, com variância dividida por cerca de 1 300. **O controle:** esquecer o $-\sigma^2/2$ de Itô em $\mu\Delta$ dá $z = 8,7$ — o estimador viciado é pego, não promediado.

Desempenho.

Numa máquina de 2 vCPUs com AVX-512, um núcleo do worker: 8 192 trajetórias $\times$ 64 passos em 125 ms, contra 750 ms estimados para a BEAM em binary64 (6 $\times$); 16 384 $\times$ 32 em 86 ms contra 1 950 ms (23 $\times$). A compilação custa cerca de 0,6 s por passo desenrolado e por ISA, por causa do verificador de alocação quadrático (seção 4.3); por isso o programa tem um passo por chamada e compila só para a ISA da máquina.

Limite.

Wichmann–Hill é um gerador antigo: período de cerca de $7 \cdot 10^{12}$ e falha em baterias modernas como a BigCrush ([L'ECUYER; SIMARD, 2007](#)). Serve à precificação, não à criptografia nem a estudos de cauda extrema; a opção `rng: :host` usa *splitmix64* na BEAM e compara.

Questão em aberto (doutorado)

Geradores modernos exatos em binary32. A restrição “produtos abaixo de 2^{23} ” exclui os geradores de qualidade atual (PCG, Philox, xoshiro), que dependem de aritmética inteira de 32 ou 64 bits. A álgebra do vapor tem um gerador `gemm_i8` em $\mathbb{Z}/2^{32}\mathbb{Z}$, com paridade inteira provada em Lean. Fica em aberto se um gerador contrabaseado como o Philox ([SALMON et al., 2011](#)) pode ser expresso com esses operadores inteiros, mantendo a exatidão e a igualdade em GPU, e qual o custo comparado ao Wichmann–Hill em ponto flutuante.

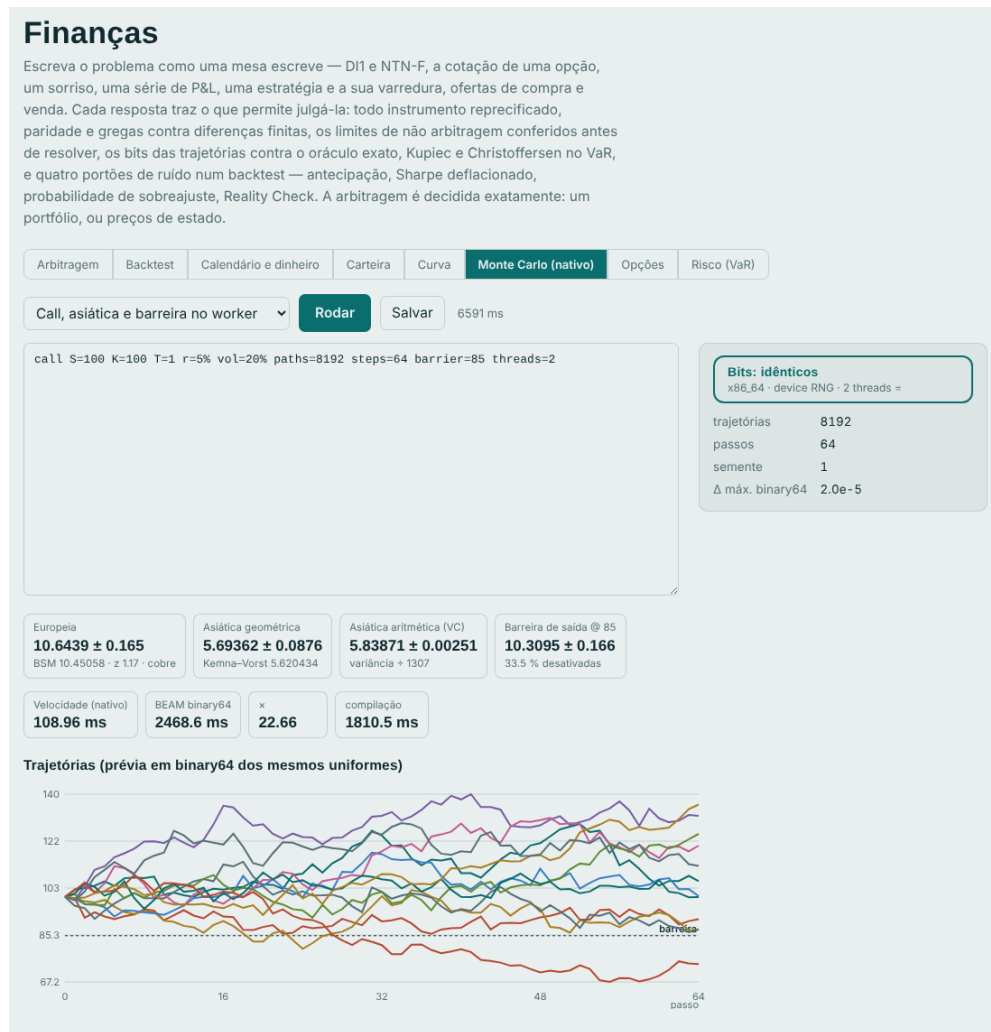


Figura 11 – Monte Carlo no worker nativo: europeia, asiáticas e barreira, com o selo de bits idênticos (oráculo e 2 threads) e a comparação com a BEAM em binary64.

5.6 RISCO DE MERCADO E CARTEIRAS

O VaR é calculado pelos métodos histórico, normal, Cornish–Fisher e EWMA ($\lambda = 0,94$), em janela móvel, e testado. Se x exceções ocorrem em n dias para um VaR de cobertura $1 - p$, a estatística de [Kupiec \(1995\)](#) é

$$\text{LR}_{\text{pof}} = -2 \ln \left[(1 - p)^{n-x} p^x \right] + 2 \ln \left[\left(1 - \frac{x}{n} \right)^{n-x} \left(\frac{x}{n} \right)^x \right] \sim \chi_1^2, \quad (5.5)$$

e a de [Christoffersen \(1998\)](#) testa, por uma cadeia de Markov de dois estados, se as exceções se agrupam. A zona de Basileia (verde 0–4, amarela 5–9, vermelha ≥ 10 exceções em 250 dias a 99%) acompanha ([Basel Committee on Banking Supervision, 1996](#)).

Tamanho e poder.

Um teste que nunca rejeita é tão suspeito quanto um que sempre rejeita. O vapor mede o *tamanho* do Kupiec sorteando exceções a exatamente 1% em 400 séries (rejeita

em 1,5–10% delas) e o seu *poder* numa série de retornos t de Student com $\nu = 3$: o VaR normal é rejeitado ($p = 0,038$) e o histórico não ($p = 0,43$) (Figura 12).

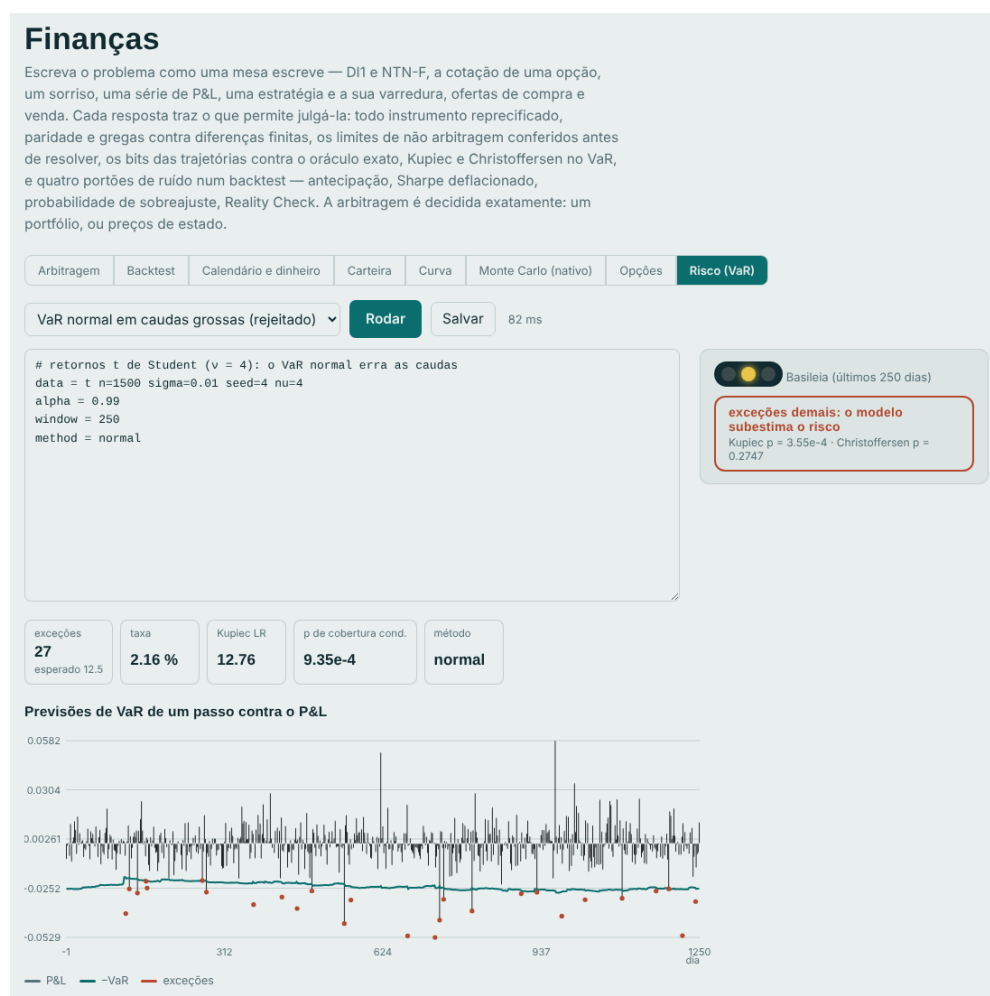


Figura 12 – VaR normal em caudas grossas: as previsões de um passo, as exceções, os testes de Kupiec e Christoffersen e a zona de Basileia.

Carteiras.

Covariância encolhida de Ledoit e Wolf (2004); *variância mínima só compra* por conjunto ativo, com as condições KKT como certificado (estacionaridade abaixo de 10^{-12} , multiplicadores com o sinal certo); *paridade de risco* por Newton na formulação convexa de Spinu (2013), com contribuições iguais aos orçamentos a 10^{-12} ; e HRP de López de Prado (2016).

5.7 BACKTESTS COM PORTÕES DE RUÍDO

Um *backtest* é uma máquina de produzir índices de Sharpe; tentando variantes bastantes, uma parecerá boa sobre puro ruído. As duas falhas silenciosas da indústria são

a **antecipação** (o sinal usou informação que não teria) e a **seleção** (a melhor de muitas tentativas apresentada como a única) (BAILEY et al., 2017; HARVEY; LIU; ZHU, 2016). O **vapor** trata as duas com quatro portões.

A estratégia é escrita numa linguagem pequena:

```
data = ar1 n=5040 phi=0.15 sigma=0.01 seed=3
sweep k = 1..3
signal = sign(sma(ret(close), k))
cost = 1bp
```

Os operadores são causais (`lag`, `diff`, `ret`, `sma`, `ema`, `std`, `zscore`, `rsi`, `sign`, `if...`), mas a linguagem aceita, porque as pessoas os escrevem, três que espiam: `lead`, `center` e `normalize` pela amostra inteira. A posição em $(t, t + 1]$ é o sinal em t ; o custo incide sobre $|\Delta\text{posição}|$.

5.7.1 Portão 1: invariância de prefixo

Definição 5.1 (Causalidade observável). *Seja f o pipeline que leva uma série de preços $x_{0..n}$ a uma série de sinais $f(x)_{0..n}$. Diz-se que f é causal se, para todo $c \leq n$ e todo $t \leq c$, $f(x_{0..c})_t = f(x_{0..n})_t$.*

Proposição 5.2. *Se f é causal, então para qualquer conjunto de cortes C o teste “ $f(x_{0..c})_{0..c} = f(x_{0..n})_{0..c}$ bit a bit para todo $c \in C$ ” aceita. Reciprocamente, se o teste rejeita em c , existe $t \leq c$ em que o sinal calculado com a história inteira difere do que seria calculado com os dados disponíveis em c : f usa informação posterior a c .*

A proposição é imediata, e é o seu caráter de *caixa-preta* que a torna útil: o teste não precisa conhecer nem confiar nos operadores — trata o *pipeline* inteiro, inclusive o analisador e o avaliador, como função. Um primeiro rascunho da ideia conferia só os operadores, e teria deixado passar o `center`, que é causal em cada passo e não causal no todo (a média é da amostra inteira). Por isso o teste é sobre o resultado, não sobre a gramática. A igualdade é bit a bit porque o avaliador é determinístico; com aritmética não reproduzível, seria preciso uma tolerância, e uma antecipação pequena poderia se esconder nela. O **vapor** usa oito cortes e devolve o *dia* em que o sinal muda.

Aprofundamento (mestrado)

Compleitude. O teste é correto (nunca acusa um *pipeline* causal) mas não completo: uma antecipação que só se manifeste entre dois cortes, ou que mude o sinal sem mudar o seu valor arredondado, passa. Com $C = \{0, \dots, n\}$ o teste é completo para a definição acima, ao custo de n avaliações. Os oito cortes são um compromisso; como o custo de cada avaliação é linear, cortes adicionais podem ser pedidos.

5.7.2 Portões 2–4: seleção

O **Sharpe deflacionado** (BAILEY; López de Prado, 2014) é a probabilidade de o Sharpe verdadeiro exceder o máximo esperado de N tentativas sem habilidade:

$$\text{DSR} = \Phi \left(\frac{(\widehat{SR} - SR_0) \sqrt{T-1}}{\sqrt{1 - \hat{\gamma}_3 \widehat{SR} + \frac{\hat{\gamma}_4 - 1}{4} \widehat{SR}^2}} \right), \quad (5.6)$$

$$SR_0 = \sqrt{V} \left[(1 - \gamma) \Phi^{-1} \left(1 - \frac{1}{N} \right) + \gamma \Phi^{-1} \left(1 - \frac{1}{Ne} \right) \right], \quad (5.7)$$

em que V é a variância dos Sharpes entre tentativas, $\gamma \approx 0,5772$ é a constante de Euler–Mascheroni e $\hat{\gamma}_3, \hat{\gamma}_4$ são assimetria e curtose. N inclui as tentativas declaradas antes do arquivo (`trials =`). A **probabilidade de sobreajuste** (PBO) por validação cruzada combinatória (BAILEY et al., 2017) divide a série em 16 blocos e, nas $\binom{16}{8} = 12\,870$ metades, mede com que frequência o campeão dentro da amostra cai abaixo da mediana fora dela. O **Reality Check** de White (2000) testa se a melhor estratégia supera a nula com o *bootstrap* estacionário de Politis e Romano (1994).

Medido.

O melhor de 30 cruzamentos de médias sobre um passeio aleatório tem Sharpe positivo e DSR 0,10 — reprovado (Figura 13). O sinal plantado (momento sobre AR(1) com $\phi = 0,15$) passa os quatro portões (DSR 1,0; PBO < 0,5; *Reality Check* $p < 0,05$). `sign(lead(close) - close)` tem Sharpe anual 20 e é pego no dia 89.

5.8 ARBITRAGEM DECIDIDA EXATAMENTE

Proposição 5.3 (Lema de Farkas, forma de mercado). *Sejam n ativos com pagamentos $X \in \mathbb{Q}^{n \times S}$ em S estados e cotações $\text{bid}_i \leq \text{ask}_i$. Exatamente uma das alternativas vale:*

1. *existe um portfólio — compras $b \geq 0$ ao ask e vendas $s \geq 0$ ao bid — com custo $\text{ask}^\top b - \text{bid}^\top s \leq 0$, pagamento $X^\top(b - s) \geq 0$ em todo estado, e pelo menos uma das desigualdades estrita;*
2. *existem preços de estado $\psi \in \mathbb{Q}^S$, $\psi > 0$, com $\text{bid}_i \leq (X\psi)_i \leq \text{ask}_i$ para todo i .*

É o teorema fundamental da precificação (HARRISON; KREPS, 1979; JOUINI; KALLAL, 1995) visto como teorema da alternativa (FARKAS, 1902). O vapor acha os dois lados com um **simplex racional exato** (duas fases, regra de Bland contra ciclagem) e devolve o objeto que prova a alternativa encontrada, conferido *só por multiplicação* de racionais:



Figura 13 – Trinta cruzamentos de médias sobre ruído: o melhor parece bom, e os portões dizem que é ruído (DSR 0,10; *Reality Check* $p = 0,78$).

- uma arbitragem: o portfólio, o custo e o pagamento em cada estado. No exemplo da *call* mal precificada, vender um título, comprar 1/90 da ação e vender 1/60 da *call* recebe 1/15 hoje e paga 0 nos dois estados;
- ausência de arbitragem: o vetor $\psi > 0$ — (1/2, 9/20) no binomial do exemplo.

Aprofundamento (mestrado)

Por que racionais. Em ponto flutuante, a decisão entre as duas alternativas é instável na fronteira: um custo de -10^{-17} é arbitragem ou erro de arredondamento? Em racionais, a resposta é exata, e o certificado é conferido por quem quiser com aritmética de inteiros. O custo é que coeficientes irracionais (como e^{-rT}) entram arredondados a 15 casas — e a resposta diz isso. O mesmo simplex racional, com certificados dual, de Farkas e de raio ilimitado, resolve qualquer programa linear na mesa de lógica

(*maximize/minimize*), fechando um item da lista de pendências da rodada 0.12.

Calls entre *strikes*.

Para cotações de *calls* num vencimento, os estados são 0, cada *strike*, $2K_{\max}$ e a *inclinação* além dele. Como o pagamento de qualquer combinação estática é linear por partes com vértices nos *strikes*, conferir os vértices e a inclinação confere todo preço terminal: a decisão é completa para portfólios estáticos (Figura 14). Câmbio: todos os ciclos simples de até cinco conversões, com o produto das taxas em racionais.

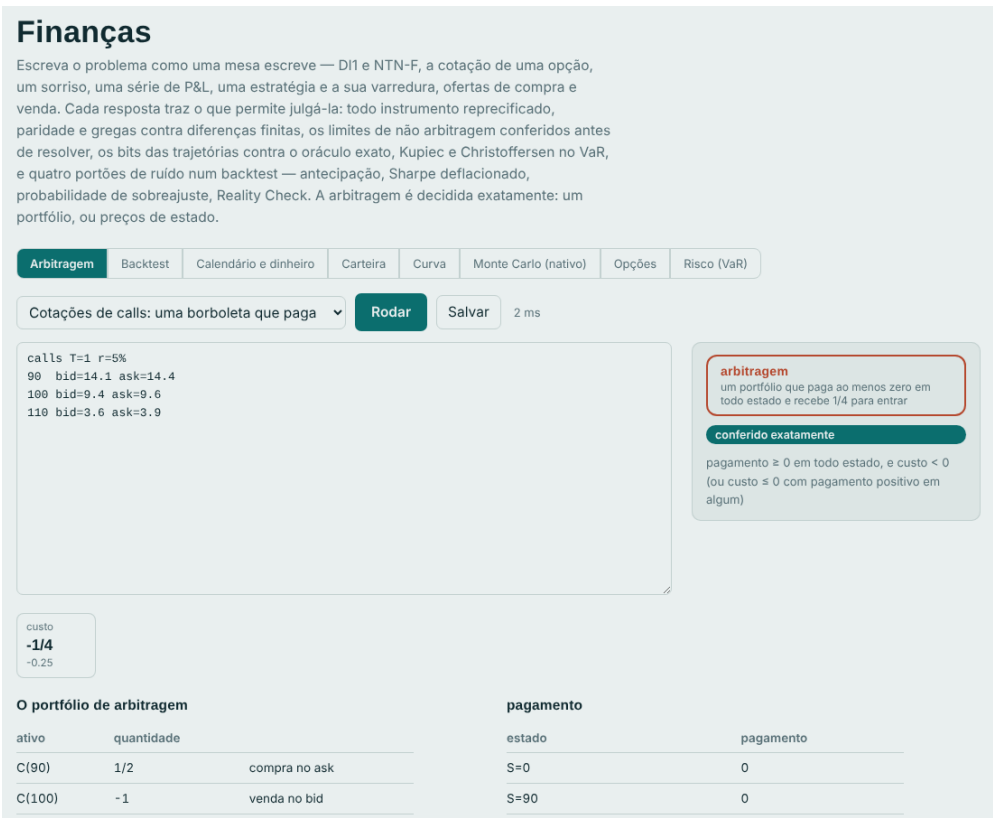


Figura 14 – Cotações de *calls* com uma borboleta que paga: o portfólio de arbitragem, o custo exato e o pagamento em cada vértice.

Humano ou IA no loop.

A ferramenta MCP `arbitrage_check` recebe a *proposta* de qualquer um — um portfólio que alguém diz ser arbitragem, ou preços de estado que alguém diz provarem que não há — e só a aritmética exata decide. O controle do teste: “compre a ação e venda a *call*” paga nos dois estados mas custa 86,5 — não é arbitragem, e a mesa diz por quê.

5.9 O LIVRO DE OFERTAS COMO OBJETO VERIFICÁVEL

O motor de casamento implementa prioridade preço-tempo; ordens limitadas e a mercado; validades GTC, IOC e FOK; *post-only*; cancelamento; alteração (reduzir a quantidade no mesmo preço mantém a prioridade; qualquer outra mudança é cancelar e substituir); prevenção de autonegociação; e *kill switch* por dono. Preços são *ticks* inteiros e quantidades são inteiras: nada arredonda. O motor é uma **função pura** (livro, evento) $\mapsto$ (livro', relatórios).

Diário encadeado.

Cada entrada é $h_i = \text{SHA256}(h_{i-1} \parallel \text{CBOR}(\text{evento}_i, \text{relatórios}_i))$; a sessão fecha com uma raiz de Merkle sobre as entradas (folhas RFC 6962), e uma prova de inclusão mostra que um negócio pertence à sessão sem revelar os outros — o que um regulador ou um cliente pede sem acesso ao livro inteiro.

O juiz independente.

O verificador (`Book.Check`) não compartilha nada com o motor além da especificação: o livro é uma lista, a prioridade é uma ordenação por (preço, tempo), cada casamento varre a lista. Do diário sozinho, ele refaz a cadeia de *hashes*, *reexecuta cada evento no motor ingênuo* e *exige os mesmos relatórios*, e confere nove invariantes (Figura 15).

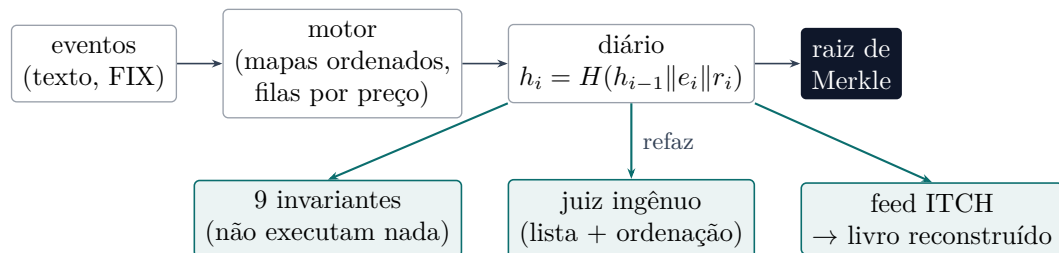


Figura 15 – O livro de ofertas e os seus três verificadores independentes: o motor ingênuo, os invariantes e o feed de mercado.

Os invariantes são: preço do negócio igual ao do *maker* e dentro do limite do agressor; o *maker* é o primeiro na prioridade naquele instante; quantidades conservadas (executado + repousado + cancelado = ordenado); livro nunca cruzado; FOK tudo ou nada; IOC e ordem a mercado nunca repousam; *post-only* nunca agride; nenhum negócio entre ordens do mesmo dono sob prevenção; e cadeia íntegra.

O defeito que o juiz achou.

O *fuzzing* diferencial roda 6 000 eventos aleatórios por política de prevenção, comparando motor e juiz. Na primeira rodada, o juiz apontou `:fok_partial`: uma ordem

FOK a mercado virava IOC nos *dois* motores — a mesma leitura errada da especificação, escrita duas vezes. A reexecução não a pegou, porque os dois motores concordavam; só o invariante, que não executa nada, viu. O defeito foi corrigido nos dois motores e o invariante ficou. O episódio ilustra por que o juiz tem duas camadas de natureza diferente: reexecução (pega divergência de implementação) e invariantes (pegam erro de especificação comum).

Adulteração.

Um preço trocado num diário quebra a cadeia. O mesmo negócio forjado *com o hash recalculado* — uma mentira coerente — passa na cadeia e cai na reexecução.

Desempenho, honestamente.

Cerca de 8,5 μ s por evento na mediana (p99 de cerca de 100 μ s), *incluindo* o SHA-256 e a codificação canônica de cada entrada; o juiz reexecuta 12 500 eventos em cerca de 150 ms. A BEAM é tempo real brando: esses números servem a um casamento de mesa, a um internalizador, a um simulador de *backtest* — não a uma bolsa de nanossegundos.

5.10 PROTOCOLOS E RISCO PRÉ-NEGOCIAÇÃO

ITCH 5.0.

Codificador e decodificador das mensagens S, R, A, F, E, C, X, D, U e P da especificação da Nasdaq (Nasdaq, 2020), com os comprimentos exatos (12 a 44 bytes, *big-endian*, preço com 4 casas implícitas, *timestamp* de 6 bytes em nanossegundos). A sessão gera o feed que uma bolsa publicaria, e **rebuild/1** reconstrói o livro *só a partir do feed* — que tem de ser igual ao do motor, nível a nível, e com o mesmo volume executado: um terceiro certificado independente (o feed só vê o que repousa; o motor vê tudo).

FIX 4.4.

Mensagens *tag=valor* com *BodyLength* (9) e *Checksum* (10) conferidos na leitura e calculados na escrita — iguais ao **simplefix** byte a byte (FIX Trading Community, 2003). *NewOrderSingle*, *OrderCancelRequest* e *OrderCancelReplaceRequest* viram eventos do livro; os relatórios viram *ExecutionReports* com numeração própria.

Risco pré-negociação.

O que a Regra 15c3-5 da SEC (U.S. Securities and Exchange Commission, 2010) e o RTS 6 da MiFID II (Comissão Europeia, 2017) pedem a quem tem acesso ao mercado —



Figura 16 – O livro de ofertas no console: profundidade, o diário encadeado (cada entrada selecionável), a prova de Merkle do primeiro negócio, o feed ITCH e os relatórios FIX.

e que possa mostrar que tinha: quantidade máxima (“dedo gordo”), nocional máximo, colar de preço contra a referência, posição máxima no pior caso (executado + ordens abertas daquele lado + esta), taxa de mensagens numa janela e *kill switch*. Uma recusa nunca chega ao livro e *entra no diário* com a causa. O verificador refaz posições, ordens abertas, referências e contagens a partir do diário e confere que nenhuma ordem aceita violou um limite e que toda recusa teve a causa dita. O controle atribui a uma recusa por quantidade a causa “colar de preço”, e é pego.

5.11 MICROESTRUTURA, CADA MODELO COM A MEDIDA QUE DIZ SE SERVE

Hawkes.

Chegadas que se excitam, com intensidade $\lambda(t) = \mu + \sum_{t_i < t} \alpha e^{-\beta(t-t_i)}$ (HAWKES, 1971; BACRY; MASTROMATTEO; MUZY, 2015): simulação por *thinning* de Ogata (1981), máxima verossimilhança pela recursão $O(n)$, e o **teste de reescala do tempo**: sob o modelo certo, os incrementos do compensador $\Lambda(t_i) - \Lambda(t_{i-1})$ são $\text{Exp}(1)$, julgados por Kolmogorov–Smirnov. Plantado $(\mu, \alpha, \beta) = (1; 0,6; 1,5)$, ajustado $(1,08; 0,61; 1,69)$, razão de ramificação 0,36 (plantada 0,40), KS $p = 0,73$. O controle — um Poisson ajustado às mesmas chegadas — tem $p = 1,7 \cdot 10^{-20}$ (Figura 17).

Avellaneda–Stoikov.

O formador de mercado de Avellaneda e Stoikov (2008) cota em torno do preço de reserva $r = s - q\gamma\sigma^2(T - t)$ com *spread* total $\gamma\sigma^2(T - t) + \frac{2}{\gamma} \ln(1 + \frac{\gamma}{k})$. A simulação do §4 do artigo ($s_0 = 100$, $T = 1$, $\sigma = 2$, $dt = 0,005$, $k = 1,5$, $A = 140$, 1 000 trajetórias, $\gamma = 0,1$) dá, daqui contra o artigo, $\sigma(\text{P\&L})$ de 6,44 / 5,89 para a estratégia de estoque e 13,57 / 13,43 para a simétrica. O *spread* médio daqui (1,49) inclui o termo $\gamma\sigma^2(T - t)$ que a tabela do artigo parece omitir (1,29 é só o segundo termo) — dito, não escondido.

Almgren–Chriss.

A trajetória ótima de liquidação $x_j = X \sinh(\kappa(T - t_j)) / \sinh(\kappa T)$ (ALMGREN; CHRISS, 2000), com κ da relação discreta $\cosh(\kappa\tau) = 1 + \tilde{\kappa}^2\tau^2/2$, é *certificada* resolvendo o mesmo problema de média–variância numericamente (um sistema tridiagonal): diferença relativa 10^{-16} .



Figura 17 – Hawkes: os parâmetros plantados e ajustados, o Poisson como controle e o teste de reescala do tempo.

5.12 UMA SESSÃO DE BOLSA QUE SE AUDITA

A sessão de bolsa junta tudo: formadores de mercado cotando *post-only* pela regra de Avellaneda–Stoikov em torno do preço público do passo anterior; agressores de ruído cujas chegadas seguem um Hawkes exato em tempo contínuo; um operador informado que conhece o fundamental e agride quando o livro se afasta dele. Toda ordem passa pelo mesmo portão pré-negociação e pelo mesmo motor que uma implantação usaria — **o backtest é o código da bolsa**, não um modelo dela.

A sessão devolve a própria auditoria (Figura 18): o diário conferido pelo motor ingênuo, os limites conferidos a partir do diário, o feed ITCH reconstruindo o mesmo livro, e a mesma semente reproduzindo a mesma cabeça de *hash*. E a microestrutura medida na fita recupera o que foi plantado: o ajuste de Hawkes das chegadas acha a autoexcitação (ramificação 0,37 contra 0,40; o Poisson rejeitado), e o *spread* de Roll (5,66) fica próximo do cotado (5,85).



Figura 18 – Uma sessão de 1 500 passos: 12 436 eventos refeitos pelo motor ingênuo, limites pré-negociação conferidos, feed ITCH igual ao livro; preços, estoques dos formadores e profundidade.

Questão em aberto (doutorado)

Simuladores de mercado verificáveis. Simuladores baseados em agentes (BYRD; HYBINETTE; BALCH, 2020) são usados para treinar e avaliar estratégias, mas o seu motor de casamento raramente é o de uma bolsa real, e os resultados raramente são reproduzíveis entre versões. A sessão do **vapor** mostra que é possível ter as duas coisas — o motor de produção e a reprodutibilidade por *hash* —, mas numa escala de microssegundos. Ficam em aberto: (i) se o motor, compilado como programa canônico ou escrito no worker, pode chegar à escala de nanossegundos mantendo o diário e o juiz fora do caminho crítico; (ii) como calibrar os agentes a dados reais de alta frequência de modo que a microestrutura medida (Hawkes, Roll, Kyle, assinatura da variância) seja um teste de aderência do simulador, e não só uma ilustração.

5.13 O QUE ESTE CAPÍTULO NÃO AFIRMA

- **Latência de bolsa.** O motor roda na BEAM: microssegundos por evento com *hash*, não nanossegundos. O que se afirma é a verificabilidade e a identidade entre *backtest* e motor.
- **Dados reais.** Nenhum arquivo ITCH da Nasdaq e nenhuma curva da ANBIMA foram usados; os preços dos exemplos são ilustrativos. Os formatos são os das especificações e as convenções as publicadas; os oráculos são o QuantLib, o **simplefix**, o SciPy e o ngspice, não os dados.
- **Conselho de investimento.** Os portões dizem quando um *backtest* é indistinguível de ruído; passar neles não torna uma estratégia lucrativa (custos reais, capacidade, mudança de regime).
- **Modelos completos de mercado.** Não há XVA, modelos de taxa de juros de vários fatores (Hull–White, LMM), volatilidade local ou estocástica calibrada a uma superfície inteira, nem crédito.

6 A BANCADA ABERTA: A RODADA 0.14

As rodadas 0.11 e 0.12 aplicaram a separação “uma busca propõe, um verificador decide” a domínios escolhidos: redes de ordenação, geometria, engenharia, lógica, tabuleiros. Cada domínio era uma mesa com o seu formato de entrada e os seus exemplos. A crítica que abriu a rodada 0.14 foi direta: o módulo de ciência apresentava *problemas já resolvidos*; as cenas aceitavam só ações pré-determinadas; o sistema era expositivo. Este capítulo descreve a resposta — trocar N mesas por **uma linguagem, uma fornalha e uma pedra de toque** — e o que as verificações com controle disseram dela.

Para começar (graduação)

Glossário mínimo. Um *espaço de busca* é o conjunto de candidatos possíveis (permutações, subconjuntos, grafos...). Uma *meta-heurística* (recozimento simulado, algoritmo evolutivo) explora esse espaço sem garantia de ótimo. Um *processo gaussiano* é um modelo estatístico de uma função desconhecida, usado para escolher o próximo ponto a medir quando cada medida é cara. Um *holdout* é um conjunto de dados reservado que a busca não vê, usado para descobrir se o vencedor só venceu por acaso.

6.1 ESCRUTÍNIO DO PEDIDO

O pedido pedia “uma grande coletânea” de buscadores famosos, generalizados. Tomado ao pé da letra, isso produziria N ferramentas estreitas com N formatos — exatamente a falta de liberdade criticada. O denominador comum desses buscadores é um laço de três passos: *proponha* um candidato, *avalie-o* com um verificador, *guarde* o resultado com a prova. O produto, então, é o caso geral; os casos famosos viram *exemplos de partida* escritos na mesma linguagem que o usuário usa (régua de Golomb, Ramsey, a conjectura de Euler, redes de ordenação, caixeiro-viajante, portfólios, hiperparâmetros, exemplos adversariais, regras de *trading*, fórmulas). Os nomes do sistema seguem a metáfora do laboratório alquímico — *Alembic*, *Athanor*, *Touchstone*, *Crucible*, *Assay* — e nenhum nome de produto de terceiros é usado.

Dois riscos do pedido foram tratados como requisitos. **Entrada aberta** sem sanitização é uma porta de ataque; sanitização por lista de palavras é frágil. “**AI em tudo**” sem verificação é autoridade sem prova. As duas seções seguintes mostram as respostas.

6.2 ALEMBIC: UMA LINGUAGEM PURA E SANEADA

Alembic é uma linguagem de expressões pura: inteiros de tamanho arbitrário, listas, tuplas, mapas, compreensões, *lambdas*, *pipes* e cerca de 120 funções embutidas, sem E/S, sem relógio e sem aleatoriedade fora de **noise** e **hash**, que são determinísticas. Um programa é, portanto, uma função do seu texto — o que torna reproduzível o diário de toda busca que o use.

A sanitização é feita pela *semântica*, não por filtros (Tabela 10): todo passo paga combustível; tamanhos têm teto; a avaliação roda num processo com teto de memória que a máquina virtual mata; identificadores nunca viram átomos (a tabela de átomos da BEAM não é coletada); dados são lidos por um leitor que recusa código; e, no navegador, as expressões das cenas são uma árvore JSON *interpretada*, nunca avaliada como JavaScript. A mesma análise achou um vazamento antigo: `Vapor.Expr.compile/2` criava um módulo BEAM (e um átomo) por expressão nova; agora há um teto, e o excedente é interpretado sem criar átomo algum.

Tabela 10 – Os limites da Alembic e o teste que tenta quebrar cada um

risco	defesa	teste
laço infinito	combustível em toda chamada e iteração	recursão sem fim e fold gigante recusados
números enormes	inteiros até 65 536 bits	2^{10^8} recusado
memória	processo com <code>max_heap_size</code>	bomba de memória morta sob 16 MB; o chamador vive
átomos	identificadores ficam binários	2 000 nomes novos: 0 átomos
código como dado	leitor de literais	f(1) e compreensões recusados
injeção no navegador	árvore interpretada	funções não numéricas recusadas

6.3 ATHANOR E TOUCHSTONE

Um problema Athanor é um programa Alembic com nomes reservados: **space** (dez construtores: **bits**, **ints**, **reals**, **perm**, **subset**, **subsets**, **seq**, **graph**, **partition** e **program**, árvores de expressão), um objetivo (**minimize**, **maximize**) ou uma afirmação (**claim**), e opcionalmente **violation** (a distância até a validade, que ordena os inválidos e deixa a busca subir até eles), **holdout**, **describe** (para um arquivo MAP-Elites), **neighbor** e **measured**.

A fornalha reparte o orçamento entre estratégias — exaustiva retomável, aleatória, recozimento, evolução, CMA-ES, otimização bayesiana com processo gaussiano Matérn-5/2, *mente* (um modelo de linguagem) e *humano* — por um bandido UCB descontado ($\gamma = 0,97$). A primeira versão, com UCB simples, gastava demais na aleatória; sem a

aleatória, perdia nos problemas rugosos; o desconto resolveu os dois. Toda corrida termina com três objetos que a tornam julgável:

1. o **controle**: o mesmo orçamento gasto em amostras uniformes; se o acaso nunca alcança o melhor, a regra de três dá o limite superior da chance por amostra;
2. o **holdout**: os finalistas reavaliados num objetivo não visto, com a correlação de postos de Spearman e o máximo esperado do ruído, $\sqrt{2 \ln N}$;
3. o **diário**: uma cadeia SHA-256 sobre a codificação canônica de cada avaliação, cuja raiz depende só do texto e da semente.

A **Touchstone** reconfere um certificado sem confiar na busca: reavalia o candidato, confere pertença e valor; com **full**, refaz a enumeração; com **replay**, refaz a corrida e compara a raiz do diário. “Provado” significa exatamente *enumeração completa de um espaço finito*, e o certificado diz o tamanho.

Humano e modelo no laço.

Uma sessão corre sob supervisão; a pessoa observa, propõe candidatos, fixa e bane finalistas, estende o orçamento ou fornece medidas de um objetivo externo (um experimento, um treino). Um modelo de linguagem pode *formalizar* um problema descrito em palavras — o rascunho é compilado, reparado até três vezes com o erro do compilador e devolvido com uma *retrotradução* para a pessoa conferir — e pode *propor* candidatos. As propostas, venham de onde vierem, entram pela mesma porta e são julgadas pelo mesmo verificador. O modelo nunca decide.

6.4 CRUCIBLE: EVIDÊNCIA SEM GABARITO

Os experimentos científicos das rodadas anteriores comparavam a saída com uma resposta conhecida — útil como calibração, inútil para quem traz um sistema novo, cuja resposta ninguém conhece. O Crucible aceita o sistema do usuário (EDOs, potenciais, hamiltonianos, reações, sequências, moléculas, dados) e responde com evidência que *não precisa de gabarito*: a ordem de convergência observada por Richardson; teoremas que valem para qualquer entrada (virial, Ehrenfest, trabalho–energia); dois métodos independentes que precisam concordar; e controles que um método errado reprovava.

O caso mais forte são as **leis de conservação provadas sobre $\mathbb{Q}$** . Para um campo vetorial polinomial, um candidato $I = \sum_k c_k m_k$ (monômios e, opcionalmente, logaritmos das variáveis) é conservado se e só se a derivada de Lie $\sum_i f_i \partial I / \partial x_i$ se anula identicamente; isso é um sistema linear nos c_k , resolvido exatamente em racionais. Para

a epidemia SIR, o método acha $S + I + R$ e $3S + 3I - \ln S$, ambos provados, e mostra que sobrevivem a perturbar cada coeficiente em 1 % (são estruturais). Na verificação com controle (Tabela 14), seis hamiltonianos cúbicos aleatórios têm o seu H redescoberto e provado, e seis sistemas dissipativos aleatórios não recebem lei nenhuma.

6.5 ASSAY: ESSA DIFERENÇA É REAL?

Voltando às raízes do projeto em pesquisa de IA, o Assay ataca a dor mais comum do campo, que não é treinar, mas *decidir se uma diferença de avaliação é real*: comparação pareada com *bootstrap*, permutação por troca de sinal e McNemar exato, junto do efeito mínimo detectável; *leaderboards* com empates declarados; calibração medida contra o seu próprio piso (o ECE que um modelo perfeitamente calibrado teria com aquelas confianças e aquele n); concordância de anotadores (Krippendorff, Fleiss, Cohen); vies de posição de juízes; contaminação por 13-gramas; deduplicação por MinHash verificada por Jaccard exato; e leis de escala na forma de Hoffmann, que precisam *prever as maiores corridas sem tê-las visto*.

O controle da lei de escala — as mesmas corridas com as perdas embaralhadas — achou um defeito real: sem estrutura nos dados, os parâmetros em escala logarítmica cresciam até estourar a exponencial. O ajuste agora permanece finito, e o *holdout* reporta erro de 33 % contra 0,3 % com os dados verdadeiros: a lei não prevê nada, que é a resposta certa.

6.6 INTERFACES: O TERMINAL PRIMEIRO

Toda capacidade da bancada está disponível na linha de comando (**bin/vapor**), na filosofia Unix: leitura da entrada padrão, JSON quando a saída é um *pipe*, e códigos de saída com significado (0 positivo, 1 negativo — refutado, não achado, verificação falhou, ruído —, 2 uso, 3 entrada inválida, 4 falha). O console web, a TUI e as 20 ferramentas MCP chamam as mesmas funções. O console ganhou uma identidade visual própria em que o ornamento coincide com a informação: a *fornalha* é o gráfico ao vivo da busca (faíscas por avaliação, a linha do melhor, a linha tracejada do controle, a partilha do portfólio) e a *pedra de toque* é o veredito, um risco de ouro por verificação aprovada e de chumbo pela reprovada (Figura 19). As cenas passaram a ser documentos editados por operações de texto, com expressões por quadro e direção por modelo.



Figura 19 – O espaço de trabalho: a régua de Golomb de 7 marcas achada no ótimo (25); a linha tracejada é a busca aleatória com o mesmo orçamento, que não passou de 28; abaixo, a partilha do portfólio.

6.7 O QUE ESTE CAPÍTULO NÃO AFIRMA

- Que “provado” signifique mais que enumeração completa de um espaço finito; espaços infinitos dão evidência, e o certificado diz qual.
- Que a fornalha seja competitiva com resolvidores especializados (SAT, MIP) nos seus domínios — as mesas exatas de lógica e de programação linear continuam existindo.
- Que o rascunho de um modelo esteja certo: a retrotradução existe para que a pessoa confira.
- Química além de STO-3G de camada fechada com H e He.

7 AVALIAÇÃO

Este capítulo responde a uma pergunta: *como se sabe que o que os capítulos anteriores afirmam é verdade?* A resposta tem três partes — oráculos externos, controles e propriedades bit a bit —, e termina com as ameaças à validade, isto é, com as razões pelas quais a resposta pode estar errada.

7.1 METODOLOGIA

Três tipos de evidência.

(1) **Oráculos externos**: uma implementação independente, de terceiros, amplamente usada, contra a qual o resultado do **vapor** é comparado — com igualdade bit a bit quando a semântica é a mesma, com tolerância declarada quando não é. (2) **Controles**: para cada verificação, o valor que uma implementação quebrada, ingênua ou trapaceira produziria; a verificação só é aceita se o controle cai do outro lado do limiar. (3) **Propriedades**: igualdades bit a bit entre substratos, *threads*, lotes e réplicas, que não precisam de oráculo porque comparam o sistema consigo mesmo sob transformações que não deveriam mudar nada.

Oráculos.

A Tabela 11 lista os oráculos por domínio. Todos aparecem só nos testes; o produto não depende de nenhum.

Máquina.

Os números desta monografia foram medidos num Xeon de 2,1 GHz com 2 vCPUs e AVX-512, sem GPU física e sem contadores de desempenho, com Elixir 1.14/OTP 25 e Zig 0.16. O Apêndice A lista os comandos.

7.2 TESTES

A suíte é organizada em *níveis*, cada um ligado à ferramenta externa de que depende; um nível sem a ferramenta presente é excluído e relatado como tal, nunca contado como aprovado. A Tabela 12 mostra a evolução por rodada.

Na rodada 0.13, a execução completa da suíte foi interrompida por um reinício da máquina virtual e não foi repetida; o que a tabela registra são os 108 testes dos oito

Tabela 11 – Oráculos externos usados nos testes

domínio	oráculo e critério
codificação de instruções	GNU binutils (x86 com EVEX, AArch64, rv64gcv); <code>spirv-val</code> — toda instrução emitida
execução cruzada	QEMU <i>user</i> (aarch64, riscv64, VLEN 128/256/512); Mesa lavapipe (Vulkan)
aritmética	<code>fmaf</code> , <code>+</code> , <code>*</code> do <i>hardware</i> — 800 000 operações, bit a bit; <code>mpmath</code> para funções
modelos	<code>transformers</code> , <code>diffusers</code> (PyTorch) — logits, <i>hidden states</i> , imagens
formatos	<code>gguf-py</code> , <code>llama.cpp</code> , <code>libjpeg</code> , <code>libtiff</code> , <code>jbig2dec</code> , <code>jinja2</code> , <code>cbor2</code>
engenharia	<code>ngspice</code> (circuitos, inclusive MOSFET nível 1 e Ebers–Moll); soluções analíticas
finanças	QuantLib 1.43 (calendários, bases, curvas, BSM, Leisen–Reimer, Heston); <code>simplefix</code> (FIX); SciPy; Python <code>decimal</code> ; artigos (Longstaff–Schwartz, Avellaneda–Stoikov, Gatheral–Jacquier)
protocolos	SDK oficial do MCP; cliente <code>openai</code>

Tabela 12 – Testes por rodada (execução completa de `mix test`)

rodada	executados	falhas	observação
0.5	343	0	32 excluídos por ferramenta ausente
0.6	400	2 → 0	uma violação real de arquitetura achada e corrigida
0.7	425	0	
0.8	469	1 → 0	teste antigo exigia recusar JBIG2, agora decodificado
0.9	501	0	todos os níveis presentes, salvo 18
0.11	611	0	
0.12	702	0	115 excluídos por ferramenta ausente na sessão
0.13	108	0	só os 8 arquivos tocados (ver texto)
0.14	87	0	os 8 arquivos da rodada (ver texto)

arquivos que a rodada tocou (finanças, console de mercados, rodada 13, arquivos, MCP, engenharia, bancada e lógica), todos aprovados. Os demais módulos não mudaram desde a execução completa da 0.12. Os testes novos de finanças (41) usam o QuantLib, o SciPy e o `simplefix` como oráculos; os de transistores, o `ngspice`; os do console, um navegador sem cabeça que roda *todo exemplo de todo painel* das duas mesas, em inglês e em português, e confere que nenhum rótulo ou frase em inglês sobra na versão em português.

Na rodada 0.14, os 87 testes dos oito arquivos da bancada aberta (Alembic, Athanor, Crucible, Assay, mente, operações de cena, espaço de trabalho de ponta a ponta e MCP) passaram, assim como o teste com o SDK oficial de MCP em Python, o de paridade do ruído em JavaScript e os dois testes de navegador sem cabeça das mesas antigas. Estes últimos acharam duas coisas: a ordem alfabética exigida na barra lateral, que o novo grupo “Bancada aberta” quebra de propósito (a ordem dele é a do trabalho: escrever, conferir, medir — o teste passou a respeitar grupos marcados como fixos); e um defeito real e antigo no Monte Carlo sem o processo nativo, em que o oráculo não carregava o estado do

gerador entre chamadas — invisível na máquina da rodada anterior, que tinha o processo nativo, e corrigido. Nesta máquina, sem o compilador Zig, o processo nativo não existe; o caminho pelo oráculo exato agora está correto, mas os três exemplos de Monte Carlo do console (8 192 trajetórias) passam do limite de 120 s e são interrompidos com a mensagem de tempo. É uma limitação do ambiente, registrada aqui em vez de omitida; os demais exemplos das duas mesas passaram nas duas línguas.

7.3 QUALIDADE: SINAL OU RUÍDO

A suíte de qualidade (`mix vapor.quality`) é distinta dos testes: em vez de conferir que o código faz o que o programador pretendia, confere que a *saída* é sinal e não ruído, comparando cada valor com a verdade conhecida e com um controle. A suíte cresceu de 48 verificações (0.6) para 147 (0.12), todas aprovadas; a rodada 0.13 acrescenta 23, aprovadas em cerca de 23 s (Tabela 13), e a 0.14 acrescenta 20, aprovadas em cerca de 60 s (Tabela 14).

Três verificações não têm controle numérico (“—”). Nelas, o controle é estrutural: a reconstrução pelo feed ITCH é ela mesma o verificador independente do livro; a variância da asiática é comparada com a do estimador sem controle; e o MOSFET é comparado a um cálculo à mão cujo erro esperado é exatamente a corrente de GMIN (10^{-12} S vezes a tensão). A documentação marca esses casos em vez de inventar um controle.

Na rodada 0.14 (Tabela 14), a única verificação sem controle é a dos átomos, cuja referência é o próprio número de identificadores criados (2 000): um interpretador que os convertesse em átomos cresceria a tabela exatamente nessa quantidade.

7.4 DESEMPENHO

O vapor não afirma ser o sistema mais rápido; afirma que reprodutibilidade e verificabilidade custam pouco. A Tabela 15 mostra o GEMV, a operação que domina a inferência de modelos de linguagem, nos dois *backends* x86 e com 1 e 2 *threads*, todos com os mesmos bits.

O árbitro de três tetos acerta o GEMV de 4 bits 4096² a poucos por cento (previsto 71,7 ms, observado 72–75 ms), o que um modelo de dois tetos não faz (erro de cerca de 5×). No livro de ofertas, o juiz custa da ordem do próprio motor: reexecutar uma sessão inteira é barato o bastante para ser feito em toda sessão (Figura 20).

Tabela 13 – As 23 verificações da rodada 0.13: valor, controle e limiar

verificação	valor	controle	limiar
calendário: DU ANBIMA 02/01/2025 → 02/01/2026	252	só feriados fixos $\neq$ 252	252
dinheiro: R\$ 100,00 em três partes	33,34 + 33,33 + 33,33	piso binary64: 99,99	soma exata
curva DI1+LTN+NTN-F, todo instrumento reprecificado	$4,5 \cdot 10^{-16}$	dias corridos: $> 10^{-4}$	$< 10^{-12}$
opções: paridade BSM e gregas	$7,1 \cdot 10^{-15}$	preço abaixo do intrínseco: recusado	$< 10^{-12}$
opções: <i>put</i> americana (LS 2001, tab. 1)	4,486076	européia 3,844308	$4,486 \pm 0,001$
opções: SVI calmo sem borboleta; Vogt com Monte Carlo: bits do oráculo e de 2 <i>threads</i> ; BSM no IC 99%	$g_{\min} = 0,084$ $z = 1,17$	$g_{\min} = -0,033$ Itô esquecido: $z = 8,70$	$g \geq 0 / g < 0$ $ z < 2,58$; controle > 5
Monte Carlo: asiática com variável de controle	var. $\div$ 1 307	—	> 100
VaR em t_3 : Kupiec do histórico	$p = 0,43$	normal: $p = 0,038$	$> 0,05 / < 0,05$; tamanho 1,5–10%
<i>backtest</i> : AR(1) plantado passa os 4 portões	DSR 1,0	ruído: DSR 0,10	DSR $> 0,95$, PBO $< 0,5$, RC $p < 0,05$
<i>backtest</i> : espiada no amanhã	Sharpe 20,4	pega no dia 89	antecipação apontada
LP racional: ótimo com dual; inviável com Farkas	11	proposta errada recusada	ambos os certificados
arbitragem: borboleta em <i>calls</i> ; $\psi > 0$ nas coerentes	custo $-1/4$	6 preços de estado	exata
livro: 6 000 eventos aleatórios, motor ingênuo	6 000	negócio forjado pego	relatórios iguais
livro: feed ITCH reconstrói livro e volume	10 níveis	—	iguais
pré-negociação: recusas com causa, do diário	3	causa trocada pega	todas conferidas
Hawkes: ramificação plantada 0,4	0,358	Poisson $p = 1,7 \cdot 10^{-20}$	$ n - 0,4 < 0,08$
Avellaneda–Stoikov: σ (P&L) estoque/simétrica	0,50	simétrica σ 13,26	$< 0,6$ (artigo 0,44)
Almgren–Chriss: forma fechada vs. ótimo numérico	$1,2 \cdot 10^{-16}$	—	$< 10^{-12}$
sessão de bolsa: certificados; mesma semente, mesma cabeça	0a83993d...	outra semente: cdc60e2b...	iguais / diferentes
arquivos assinados	válido	manifesto reescrito recusado	assinatura confiável
unidades afins: 98,6 °F em °C	37,0	°C num composto recusado	37 ± 10^{-9}
circuito: MOSFET fonte comum vs. lei quadrática à mão	$3,3 \cdot 10^{-12}$	—	$< 10^{-11}$ (GMIN)

7.5 REVISÕES INDEPENDENTES

Desde a rodada 0.11, cada rodada termina com uma revisão feita por um agente que não viu o trabalho sendo produzido, com a instrução de procurar defeitos, não de confirmar. Na 0.11 a revisão achou nove problemas — dois de segurança nos arquivos (receitas sem limite, bomba de zip), um “provado” por 0/0, um “nunca perde” que vinha de amostra —, todos corrigidos e testados. Na rodada 0.13 a revisão independente foi interrompida antes de concluir, por limite de uso da sessão, e não produziu achados; a conferência desta rodada ficou com os testes, as 23 verificações com controle e uma inspeção visual de cada figura desta monografia e de cada slide — que achou e corrigiu sobreposições de rótulos, um gráfico com valores em escala logarítmica rotulados como

Tabela 14 – As 20 verificações da rodada 0.14: a bancada aberta

verificação	valor	controle	limiar
fornalha vs. acaso: régua de Golomb, 7 marcas	25 (ótimo)	aleatório: 0 acertos em 3 923	25; controle 0
prova exaustiva: max-cut	13, igual à força bruta	valor forjado 14 recusado	iguais; forjado recusado
prova vs. refutação: $R(3, 3)$	independente	K_5 : pentágono (5 arestas)	ambos
holdout de regras de médias móveis	provado em 32 768 grafos	passeio aleatório: $\rho = -0,33$	$\geq 0,5$ / $< 0,5$
replay: raiz do diário	AR(1) $\varphi = 0,5$: $\rho = 0,73$	semente 2: outra raiz	iguais / diferentes
sandbox: bomba de memória, 16 MB	mesma semente, mesma raiz	morta (:memory)	programa pequeno: 385
átomos: 2000 identificadores novos	0	—	< 50
leis: hamiltonianos cúbicos aleatórios	6/6 provados sobre $\mathbb{Q}$	dissipativos: 0 leis	6/6; 0
quântica: oscilador harmônico	$ E - (n + \frac{1}{2}) = 3 \cdot 10^{-6}$; ordem 1,99	caixa $\pm 1,5$: evidência falha	$< 2 \cdot 10^{-4}$; ordem $2 \pm 0,2$
Kepler, $e = 0,36$, ≈ 250 órbitas, $\Delta t = 0,1$	Yoshida: $\leq 1,4 \cdot 10^{-3}$	RK4: deriva 0,062	controle $\geq 10 \times$
H ₂ RHF/STO-3G a 1,4 bohr	-1,116714	8 bohr: $-0,61 > 2E_H$	$\pm 10^{-4}$
filogenia NJ + bootstrap	3 clados, todos verdadeiros	colunas embaralhadas: 0,43	controle $< 0,7$
regressão simbólica $3x^2 - 2x + 1$	$R^2 = 1,0$	alvo embaralhado: -0,32	$> 0,9999$ / $< 0,5$
comparação pareada, 60 nulos	erro tipo I 0,00	efeito plantado: $p = 2,5 \cdot 10^{-4}$	$\leq 0,15$; $< 0,001$
calibração	calibrado: $p = 0,45$	confiante: $p = 0,002$; Platt 0,23 $\rightarrow$ 0,08	$> 0,05$ / $< 0,01$
Krippendorff (exemplo publicado)	0,743	rótulos ao acaso: 0,007	$\pm 0,001$; $ \alpha < 0,1$
lei de escala, $\alpha = 0,34$ plantado	IC [0,334; 0,351], erro 0,3 %	perdas embaralhadas: 33 %	contém; $< 1\%$ / $> 1\%$
deduplicação MinHash	mantém {0, 2}	erro da estimativa 0,0	exato; $< 0,1$
jogo da velha resolvido	empate em 5 478; MCTS 0,95	aleatório vs. aleatório 0,60	ambos
árvore portátil: noise	3 valores de ouro	len numa cena recusado	bit a bit

Tabela 15 – GEMV nesta máquina (ms por chamada; todos com os mesmos bits)

operação	ISA	1 thread	2 threads	GB/s (2 thr.)
GEMV f32 2048 ²	AVX2	0,661	0,296	56,8
GEMV f32 2048 ²	AVX-512	0,582	0,282	59,7
GEMV bf16 2048 ²	AVX2	0,428	0,201	41,9
GEMV bf16 2048 ²	AVX-512	0,300	0,174	48,4
GEMV 4 bits 1024 $\times$ 4096	AVX2	0,567	0,287	8,7
GEMV 4 bits 1024 $\times$ 4096	AVX-512	0,537	0,272	9,2

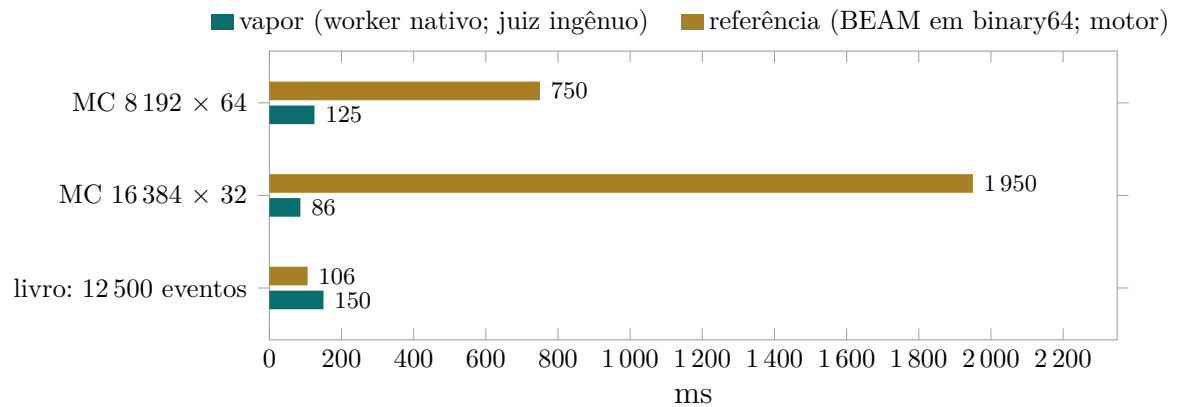


Figura 20 – Rodada 0.13: Monte Carlo no worker contra a BEAM em binary64 ($6\text{--}23\times$), e o custo do juiz ingênuo contra o do motor ($\approx 8,5 \mu\text{s}$ por evento) para 12 500 eventos.

lineares, números de tamanho e de linhas desatualizados nos slides e, no console, rótulos em inglês na versão em português.

7.6 AMEAÇAS À VALIDADE

Validade interna.

O mesmo projeto escreve o sistema, os testes e os controles. A mitigação principal são os oráculos externos; mas o episódio do FOK a mercado (seção 5.9) mostra o risco residual: dois motores escritos pelo mesmo autor podem compartilhar a mesma leitura errada da especificação. Por isso o juiz do livro tem invariantes que não executam nada, e por isso a revisão independente existe.

Validade externa.

Uma única máquina modesta, sem GPU física (Vulkan sob lavapipe), sem *hardware* ARM ou RISC-V (sob QEMU) e sem contadores de desempenho. A escala a muitos núcleos não foi medida. O caminho de *cooperative matrix* do SPIR-V foi emitido e validado mas não executado. Em finanças, nenhum dado de mercado real foi usado.

Validade de construto.

Os controles são escolhidos pelo autor como “o erro mais plausível”; um controle fraco faria uma verificação parecer mais forte do que é. A documentação publica cada controle, para que possa ser contestado.

Validade de conclusão.

Vários resultados estatísticos (Hawkes, Avellaneda–Stoikov, portões de *backtest*) vêm de uma semente; os limiares têm folga, e o tamanho do teste de Kupiec é medido sobre 400 séries, mas não há, para todos, uma distribuição sobre sementes. Os níveis de teste sem ferramenta na sessão foram excluídos e não reexecutados nesta rodada; os módulos desses níveis não mudaram.

8 TRABALHOS RELACIONADOS

O **vapor** atravessa várias áreas; este capítulo o situa em cada uma, dizendo o que toma emprestado, o que faz diferente e onde fica atrás.

8.1 COMPILADORES DE TENSORES

XLA (OpenXLA Project, 2024), TVM (CHEN et al., 2018), Halide (RAGAN-KELLEY et al., 2013), Triton (TILLET; KUNG; COX, 2019) e o ecossistema MLIR (LATTNER et al., 2021) compilam programas de tensores para CPUs, GPUs e aceleradores, e são muito mais rápidos e completos que o **vapor**. Halide introduziu a separação entre algoritmo e *schedule* que o **vapor** também usa (a álgebra é o algoritmo; o fator de grupo, as linhas por iteração e o corte de regiões são o *schedule*). A diferença está nos objetivos: esses compiladores otimizam desempenho e tratam a ordem das reduções como um grau de liberdade do *schedule*; o **vapor** fixa a ordem e trata a igualdade de bits como restrição. Nenhum deles emite um certificado da compilação nem isola o código gerado do processo do *runtime*.

8.2 REPRODUTIBILIDADE NUMÉRICA

Demmel e Nguyen (2015) propõem somas reprodutíveis independentes da ordem por pré-arredondamento a uma grade comum (ReproBLAS); o custo é um fator constante por redução, e a técnica vale para qualquer ordem — o que é mais forte do que o **vapor** precisa. O **vapor** fixa a ordem e paga zero por redução, mas só porque controla o código inteiro. Os modos determinísticos de *frameworks* (PyTorch Contributors, 2024) garantem repetição na mesma máquina e versão, não entre arquiteturas. Goldberg (1991) e Higham (2002) são a base da análise de erro que o envelope implementa em racionais exatos; Monniaux (2008) cataloga as armadilhas de verificar programas de ponto flutuante, várias das quais (FMA implícita, precisão estendida) o **vapor** elimina por construção.

8.3 COMPILADORES VERIFICADOS E VALIDAÇÃO DE TRADUÇÃO

O CompCert (LEROY, 2009) prova o compilador inteiro em Coq, inclusive a alocação de registradores, que é validada *a posteriori* por um verificador provado (RIDEAU; LEROY, 2010) — o mesmo desenho que o **vapor** usa, em escala muito maior e com garantias

muito mais completas. O CakeML (KUMAR et al., 2014) faz o mesmo para uma linguagem funcional, até o código de máquina. Alive2 (LOPES et al., 2021) valida otimizações do LLVM por tradução. A validação de tradução (PNUELI; SIEGEL; SINGERMAN, 1998) e o *proof-carrying code* (NECULA, 1997) são as ideias de que o **vapor** deriva a escada de verificação e o certificado. A contribuição do **vapor** aqui não é teórica: é mostrar que essas ideias cabem num compilador de tensores pequeno, sem dependências, que emite cinco ISAs, e combiná-las com execução isolada e co-assinatura.

8.4 ALGORITMOS CERTIFICADORES E VERIFICAÇÃO DE SAÍDAS

McConnell et al. (2011) sistematizam os *algoritmos certificadores*: algoritmos que produzem, com a saída, uma testemunha que um verificador simples confere. O padrão proponente/verificador do **vapor** é uma aplicação direta, estendida em duas direções: o proponente pode ser qualquer um (inclusive um modelo de linguagem pelo MCP (Anthropic, 2024)), e cada verificador vem acompanhado de um controle. Em SAT, os formatos DRUP/DRAT (GOLDBERG; NOVIKOV, 2003; WETZLER; HEULE; JR., 2014) são o exemplo canônico, usado pelo **vapor** na mesa de lógica. Em programação linear, os certificados de dualidade e de Farkas (SCHRIJVER, 1986) são o que a mesa de finanças devolve para arbitragem. Em transparência, o *Certificate Transparency* (LAURIE; LANGLEY; KASPER, 2013; LAURIE; MESSERI; STRADLING, 2021) fornece a estrutura dos logs e diários.

8.5 INFERÊNCIA DE MODELOS DE LINGUAGEM

O llama.cpp e o vLLM (KWON et al., 2023) são as referências de inferência em CPU e GPU; o vLLM introduziu o cache KV paginado que o **vapor** também implementa. A decodificação especulativa (LEVIATHAN; KALMAN; MATIAS, 2023) e o MLA (DeepSeek-AI, 2024) são técnicas que o **vapor** reformula para não mudar bits. Esses sistemas são várias vezes mais rápidos que o **vapor**; nenhum oferece igualdade de bits entre substratos ou certificados.

8.6 FINANÇAS QUANTITATIVAS

O QuantLib (The QuantLib Contributors, 2026) é a biblioteca aberta de referência e o oráculo dos testes de calendários, curvas e opções; o **vapor** cobre uma fração pequena dele. A diferença é o certificado: o QuantLib devolve números; o **vapor** devolve números com o objeto que os julga (reprecificação, paridade, limites, $g(k)$, KKT). Em *backtesting*,

frameworks abertos como o zipline e o backtrader oferecem simulação de carteiras, mas não portões de seleção nem certificados de causalidade; a literatura de Bailey e López de Prado (2014), Bailey et al. (2017), Harvey, Liu e Zhu (2016) e White (2000) fornece os testes que o **vapor** implementa. Até onde este trabalho pôde verificar, a *invariância de prefixo bit a bit como certificado de ausência de antecipação* não aparece nessas ferramentas; a ideia é próxima da validação *walk-forward*, mas com outro objetivo (detectar vazamento, não estimar desempenho fora da amostra) e com igualdade exata em vez de comparação estatística.

Em microestrutura, os simuladores baseados em agentes como o ABIDES (BYRD; HYBINETTE; BALCH, 2020) modelam bolsas para pesquisa; o LOBSTER fornece dados reconstruídos de livros da Nasdaq. A sessão de bolsa do **vapor** é menor que esses simuladores, mas usa o próprio motor de produção, verificado por um juiz independente, com reprodução por *hash*. A arbitragem como problema de LP é clássica (HARRISON; KREPS, 1979; JOUINI; KALLAL, 1995); a contribuição é a decisão em aritmética racional com o certificado entregue para conferência por terceiros.

8.7 SÍNTESE

A Tabela 16 resume as propriedades que distinguem o **vapor**. Ela não mede desempenho nem completude, em que os sistemas comparados são superiores.

Tabela 16 – Propriedades de verificabilidade em sistemas representativos

sistema	bits iguais entre ISAs	fase crítica verificada	código isolado	certificado co-assinável	verificador por resposta
XLA, TVM, Triton	não	não	não	não	não
PyTorch (determinístico)	não	não	não	não	não
ReproBLAS	sim (somas)	não	não	não	não
CompCert	n/a	sim (tudo)	não	não	não
llama.cpp, vLLM	não	não	não	não	não
QuantLib	n/a	não	não	não	não
vapor	sim	sim (alocação)	sim	sim	sim

9 CONCLUSÃO

9.1 SÍNTESE

Esta monografia defendeu que **verificabilidade pode ser uma propriedade de arquitetura**, obtida por construção e a custo baixo, se três decisões forem tomadas desde o início: semântica simbólica e fixa, compilador não confiado com código isolado, e toda resposta acompanhada do seu juiz. O **vapor** é a evidência: um compilador de tensores sem dependências que emite cinco ISAs com os mesmos bits, confere a alocação com um verificador provado, nunca executa código gerado dentro da máquina virtual e sai de cada compilação com um certificado que nós independentes reproduzem e co-assinam — e, sobre ele, catorze rodadas de aplicações que herdaram essas garantias.

A rodada 0.13 testou a tese num domínio em que ela tem consequências econômicas diretas. O resultado foi o previsto no Capítulo 1: um sistema construído para a verificabilidade produziu, sem esforço extraordinário, coisas que a prática usual não produz — calendários e curvas iguais à referência dia a dia por 89 anos; Monte Carlo com os mesmos bits do oráculo e de outra contagem de *threads*; *backtests* com um certificado de ausência de antecipação; arbitragem decidida exatamente, com a prova entregue a quem quiser conferir; e um livro de ofertas cujo juiz independente achou um defeito real que dois motores compartilhavam.

A rodada 0.14 testou a tese no sentido oposto: em vez de um domínio novo, *qualquer* domínio. Ao trocar as mesas por categoria por uma linguagem pura, uma fornalha de estratégias e uma pedra de toque, o sistema passou de vitrine a ferramenta — o usuário escreve o problema, a busca o enfrenta contra o acaso com o mesmo orçamento, e o certificado é reconferido por quem não confia nela. Os controles da rodada acharam, mais uma vez, defeitos que os testes não tinham visto: uma lei de escala que estourava em dados sem estrutura, um vazamento de átomos na compilação de expressões e um estado de gerador perdido no caminho sem processo nativo.

9.2 RESPOSTAS ÀS CINCO PERGUNTAS

O quê. Um compilador e *runtime* de tensores certificados e um ecossistema de mesas — de modelos de linguagem a finanças — em que cada número sai com o que permite julgá-lo.

Para quem. Para quem precisa *demonstrar* um resultado: equipes reguladas, auditores

de IA, ciência reprodutível, pesquisadores de compiladores e verificação, e estudantes que queiram ver um sistema inteiro, do termo ao registrador.

Para que. Para que “rodou de novo e deu diferente”, “o modelo disse” e “o *backtest* mostrou” deixem de ser respostas aceitáveis.

Por que. Porque as falhas que mais custam são de prova, não de desempenho, e porque a prova, feita por construção, custou aqui um fator pequeno — quando custou.

Como. Pelas inversões da Tabela 7: ordem de soma fixa e paralelismo entre saídas; verificar em vez de confiar; isolar em vez de embutir; contar o trabalho em vez de prescrever o despacho; e acompanhar cada juiz de um controle.

9.3 LIMITAÇÕES

As limitações estão espalhadas pelos capítulos, cada uma ao lado da afirmação que limita; aqui estão reunidas as mais importantes.

- **Desempenho absoluto.** O vapor é mais lento que os compiladores e servidores de inferência de produção. A afirmação é que reprodutibilidade custa pouco *relativamente*, não que o sistema seja o mais rápido.
- **Hardware.** Nenhum silício RVV, ARM ou GPU discreta foi usado; RVV e NEON foram validados sob QEMU, Vulkan sob lavapipe. O caminho de *cooperative matrix* não foi executado.
- **Transcendentais.** Iguais em todo substrato, mas não corretamente arredondadas (1–2 ulps).
- **Base confiável.** O modelo padrão da IEEE-754 no *hardware*, o *printer* do extrator, a BEAM, o kernel e o *driver* Vulkan (contido em processo).
- **Finanças.** Latência de microssegundos, não de nanossegundos; nenhum dado real de mercado; sem XVA, modelos de taxa de vários fatores ou volatilidade calibrada a uma superfície; o gerador do Monte Carlo no worker é antigo.
- **Bancada aberta.** “Provado” é enumeração de um espaço finito; a fornalha não substitui resolvidores especializados; a Alembic é interpretada (de 20 a 50 vezes mais lenta que código nativo da BEAM); a busca roda num nó só.
- **Avaliação.** Uma máquina; resultados estatísticos de uma semente em vários casos; controles escolhidos pelo autor.

9.4 TRABALHOS FUTUROS

A lista de pendências do repositório (`docs/TODO.md`) é mantida item a item, cada um com o motivo e o que o fecha. Os mais importantes para a tese são:

1. **Motor de ofertas de baixa latência** como programa do worker (sem alocação, livro em arranjos por nível), com o mesmo diário e o mesmo juiz fora do caminho crítico, e latência medida com contadores de *hardware*.
2. **Multiplicação inteira de 32/64 bits na álgebra**, nos cinco codificadores: abriria geradores modernos (Philox, Threefry) para o Monte Carlo e o kernel NTT para criptografia homomórfica e provas de conhecimento zero.
3. **Dados de mercado reais**: um dia de ITCH da Nasdaq reconstruído e conferido contra instantâneos publicados; curvas e preços indicativos da ANBIMA.
4. **Juiz em outra linguagem**: o motor ingênuo também em Python, para que a independência seja de linguagem e não só de algoritmo — a lição do defeito do FOK a mercado.
5. **Arbitragem na superfície inteira**: calendário $\times$ *strike* num só programa linear racional.
6. **Athanor distribuído e Alembic compilada**: repartir o orçamento entre nós e juntar os diários por raiz de Merkle; descer os objetivos numéricos para a álgebra de tensores com o combustível contado por bloco; exportar afirmações para SAT/SMT quando o espaço não cabe na enumeração.
7. **Silício**: RVV 1.0 em placas reais, *cooperative matrix* numa GPU, contadores de desempenho e energia (J/token) em *bare metal*.
8. **Codificadores ligados a semânticas formais de ISA**, reduzindo a base confiável (seção 3.8).

9.5 CONSIDERAÇÕES FINAIS

O vapor começou com uma pergunta estreita — é possível obter os mesmos bits em todo substrato sem perder desempenho? — e a resposta afirmativa se mostrou mais útil do que a pergunta sugeria. Bits canônicos tornam toda execução repetível; execuções repetíveis tornam diários verificáveis; diários verificáveis tornam agentes, sessões de bolsa e *backtests* auditáveis. Em cada rodada, a mesma separação — *uma busca propõe, um verificador decide* — resolveu um problema novo sem mudar o núcleo. A lição de método que fica é modesta e, ao mesmo tempo, exigente: *todo número deve sair com o que permite julgá-lo, e todo juiz deve mostrar que poderia ter recusado*.

REFERÊNCIAS

- ALBRECHER, H. et al. The little Heston trap. **Wilmott Magazine**, p. 83–92, jan 2007.
- ALMGREN, R.; CHRISS, N. Optimal execution of portfolio transactions. **Journal of Risk**, v. 3, n. 2, p. 5–39, 2000.
- Anthropic. **Model Context Protocol: specification**. [S.l.], 2024.
- ARMSTRONG, J. **Making reliable distributed systems in the presence of software errors**. Tese (Doutorado) — Royal Institute of Technology (KTH), Estocolmo, 2003.
- AVELLANEDA, M.; STOIKOV, S. High-frequency trading in a limit order book. **Quantitative Finance**, v. 8, n. 3, p. 217–224, 2008.
- BACRY, E.; MASTROMATTEO, I.; MUZY, J.-F. Hawkes processes in finance. **Market Microstructure and Liquidity**, v. 1, n. 1, p. 1550005, 2015.
- BAILEY, D. H. et al. The probability of backtest overfitting. **Journal of Computational Finance**, v. 20, n. 4, p. 39–69, 2017.
- BAILEY, D. H.; López de Prado, M. The deflated Sharpe ratio: correcting for selection bias, backtest overfitting and non-normality. **The Journal of Portfolio Management**, v. 40, n. 5, p. 94–107, 2014.
- Basel Committee on Banking Supervision. **Supervisory framework for the use of “backtesting” in conjunction with the internal models approach to market risk capital requirements**. Basileia, 1996.
- BERNSTEIN, D. J. et al. High-speed high-security signatures. **Journal of Cryptographic Engineering**, v. 2, n. 2, p. 77–89, 2012.
- BLACK, F.; SCHOLES, M. The pricing of options and corporate liabilities. **Journal of Political Economy**, v. 81, n. 3, p. 637–654, 1973.
- BYRD, D.; HYBINETTE, M.; BALCH, T. H. ABIDES: towards high-fidelity multi-agent market simulation. In: **Proceedings of the 2020 ACM SIGSIM Conference on Principles of Advanced Discrete Simulation**. [S.l.: s.n.], 2020. p. 11–22.
- CHEN, T. et al. TVM: an automated end-to-end optimizing compiler for deep learning. In: **13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)**. [S.l.: s.n.], 2018. p. 578–594.
- CHRISTOFFERSEN, P. F. Evaluating interval forecasts. **International Economic Review**, v. 39, n. 4, p. 841–862, 1998.
- Comissão Europeia. **Regulamento Delegado (UE) 2017/589 (RTS 6): requisitos organizacionais das empresas de investimento que operam negociação algorítmica**. [S.l.], 2017.

- DeepSeek-AI. **DeepSeek-V2: a strong, economical, and efficient mixture-of-experts language model**. 2024. ArXiv:2405.04434.
- DEMME, J.; NGUYEN, H. D. Parallel reproducible summation. **IEEE Transactions on Computers**, v. 64, n. 7, p. 2060–2070, 2015.
- FARKAS, J. Theorie der einfachen Ungleichungen. **Journal für die reine und angewandte Mathematik**, v. 124, p. 1–27, 1902.
- FIX Trading Community. **Financial Information eXchange Protocol, version 4.4**. [S.l.], 2003.
- GATHERAL, J.; JACQUIER, A. Arbitrage-free SVI volatility surfaces. **Quantitative Finance**, v. 14, n. 1, p. 59–71, 2014.
- GLASSERMAN, P. **Monte Carlo methods in financial engineering**. Nova York: Springer, 2003.
- GOLDBERG, D. What every computer scientist should know about floating-point arithmetic. **ACM Computing Surveys**, v. 23, n. 1, p. 5–48, 1991.
- GOLDBERG, E.; NOVIKOV, Y. Verification of proofs of unsatisfiability for CNF formulas. In: **Design, Automation and Test in Europe (DATE)**. [S.l.: s.n.], 2003. p. 886–891.
- HARRISON, J. M.; KREPS, D. M. Martingales and arbitrage in multiperiod securities markets. **Journal of Economic Theory**, v. 20, n. 3, p. 381–408, 1979.
- HARRISON, J. M.; PLISKA, S. R. Martingales and stochastic integrals in the theory of continuous trading. **Stochastic Processes and their Applications**, v. 11, n. 3, p. 215–260, 1981.
- HARVEY, C. R.; LIU, Y.; ZHU, H. ... and the cross-section of expected returns. **The Review of Financial Studies**, v. 29, n. 1, p. 5–68, 2016.
- HAWKES, A. G. Spectra of some self-exciting and mutually exciting point processes. **Biometrika**, v. 58, n. 1, p. 83–90, 1971.
- HESTON, S. L. A closed-form solution for options with stochastic volatility with applications to bond and currency options. **The Review of Financial Studies**, v. 6, n. 2, p. 327–343, 1993.
- HIGHAM, N. J. **Accuracy and stability of numerical algorithms**. 2. ed. Filadélfia: SIAM, 2002.
- IEEE. **IEEE Standard for Floating-Point Arithmetic (IEEE Std 754-2019)**. Nova York, 2019.
- Intel Corporation. **Intel 64 and IA-32 Architectures Software Developer’s Manual**. [S.l.], 2024.
- JOUINI, E.; KALLAL, H. Martingales and arbitrage in securities markets with transaction costs. **Journal of Economic Theory**, v. 66, n. 1, p. 178–197, 1995.

- KEMNA, A. G. Z.; VORST, A. C. F. A pricing method for options based on average asset values. **Journal of Banking & Finance**, v. 14, n. 1, p. 113–129, 1990.
- KUMAR, R. et al. CakeML: a verified implementation of ML. In: **Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)**. [S.l.: s.n.], 2014. p. 179–191.
- KUPIEC, P. H. Techniques for verifying the accuracy of risk measurement models. **The Journal of Derivatives**, v. 3, n. 2, p. 73–84, 1995.
- KWON, W. et al. Efficient memory management for large language model serving with PagedAttention. In: **Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)**. [S.l.: s.n.], 2023. p. 611–626.
- LATTNER, C. et al. MLIR: scaling compiler infrastructure for domain specific computation. In: **IEEE/ACM International Symposium on Code Generation and Optimization (CGO)**. [S.l.: s.n.], 2021. p. 2–14.
- LAURIE, B.; LANGLEY, A.; KASPER, E. **Certificate Transparency (RFC 6962)**. [S.l.], 2013.
- LAURIE, B.; MESSERI, E.; STRADLING, R. **Certificate Transparency Version 2.0 (RFC 9162)**. [S.l.], 2021.
- L'ECUYER, P.; SIMARD, R. TestU01: a C library for empirical testing of random number generators. **ACM Transactions on Mathematical Software**, v. 33, n. 4, p. 22:1–22:40, 2007.
- LEDOIT, O.; WOLF, M. A well-conditioned estimator for large-dimensional covariance matrices. **Journal of Multivariate Analysis**, v. 88, n. 2, p. 365–411, 2004.
- LEISEN, D. P. J.; REIMER, M. Binomial models for option valuation – examining and improving convergence. **Applied Mathematical Finance**, v. 3, n. 4, p. 319–346, 1996.
- LEROY, X. Formal verification of a realistic compiler. **Communications of the ACM**, v. 52, n. 7, p. 107–115, 2009.
- LEVIATHAN, Y.; KALMAN, M.; MATIAS, Y. Fast inference from transformers via speculative decoding. In: **Proceedings of the 40th International Conference on Machine Learning (ICML)**. [S.l.: s.n.], 2023. p. 19274–19286.
- LONGSTAFF, F. A.; SCHWARTZ, E. S. Valuing American options by simulation: a simple least-squares approach. **The Review of Financial Studies**, v. 14, n. 1, p. 113–147, 2001.
- LOPES, N. P. et al. Alive2: bounded translation validation for LLVM. In: **Proceedings of the 42nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)**. [S.l.: s.n.], 2021. p. 65–79.
- López de Prado, M. Building diversified portfolios that outperform out of sample. **The Journal of Portfolio Management**, v. 42, n. 4, p. 59–69, 2016.
- MCCONNELL, R. M. et al. Certifying algorithms. **Computer Science Review**, v. 5, n. 2, p. 119–161, 2011.

- MERKLE, R. C. A digital signature based on a conventional encryption function. In: **Advances in Cryptology – CRYPTO ’87**. [S.l.]: Springer, 1987. (Lecture Notes in Computer Science, v. 293), p. 369–378.
- MERTON, R. C. Theory of rational option pricing. **The Bell Journal of Economics and Management Science**, v. 4, n. 1, p. 141–183, 1973.
- MONNIAUX, D. The pitfalls of verifying floating-point computations. **ACM Transactions on Programming Languages and Systems**, v. 30, n. 3, p. 12:1–12:41, 2008.
- MOURA, L. de; ULLRICH, S. The Lean 4 theorem prover and programming language. In: **Automated Deduction – CADE 28**. [S.l.]: Springer, 2021. (Lecture Notes in Computer Science, v. 12699), p. 625–635.
- Nasdaq. **Nasdaq TotalView-ITCH 5.0**. [S.l.], 2020.
- NECULA, G. C. Proof-carrying code. In: **Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)**. [S.l.: s.n.], 1997. p. 106–119.
- OGATA, Y. On Lewis’ simulation method for point processes. **IEEE Transactions on Information Theory**, v. 27, n. 1, p. 23–31, 1981.
- OpenXLA Project. **XLA: optimizing compiler for machine learning**. [S.l.], 2024.
- PNUELI, A.; SIEGEL, M.; SINGERMAN, E. Translation validation. In: **Tools and Algorithms for the Construction and Analysis of Systems (TACAS)**. [S.l.]: Springer, 1998. (Lecture Notes in Computer Science, v. 1384), p. 151–166.
- POLETTTO, M.; SARKAR, V. Linear scan register allocation. **ACM Transactions on Programming Languages and Systems**, v. 21, n. 5, p. 895–913, 1999.
- POLITIS, D. N.; ROMANO, J. P. The stationary bootstrap. **Journal of the American Statistical Association**, v. 89, n. 428, p. 1303–1313, 1994.
- PyTorch Contributors. **Reproducibility**. [S.l.], 2024.
- RAGAN-KELLEY, J. et al. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In: **Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)**. [S.l.: s.n.], 2013. p. 519–530.
- RIDEAU, S.; LEROY, X. Validating register allocation and spilling. In: **Compiler Construction (CC)**. [S.l.]: Springer, 2010. (Lecture Notes in Computer Science, v. 6011), p. 224–243.
- RISC-V International. **RISC-V “V” Vector Extension, version 1.0**. [S.l.], 2021.
- SALMON, J. K. et al. Parallel random numbers: as easy as 1, 2, 3. In: **Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC11)**. [S.l.: s.n.], 2011.
- SCHRIJVER, A. **Theory of linear and integer programming**. Chichester: Wiley, 1986.

- SPINU, F. **An algorithm for computing risk parity weights**. 2013. SSRN 2297383.
- The Khronos Group. **SPIR-V Specification**. [S.l.], 2024.
- The QuantLib Contributors. **QuantLib: a free/open-source library for quantitative finance**. 1.43. ed. [S.l.], 2026.
- TILLET, P.; KUNG, H. T.; COX, D. Triton: an intermediate language and compiler for tiled neural network computations. In: **Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)**. [S.l.: s.n.], 2019. p. 10–19.
- U.S. Securities and Exchange Commission. **Risk management controls for brokers or dealers with market access (Rule 15c3-5, 17 CFR 240.15c3-5)**. [S.l.], 2010.
- WETZLER, N.; HEULE, M. J. H.; JR., W. A. H. DRAT-trim: efficient checking and trimming using expressive clausal proofs. In: **Theory and Applications of Satisfiability Testing – SAT 2014**. [S.l.]: Springer, 2014. (Lecture Notes in Computer Science, v. 8561), p. 422–429.
- WHITE, H. A reality check for data snooping. **Econometrica**, v. 68, n. 5, p. 1097–1126, 2000.
- WICHMANN, B. A.; HILL, I. D. Algorithm AS 183: an efficient and portable pseudo-random number generator. **Applied Statistics**, v. 31, n. 2, p. 188–190, 1982.
- WICHURA, M. J. Algorithm AS 241: the percentage points of the normal distribution. **Applied Statistics**, v. 37, n. 3, p. 477–484, 1988.
- WILKINSON, J. H. **Rounding errors in algebraic processes**. Englewood Cliffs: Prentice-Hall, 1963.
- WILLIAMS, S.; WATERMAN, A.; PATTERSON, D. Roofline: an insightful visual performance model for multicore architectures. **Communications of the ACM**, v. 52, n. 4, p. 65–76, 2009.

Apêndices

APÊNDICE A – REPRODUÇÃO

Este apêndice diz como refazer cada número da monografia. Todos os comandos rodam a partir da raiz do repositório.

A.1 AMBIENTE

<code>nix develop</code>	# Elixir, Zig 0.16, QEMU, Vulkan+lavapipeline, spirv-tools, # binutils cruzado, elan (Lean)
<code>make native cross</code>	# vapor-worker + vapor-fabric e workers aarch64/riscv64
<code>make fixtures</code>	# vocabulários reais (SHA-256 fixado)

Sem Nix, o mínimo é Elixir ≥ 1.14 e Zig 0.16 (os testes de finanças pedem também `pip install QuantLib simplefix scipy`; os de transistores, o `ngspice`). Os níveis cuja ferramenta não está presente são excluídos e relatados no início da execução.

A.2 COMANDOS POR AFIRMAÇÃO

Tabela 17 – Onde cada número é produzido

afirmação	comando
mesmos bits em todos os substratos	<code>mix test test/vapor/canon_test.exs</code> <code>test/vapor/native_test.exs</code> <code>test/vapor/cross_test.exs</code> <code>test/vapor/fabric_test.exs</code>
invariância a <i>threads</i> , lote, réplica	<code>mix test test/vapor/threads_test.exs</code> <code>test/vapor/engine_test.exs</code>
codificação contra binutils e <code>spirv-val</code>	<code>mix test test/vapor/encoding_test.exs</code>
verificador de alocação extraído	<code>mix test test/vapor/regalloc_test.exs</code> <code>test/vapor/extracted_conformance_test.exs</code> ; provas: <code>cd proofs && lake build</code>
regras de reescrita provadas	<code>mix test test/vapor/rewrite_soundness_test.exs</code>
oráculo <code>binary32</code> contra o <i>hardware</i>	<code>mix test test/vapor/f32_test.exs</code>
contenção das quatro classes de falha	<code>mix test test/vapor/native_test.exs (tag :native)</code>
escada e certificado	<code>mix test test/vapor/ladder_test.exs</code>

APÊNDICE B – EXERCÍCIOS

Os exercícios estão organizados por capítulo e marcados por nível: [G] graduação, [M] mestrado, [D] doutorado (questões abertas, sem resposta conhecida pelo autor). Os de nível G e M podem ser resolvidos com o repositório; vários pedem um experimento.

FUNDAMENTOS

Referentes ao Capítulo 2.

1. [G] Mostre um exemplo de três números binary32 a, b, c tais que $(a + b) + c \neq a + (b + c)$. Confira com `Vapor.F32`.
2. [G] Por que $x + 0 \rightarrow x$ não é uma reescrita válida em IEEE-754, mas $x + (-0) \rightarrow x$ é? E $x \cdot 0 \rightarrow 0$?
3. [M] Prove que, para binary32, calcular $\text{fl}_{32}(\text{fl}_{64}(a + b))$ dá o mesmo que $\text{fl}_{32}(a + b)$ quando a, b são binary32. Onde a hipótese $53 \geq 2 \cdot 24 + 2$ entra?
4. [M] A cota (2.2) vale para qualquer ordem de redução. Derive a cota mais apertada para a árvore de 16 acumuladores da Figura 1 aplicada a $n = 16k$ termos.
5. [G] Construa uma árvore de Merkle RFC 6962 com 5 folhas e escreva a prova de inclusão da terceira. Por que folhas e nós internos usam prefixos diferentes?

ARQUITETURA

Referentes ao Capítulo 3.

6. [G] Explique por que um GEMV com 16 acumuladores por linha e R linhas por iteração tem os mesmos bits para qualquer R e qualquer número de *threads*, mas um GEMV que divide cada linha entre duas *threads* não tem.
7. [G] Liste as quatro condições que `check_alloc` confere. Dê um exemplo de alocação que viola cada uma.
8. [M] O `vapor` não tem *spill*. Descreva um kernel para o qual nenhuma das três estratégias (menor g , menos linhas, corte de região) basta. O que o compilador deveria fazer?
9. [M] O certificado não inclui tempos nem a decisão de despacho. Que ataque ou falha seria possível se incluísse? E se não incluísse o resumo das fontes Lean?

10. [M] Usando a (3.1), explique por que o GEMV de 4 bits quase não acelera com AVX-512 (Tabela 15). Que mudança no kernel atacaria o teto de instruções?
11. [D] Proponha um método para ligar os codificadores de instruções do `vapor` a uma semântica formal de ISA sem introduzir dependências no produto. O que seria provado, e o que continuaria testado?

INOVAÇÕES

Referentes ao Capítulo 4.

12. [G] Para cada linha da Tabela 7, dê um exemplo de sistema que segue a suposição usual.
13. [M] Paralelismo de tensor por linha com *all-reduce* reassocia somas; por coluna com *all-gather*, não. Mostre por quê e calcule o tráfego de rede de cada um para uma camada $d \times d$ em p nós.
14. [M] Escolha uma verificação da Tabela 13 e proponha um controle mais forte que o atual. Implemente-o e meça.
15. [D] Formalize a noção de “controle adversarial mínimo” esboçada na seção 4.9. Existe uma cota útil sobre a probabilidade de um defeito passar por um conjunto de verificações com controles?

FINANÇAS

Referentes ao Capítulo 5.

16. [G] Calcule à mão o rateio de R\$ 10,00 em sete partes iguais pelo maior resto. Mostre que o piso em binary64 de cada parte não soma o total.
17. [G] Calcule a data da Páscoa de 2027 pelo algoritmo anônimo e, a partir dela, o Carnaval e Corpus Christi. Confira com `mix vapor.finance calendar`.
18. [G] Mostre que, se o preço de uma *call* está abaixo de $\max(Se^{-qT} - Ke^{-rT}, 0)$, existe uma arbitragem. Escreva o portfólio.
19. [M] Demonstre a 5.1 em detalhe. Para quais pares (a, m) com $a \cdot (m - 1) < 2^{23}$ o truque falharia sem os seletores de correção?
20. [M] Escreva um sinal que passe na invariância de prefixo com oito cortes mas use informação futura. Quantos cortes seriam necessários para pegá-lo?
21. [M] Para o binomial de um período com $S_0 = 100$, $S_u = 120$, $S_d = 90$, título que paga 1 cotado a 0,95 e uma *call* $K = 100$ cotada a 12, decida se há arbitragem e dê o certificado (portfólio ou ψ). Confira com `mix vapor.finance arbitrage`.

22. [M] O juiz do livro tem duas camadas: reexecução e invariantes. Construa um defeito que só a reexecução pegaria e outro que só os invariantes pegariam.
23. [M] Calcule a razão de ramificação de um Hawkes exponencial e mostre que o processo é estacionário se e só se $\alpha/\beta < 1$. Simule com $\alpha/\beta = 0,8$ (exemplo “Hawkes quente”) e observe o teste de reescala.
24. [D] É possível compilar o motor de casamento como programa canônico do vapor (sem ramos dependentes de dados no caminho vetorial) de modo que o diário seja produzido no worker e o juiz rode em paralelo, fora do caminho crítico? Que estrutura de dados para o livro permitiria isso?
25. [D] A invariância de prefixo certifica a ausência de antecipação *no código*. Proponha um certificado análogo para a ausência de antecipação *nos dados* (por exemplo, séries revisadas depois da data, ou sobrevivência).

AVALIAÇÃO

Referentes ao Capítulo 7.

26. [G] Classifique cada linha da Tabela 11 quanto ao critério (igualdade bit a bit ou tolerância) e justifique.
27. [M] Repita a verificação de Avellaneda–Stoikov com 20 sementes e relate a distribuição da razão $\sigma(P\&L)$ estoque/simétrica. O limiar de 0,6 é adequado?
28. [D] Desenhe um protocolo de avaliação que reduza a ameaça de validade interna “mesmo autor do sistema e dos testes” sem depender de uma equipe independente.

APÊNDICE C – GLOSSÁRIO

Adjunta (identidade) $\langle Wx, v \rangle = \langle x, W^\top v \rangle$; usada no degrau 3 para pegar erros de *layout* e transposição.

Árbitro O componente que escolhe o substrato pelo menor tempo previsto pelo *roofline* de três tetos.

Arbitragem Portfólio que não custa nada (ou rende hoje), nunca perde e às vezes ganha.

Backtest Simulação de uma estratégia sobre dados históricos.

BEAM A máquina virtual do Erlang/OTP; no vapor, o plano de controle.

Bootstrap (de curva) Construção sequencial de uma curva de desconto, um nó por instrumento, cada um reprecificado com os anteriores fixos.

CBOR determinístico Codificação binária (RFC 8949 §4.2) com uma única representação por valor; usada em tudo que é assinado.

Certificado Objeto que acompanha uma resposta e permite conferi-la com um procedimento mais simples que o que a produziu.

Controle Valor que uma implementação quebrada, ingênua ou trapaceira produziria; uma verificação só vale se o controle for recusado.

Cut sweep Estratégia do compilador que fecha uma região fundida quando faltam registradores, em vez de derramar.

DRUP Formato de refutação SAT por propagação unitária reversa, conferido por um verificador independente.

DSR Índice de Sharpe deflacionado: a probabilidade de o Sharpe verdadeiro exceder o máximo esperado de N tentativas sem habilidade.

Eclusa Fronteira de admissão por medição: de modelos, de substratos ou de documentos.

Envelope Cota rigorosa, calculada em racionais diádicos, dentro da qual qualquer substrato conforme tem de cair.

Fabric O processo que executa SPIR-V via Vulkan.

Farkas (lema de) Teorema da alternativa: de dois sistemas lineares, exatamente um tem solução.

Fator de grupo (g) Número de registradores físicos que formam um registrador lógico; generaliza o LMUL do RVV.

- FIX** Protocolo tag=valor de mensagens de ordens e execuções.
- FOK, IOC, GTC** Validades de ordem: tudo ou nada imediato; imediato, o resto cancela; até cancelar.
- Hawkes (processo)** Processo de chegadas em que cada evento aumenta temporariamente a intensidade dos seguintes.
- Invariância de prefixo** Igualdade do sinal calculado com a história inteira e com a história truncada; certificado de ausência de antecipação.
- ITCH** Protocolo binário de dados de mercado da Nasdaq.
- KIR** A representação intermediária de kernels do *vapor*, sobre registradores virtuais tipados.
- Kupiec (teste de)** Teste de razão de verossimilhança da proporção de exceções de um VaR.
- Linear scan** Algoritmo de alocação de registradores em tempo quase linear sobre intervalos de vida.
- Merkle (árvore de)** Árvore de *hashes* cuja raiz resume um conjunto e permite provas de inclusão logarítmicas.
- Oráculo** Implementação exata da semântica (no núcleo) ou referência externa (nos testes).
- PBO** Probabilidade de sobreajuste do *backtest*, por validação cruzada combinatória.
- Política canônica** Semântica numérica fixa bit a bit; oposta à política *:fast*.
- Preços de estado** Preços positivos de um pagamento unitário em cada estado; a sua existência equivale à ausência de arbitragem.
- Proponente/verificador** Padrão em que uma busca (ou pessoa, ou modelo) propõe e um verificador independente decide.
- Roofline** Modelo de desempenho em que o tempo é limitado pelo maior dentre cálculo, banda e (no *vapor*) emissão de instruções.
- SPIR-V** Representação intermediária de *shaders* do Vulkan.
- Spill** Escrita de um valor na memória por falta de registrador.
- Substrato** Lugar onde um programa executa: uma ISA nativa, o interpretador, uma GPU ou o oráculo.
- SVI** Parametrização da variância total implícita de uma fatia de vencimento.
- VaR** Perda que só é excedida com probabilidade dada (1% para o VaR de 99%).
- W^X** Política em que uma página de memória é gravável ou executável, nunca as duas.
- Worker** O processo que executa código de CPU gerado, nativo ou interpretado.